名古屋大学情報学部
コンパイラ (2026 年度版)

結縁 祥治

注意

本資料の使用は，名古屋大学情報学部コンピュータ科学科３年生対象講義「コンパイラ」に限る．

謝辞

本資料は以下をもとにして書かれている．

大山口通夫・三橋一郎
コンパイラの理論と作成技法
ISBN978-4-7819-1251-6
サイエンス社

2024 年 10 月現在，当該書籍が絶版となり重版の見込みがたたないことから，本講義の実施にあたり適宜内容の引用・参照をご了承いただいた上記書籍の著者 (大山口先生，三橋先生) に感謝いたします．

(本資料の内容は元の教科書に含まれないトピックを含みます．)

目 次

第 1 章　序論 **7**
1.1　プログラミング言語と言語処理系 7
1.2　プログラミング言語の種類 10
1.3　中間言語 11
1.4　コンパイラ構成に必要な諸定義 12
1.4.1　語と集合演算 12
1.4.2　グラフと木 12

第 2 章　コンパイラの概要 **17**
2.1　コンパイラの構造 17
2.1.1　コンパイラにおけるパス 18
2.2　字句解析の概要 19
2.3　構文解析の概要 19
2.4　中間コード生成の概要 21
2.5　コード最適化の概要 21
2.6　目的コード生成の概要 22

第 3 章　字句解析 **25**
3.1　正規表現 25
3.2　有限オートマトン 26
3.3　字句解析の仕組み 27
3.4　字句解析器の生成 29
3.4.1　正規表現の記法 29
3.4.2　字句解析器生成ツール 30

第 4 章　構文解析 **33**
4.1　構文解析の基礎 33
4.1.1　BNF 定義と CF 文法 33
4.1.2　解析木 35
4.1.3　文法の曖昧性 38
4.1.4　解析木の表現 40
4.1.5　First と Follow 42
4.2　下降型構文解析 44
4.2.1　左再帰性 44

4.2.2 左括り出し . . . 46
4.2.3 LL 構文解析 . . . 46
4.2.4 拡張 CF 文法と ELL(1) 文法 . . . 48
4.2.5 Javacc(JJTree) による降下型構文解析プログラムの生成 . . . 49
4.2.6 例 . . . 50
4.3 上昇型構文解析 . . . 52
4.3.1 プッシュダウン変換機械 . . . 53
4.3.2 LR 構文解析 . . . 54
4.3.3 構文解析手法の能力比較 . . . 68
4.3.4 演算子順位法 . . . 69
4.4 構文エラー表示と誤り回復 . . . 71
4.5 YACC による構文解析プログラム生成 . . . 72

第 5 章 意味解析 75
5.1 記号表 . . . 75
5.1.1 記号表の実現 . . . 77
5.2 型チェック . . . 80
5.2.1 型定義と型チェックアルゴリズム . . . 81
5.2.2 多相型と型のオーバーロード . . . 82

第 6 章 中間コード生成 85
6.1 概要 . . . 85
6.1.1 抽象構文と抽象構文木 . . . 85
6.1.2 逆ポーランド記法 . . . 86
6.1.3 3 番地コード . . . 86
6.2 実行環境 . . . 88
6.2.1 駆動レコードの構成 . . . 88
6.3 構文主導型変換 . . . 91
6.3.1 構文主導型変換の基礎 . . . 91
6.3.2 上昇型構文主導型変換 . . . 94
6.3.3 下降型構文主導型変換 . . . 95
6.3.4 構文主導定義による意味定義 . . . 95
6.3.5 コード生成の実際 . . . 97
6.3.6 ３番地コード生成規則の挿入 . . . 99
6.3.7 例 . . . 101

第 7 章 コード最適化 103
7.1 基本ブロックとフローグラフ . . . 103
7.2 局所最適化 . . . 104
7.2.1 局所最適化の例 . . . 104
7.2.2 基本ブロックの DAG 表現 . . . 105

7.3 大域最適化 107
7.3.1 意味等価な変換 107
7.3.2 データフロー解析 112
7.4 静的単一代入形式と最適化 115
7.4.1 例 115
7.4.2 φ 関数 116
7.4.3 SSA 形式への変換 122
7.4.4 SSA 上での最適化 123
7.4.5 φ 関数を持たない SSA 形式への変換 125

第 8 章 目的コード生成 **129**
8.1 算術式のコード生成 129
8.2 グラフ彩色によるレジスタ割当 130
8.2.1 レジスタ干渉グラフ 132
8.2.2 SSA に対するレジスタ割当 132

付 録 A VM コードの仕様 **135**

第1章　序論

1.1　プログラミング言語と言語処理系

計算機はあらかじめ決まった手順で計算を実行して答えを得る．手順のことを**プログラム** (program) とよび，プログラムを作成することをプログラミングという．計算機のプログラムは決まった書き方で記述する必要があり，**プログラミング言語** (programming language) によって定める．プログラムは明確に計算機の手順を指定する必要があるため，解釈に曖昧さのないことが望ましい．このため，プログラミング言語は，曖昧性があったり文脈に依存したりする解釈が難しい人間が使う言語（自然言語）でなく，より機械的に解釈ができる**形式言語**として定められる．

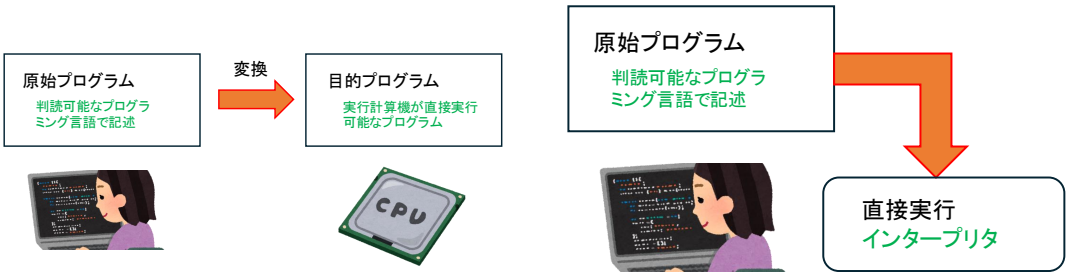
形式言語であっても自然言語にうまく対応付けることで，人間が手順をそのまま指示することができる．このような人間がそのまま読んで解釈が可能なプログラミング言語は一般に**高級言語**と呼ばれる．高級言語としては BASIC, C, Java, Python などがよく用いられる．計算の手順を適切に示すためには，対象とする計算の目的や計算機の構成に沿って記述することが自然であるため，計算のいろいろな側面に注目した高級言語が提案されている．例えば，C 言語はオペレーティングシステムを記述するために開発され，Java はオブジェクト指向という計算の捉え方に沿ってプログラムを記述する．ocaml では，プログラムは入力から出力を求める関数であるという原則に基づいて記述する．

これに対して，計算機のハードウェアを直接操作する手順は，**低級言語**と呼ばれる言語で記述され，ハードウェアごとに定められる．低級言語の中で直接，ハードウェアを操作する言語（命令列）のことを**機械語** (machine language) という．機械語は 0,1 の並びであり，計算機のアーキテクチャに応じた記述であるため，普通は人間が理解することは難しい．機械語の命令にわかりやすい名前をつけたり，メモリ領域に対して，ラベルをつけたりする機能をもたせて多少機械語のプログラムに意味を記述できるようにした**アセンブリ言語** (assembly language) がある．アセンブリ言語はハードウェア毎に存在する．アセンブリ言語のプログラムを機械語に変換するコンパイラは**アセンブラ** (assembler) と呼ばれる．

求める計算を計算機に効率的に実行させるためには，計算を実現するプログラムをプログラミング言語で記述する必要がある．人間が手順を指定するためには高級言語で記述するのが都合よい．これに対して，計算機が手順に従って計算を行うためには低級言語で操作を指定する必要がある．高級言語をなんらかの形で計算機で実行するプログラムのことを**言語処理系** (language processor) という．

計算の意味を保存したまま，高級言語から低級言語に変換すれば，効率的にプログラムを作成して実行できる．この変換を行うことを**コンパイル**（compile) といい，変換を自動的に行う計算機プログラムを**コンパイラ** (compiler) という．変換される前のプログラムを**原始プログラム** (source program)，変換の結果得られるプログラムを**目的プログラム** (object program あるいは target

program) という．人間 (プログラマー) は自然言語（主に英語）に近い形式で書かれた解釈可能な原始プログラムで手順を記述し，コンパイラで計算機ハードウェアが直接実行可能な低級言語のプログラムに変換することで，目的となるプログラムを実行することができる．コンパイラは原始プログラムを全部読み込み，目的プログラムに変換するのが普通である．



(a) コンパイラ (b) インタープリタ

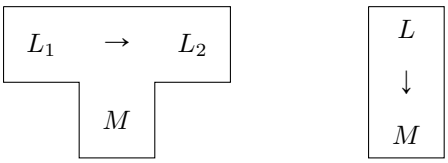
図 1.1: プログラミング言語処理系

これに対して，原始プログラムの手順としての解釈を逐次的に直接的に計算機で実行することによって原始プログラムを計算機で実行することもできる．このような実行プログラムを**インタープリタ** (interpreter) という．インタープリタは，原始プログラムの意味を直接的に実行する．インタープリタは原始プログラムを低級言語に変換することなく，直接的に解釈実行する言語処理系である．

コンパイラとインタープリタ コンパイラとインタープリタはいずれも原始プログラムを実行するための言語処理系である．コンパイラによる実行は原始プログラムを目的プログラムに変換して目的プログラムを実行する．これに対してインタープリタは原始プログラムを直接実行するようなプログラムである．インタープリタは原始プログラムを逐次的に実行するため，一般に実行は低速であるが，原始プログラムを変更すればすぐに変更が実行に反映される．これに対して，コンパイラによって変換された目的プログラムは高速に実行される．コンパイラによる実行では，原始プログラムを変更した場合には，再度原始プログラム全体を目的プログラムに変換してから実行し直す必要がある．

コンパイラによる変換においては，計算の意味を保存する必要がある．つまり，原始プログラムの計算と目的プログラムの計算は同じでなければならない．しかし，原始プログラムの計算は高級プログラミング言語で定義され，目的プログラムの計算は計算機の機械的な振る舞いで定義される．コンパイラによるプログラミングではこの**計算の意味**を保存することが必須条件である．したがって，コンパイラではプログラムが持つ意味をなんらかの形で正確に表現する必要がある．この意味表現に基づいて意味を保ちながらより効率的な変換である最適化が実現できる．コンパイラを構成するためには，プログラミング言語とプログラムの意味について正確に定式化する必要が生じる．

目的プログラムへ変換するとき，原始プログラムの計算をより効率的に実行するための**最適化** (optimization) が行われる．最適化は，プログラムの実行意味を変えることなく，より短い命令の系列でできる限りメモリを使わずに実行できるように目的プログラムを生成する．最適化は，コン



(a) コンパイラ (b) インタープリタ

図 1.2: T 図形

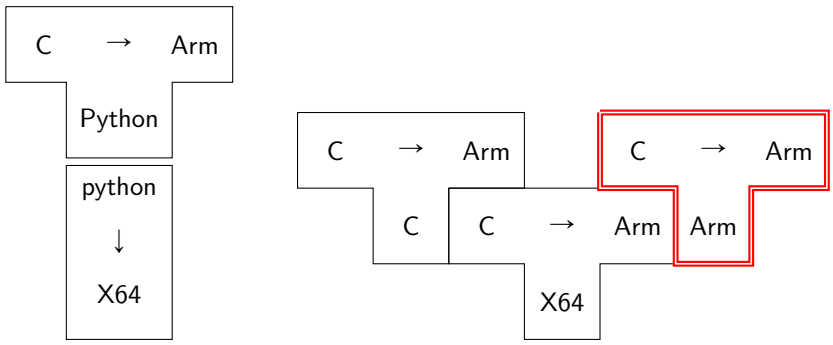
パイル技術の中核をなす概念であり，プログラミング言語を設計する観点からも重要である．

プログラムに不具合がある場合，インタープリタでは不具合は直接的に実行に反映されるが，コンパイラでは最適化の影響で不具合を特定することが難しい場合もある．プログラムの規模が大きい場合，インタープリタによる逐次的な解釈実行は現実的ではないが，小さなプログラムであれば柔軟に変更できる．Java などの言語処理系では，**JIT(Just-in-Time) コンパイラ**が導入され，インタープリタに近い柔軟な実行と目的プログラムによる効率的な実行を組み合わせる言語処理系も存在する．これに対して，原始プログラムを事前に目的プログラムに変換するコンパイラは**AOT(Ahead-Of-Time) コンパイラ**と呼ばれる．

T 図式 言語処理系の構成を示す手法として，**T 図形**が用いられることがある．図 1.2 に T 図式を示す．(a) は言語 L_1 を L_2 に変換するコンパイラが言語 M で記述されていることを示し，(b) は言語 L を M で動作するインタープリタを示す．

例 1 (図 1.3(a)): C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラが Python で記述されているとする．X64 計算機で実行する Python インタープリタがあるとき，C 言語プログラムを X64 計算機で Arm の目的プログラムに変換することができる．

例 2 (図 1.3(b)): C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラが C 言語で記述されているとする．X64 計算機で動作する C 言語を X64 の目的プログラムに変換するコンパイラがあるとき，X64 計算機で C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラを構成することができる．さらにこのコンパイラを最初の T 図式に適用すると，Arm 計算機上で動作するコンパイラを構成することができる．



(a) 例 1　　(b) 例 2

図 1.3: 変換過程

1.2　プログラミング言語の種類

プログラミング言語は計算の手順をどういった側面に注目して記述するかによっていくつかの種類がある．

手続き型言語: 計算の手順を順番に沿って命令の列として記述するプログラミング言語として最も標準的な記述を与える．条件分岐，ループといった制御構造の他に，再帰呼び出し，データ局所化，データ型定義などプログラムの対象に応じた命令，データ型を持つ．

関数型言語: ML(Meta Language) をはじめとする関数定義に沿ってプログラムを記述する言語である．関数は，入出力の対応を直接的に表す数学のモデルであり，プログラムの最も基本的な意味を表現する．コンパイラは関数の定義に従った計算方法を構築して目的コードに変換する．関数型言語では変換する際に，関数の型の無矛盾性について厳密にチェックすることが多い．厳密な型チェックによって矛盾した定義による不具合の発生を抑えることができる．関数の合成に基づいて，小規模な関数を合成して全体の関数を構成するという考え方に基づく．

論理型言語: 論理における証明が計算に対応するというアイデアに基づいて，ホーン節の反駁によってプログラムを計算する．Prolog が代表的な論理型言語である．証明という概念に基づいていることから，不具合が発生すること＝証明ができず正しくない，という考え方で設計される．

オブジェクト指向言語: 計算の実行主体をオブジェクトとし，オブジェクトにメッセージを送ることで計算が進行するというモデルに基づく．Java がこのアプローチをとっている．オブジェクトという概念は他の概念とは直交したモデルであるので，他の種類の言語でもオブジェクトに基づいた記述を導入している．(Python や caml など) オブジェクト指向モデルによっ

て，データのフローを局所化でき，継承に基づいた合成的な記述が可能となる．Smalltalk が本格的なオブジェクト指向言語として提案された．

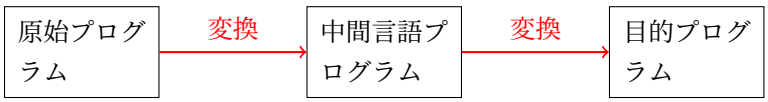
プログラミング言語は，構文 (syntax) の定義として文脈自由文法を用いることが一般的である．文脈自由文法の記法として，BNF(Backus-Naur Form) が用いられる．BNF は，文脈自由文法の生成規則を記述する．生成規則の $\rightarrow$ を ::= と記載すること，同じ左辺を持つ生成規則は | によって書くことなど，記法に違いがある．

BNF では右辺に $*$ を含めた正規表現を許す．$*$ が用いられた場合，拡張バッカス・ナウア形式 (EBNF) と呼ばれることが多い．文脈自由文法においてはメンバーシップ問題 $w \in L(G)$ は決定可能であることから，文脈自由文法で定義することにより，構文エラーは多項式時間 $\mathcal{O}(n^3)$ でチェック可能である[1]．

1.3 中間言語

コンパイラがプログラムを変換する際に一段階で変換する場合もあるが，段階を分けて変換していく場合が多い．中間的に変換される対象となる言語のことを**中間言語** (intermediate language) という．

中間言語としては，バイトコードと呼ばれるスタックマシンコードや多くのアーキテクチャに共通な特徴を持つ３番地コードなどが用いられる．中間言語はアーキテクチャに依存しない場合が多く，データフローなどプログラムそのものの性質を解析し，このレベルで最適化を行ったのち，アセンブリ言語に変換することによって，効率的に最適化を行うことができる．Java 言語処理系では，まず，バイトコードを目的プログラムとして変換し，バイトコードの実行状況に応じて，バイトコードをさらにコンパイルして，より高速に実行することが可能になる．



３番地コードは，命令コードと３つのメモリ番地を基本命令とする中間言語である．命令コードは直接機械語に対応しており，３つのメモリ番地のうち１つをデータの格納先，残り２つを計算のデータが収容されている番地を表す．３番地コードは，ハードウェアに依存しないプログラムの持つ制御フローとデータフローを表現することができる．この特徴から，計算機のアーキテクチャに依存しない基本的な最適化を適用することができる．

具体的な３番地コードについては，Java バイトコードや LLVM-IR などがある．

[1] CYK アルゴリズムと呼ばれる．

1.4　コンパイラ構成に必要な諸定義

1.4.1　語と集合演算

記号の集合をアルファベットと呼び，Σ や Γ で表す．Σ の要素を 2 個並べることによって得られる集合を $\Sigma \cdot \Sigma$ と書く．例えば，$\Sigma = \{a, b\}$ のとき $\Sigma \cdot \Sigma = \{aa, ab, ba, bb\}$ である．

一般に記号の集合 Σ, Γ に対して，**積**（あるいは連接）を $\Sigma \cdot \Gamma = \{xy \mid x \in \Sigma, y \in \Gamma\}$ と定義する．ここで，xy を x と y の**連接** (concatenation) という．記号の連接を語 (word) と呼ぶ．n 個の記号の連接 w の長さを $|w| = n$ と書く．長さ 0 の語を ε と書く．ε に対する語 w の連接は w そのものである．すなわち，$w \cdot \varepsilon = \varepsilon \cdot w = w$ である．

集合 Σ の要素を i 個連続して連接することによって得られる集合を Σ^i と書き，以下のように定められる．

$$\Sigma^i = \begin{cases} \{\varepsilon\} & i = 0 \text{ のとき} \\ \Sigma \cdot \Sigma^{i-1} & i \geqslant 1 \text{ のとき} \end{cases}$$

$\cap, \cup, \setminus$ は，2 つの集合 Σ, Γ に対して以下のようにそれぞれ積集合，和集合，差集合を構成する．

$$\Sigma \cap \Gamma = \{x \mid x \in \Sigma \text{ かつ } x \in \Gamma\} \qquad \Sigma \cup \Gamma = \{x \mid x \in \Sigma \text{ または } x \in \Gamma\}$$
$$\Sigma \setminus \Gamma = \{x \mid x \in \Sigma \text{ かつ } x \notin \Gamma\}$$

Σ の 1 個以上の連接のすべての集合を Σ^+ と書く．

$$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma \cup \Sigma^2 \cup \cdots \cup \Sigma^n \cup \cdots$$

$\{\varepsilon\} \cup \Sigma^+$ を Σ の**閉包** (closure) あるいは**クリーネ閉包** (Kleene closure) と呼び，Σ^* と書く．

$u, v \in \Sigma^+$ に対して，$uw = v$ を満たす $w \in \Sigma^*$ が存在するならば，u は v の**接頭語** (prefix)，$wu = v$ を満たす $w \in \Sigma^*$ が存在するならば u は v の**接尾語** (postfix) であるという．ここで，$u \cdot \varepsilon = u$ および $\varepsilon \cdot u = u$ であることから，語 u は自身の接頭語でもあり，接尾語である．

空集合は $\varnothing$ と記述する．$\varnothing \cup \Sigma = \Sigma \cup \varnothing = \Sigma$, $\varnothing \cap \Sigma = \Sigma \cap \varnothing = \varnothing$ である．連接の定義から，任意の $L \subseteq \Sigma^*$ に対して，$\varnothing \cdot L = L \cdot \varnothing = \varnothing$, $\varnothing^* = \{\varepsilon\}$ である．

1.4.2　グラフと木

グラフ (graph) は**頂点** (vertex) の集合 V と**辺** (edge) の集合 $E \subseteq V^2$ の対 (V, E) である．辺 $e = (v_1, v_2)$ は，頂点 $v_1 \in V$ から $v_2 \in V$ への矢印 $v_1 \to v_2$ で表す．v_1 は e の始点，v_2 は e の終点と呼ばれる．辺の始点と終点を区別するとき，**有向グラフ** (directed graph) という．辺 (v_1, v_2) と (v_2, v_1) を同じ辺であるとするとき，**無向グラフ** (undirected graph) という．頂点 u から出る辺の本数を u の**次数** (degree) という．$I \subseteq V$ に対して，$I^2 \cap E = \varnothing$ であるとき，すなわち I に含まれる任意の頂点の間に辺が存在しないとき，I を**独立集合** (independent set) という．

有向グラフ $G = (V, E)$ において，辺 $(u, v) \in E$ に対して，u を v の**先行頂点** (predecessor)，v を u の**後続頂点** (successor) という．頂点の系列 $v_1, \cdots, v_k$ に対して，$(v_i, v_{i+1}) \in E$ が $1 \leqslant i \leqslant k-1$ に対して成り立つとき，グラフの**路** (path) という．また，頂点の系列 $v_1, \cdots, v_k$ に対して，$(v_i, v_{i+1}) \in E$

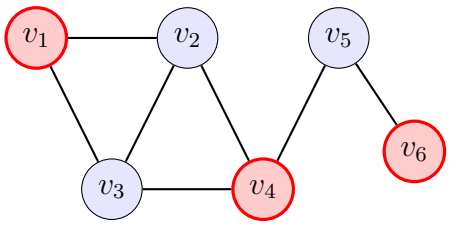
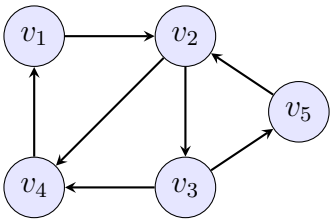


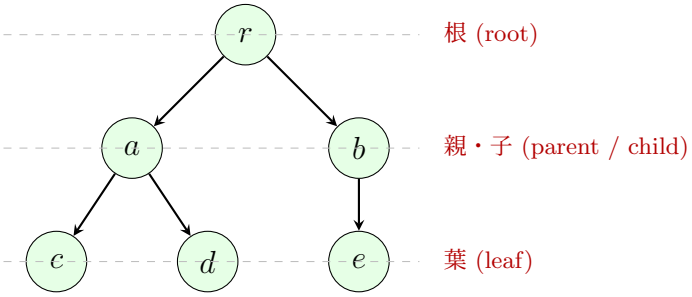
図 1.4: 無向グラフと独立集合の例

または (v_{i+1}, v_i) が $1 \leqslant i \leqslant k-1$ に対して成り立つとき，グラフの**半路** (semipath) という．任意の異なる頂点 $v_1, v_2 \in V$ に対して，v_1 から v_2 あるいは v_2 から v_1 に路が存在するとき, G は**強連結**であるといい，半路が存在するとき，G は**弱連結**であるという．

空でない頂点集合を持つ有向グラフ $G = (V, E)$ において，$v \in V$ で先行頂点を持たない頂点がただ一つ存在し，それ以外の頂点 $u \in V \backslash \{v\}$ については，ただ 1 つの先行頂点を持つとき G は**木** (tree) であるという．先行頂点を持たない頂点のことを**根** (root) という．木の条件を満たすグラフ T の根を $root(T)$ で表す．T の頂点のうち，後続頂点を持たない頂点を**葉** (leaf) という．木の条件を満たす有向グラフの頂点に対して，先行頂点を**親** (parent), 後続頂点を**子** (child) という．根は親頂点を持たず，葉は子頂点を持たない．



(a) 有向グラフの例



(b) 木の例

図 1.5: 有向グラフと木の例

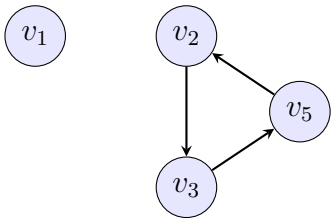
グラフ $G_s = (V_s, E_s)$ がグラフ $G = (V, E)$ の**部分グラフ**であるとは，$G_s = (V_s, E_s)$ とは，$V_s \subseteq V$, $E_s \subseteq E \cap V_s^2$ が成り立つことである．さらに，部分グラフ G_s が弱連結であるとき，G_s は，**弱連結部分グラフ**であるという．

木の条件を満たす有向グラフ T の弱結合部分グラフを T の**部分木**という．T の部分木の集合

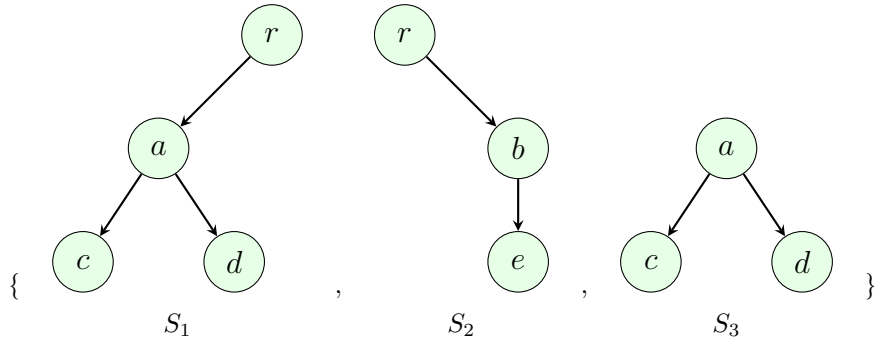
$\{S_1, \cdots, S_n\}$ の**交差グラフ (intersection graph)** は，以下のように定義される無向グラフ (V_g, E_g) である．

- $V_g = \{v_1, \cdots, v_n\}$．ここで，$v_i$ は S_i に対応する．
- $E_g = \{(v_i, v_j) \mid i \neq j, V(S_i) \cap V(S_j) \neq \varnothing\}$

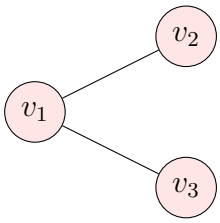
図 1.6 に，図 1.5 にあげたグラフの部分グラフ，部分木集合，および，部分木集合に対する交差グラフの例を示す．(a) については，１つの部分グラフである．部分グラフは必ずしも弱連結である必要はない．これに対して，部分木の場合は，木構造の条件を満たすために弱連結である必要がある．



(a) 部分グラフの例



(b) 部分木集合の例



(c) 交差グラフ

図 1.6: 図 1.5 の部分構造，交差グラフの例

無向グラフ (V, E) の頂点 $v \in V$ を**消去する操作**は，(V', E') というグラフを作成する操作とする．

- $V' = V \backslash \{v\}$, および
- $E' = (E \backslash E_v) \cup E_f(v)$

ここで，$E_v = \{(v_1, v_2) | v_1 = v \ or \ v_2 = v\}$, $E_f(v) = \{(v_{f1}, v_{f2}) | (v_{f1}, v), (v_{f2}, v) \in E, v_{f1} \neq v_{f2}, v_{f1} \neq v, v_{f2} \neq v\}$ とする．E_v は，削除する頂点につながっている辺であり，$E_f(v)$ は，異なる 2 辺 $e_1, e_2 \in E_v$ で v から到達する 2 つの頂点を結ぶ辺の集合である．このような辺が存在しない場合には，新たに辺を加える．すでに存在する場合はそのままとする．このようにして加えられる辺のことを **Fill-in 辺**と呼ぶ[2]．

図 1.7 において，v_1 を消去する場合は，$(v_1, v_2), (v_1, v_3)$ を消去し，(v_2, v_3) というグラフを作成する操作とする辺を追加する．v_5 を消去する場合は，すでに (v_2, v_4) に辺が存在するので，新たに辺を加えることはない．

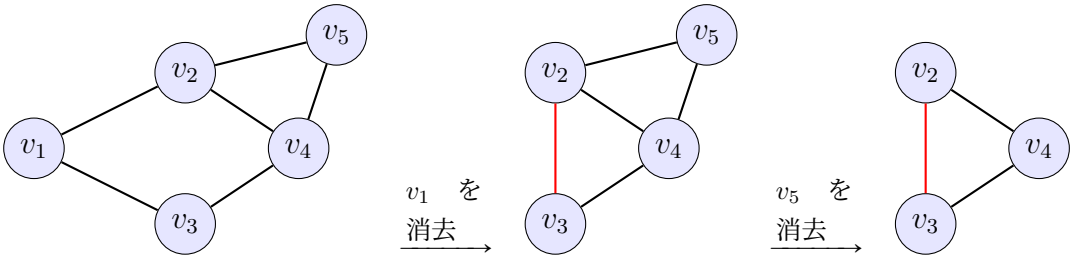


図 1.7: グラフ頂点の消去操作

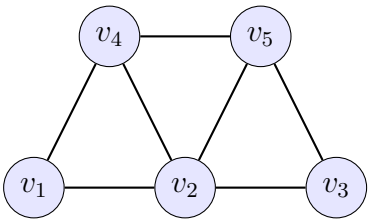
無向グラフ $G = (V, E)$ の閉路とは，相異なる頂点の系列 $(e_1, e_2, \ldots, e_k, e_1)$ $(k \geqslant 3)$ で，$(e_i, e_{i+1}) \in E$ となる辺が存在する．k を閉路の長さと呼ぶ．閉路上で隣接していない 2 頂点を結ぶ G の辺を**弦 (chord)** という．

G において $k \geqslant 4$ の任意の閉路 C に対して少なくとも 1 つの弦が存在するとき G は**弦グラフ**という．図 1.7 において，v_1 消去する前のグラフは，長さ 4 の閉路 $(v_1, v_2, v_4, v_3, v_1)$ に対して，弦が存在しないので，弦グラフではない．v_1 を消去して Fill-in 辺を追加したグラフは弦グラフである．

弦グラフ $G = (V, E)$ に対しては，以下の条件を満たす V 上の全順序が存在する．

> $v_1 < v_2 < \ldots < v_n$ のとき，$v_n, v_{n-1}, \ldots, v_1$ の順に V に消去動作を適用しても，Fill-in 辺として新たに辺が追加されない．

この順序を**完全消去順序 (Perfect Elimination Order,PEO)** という．弦グラフならば，かつそのときに限り，完全消去順序が存在することが知られている．



PEO: v_3, v_5, v_4, v_1, v_2
PEO でない: v_2, v_3, v_1, v_5, v_4

これらのグラフの概念は，主にレジスタ割り当ての際に用いる．

[2] このような Fill-in 辺を加えることをクリーク化と呼ぶこともある．

第2章　コンパイラの概要

2.1　コンパイラの構造

コンパイラはプログラマーが記述する原始プログラムを**フェーズ** (phase) と呼ばれるいくつかの段階を経て計算機が直接実行可能な 0,1 の系列である機械語で記述された目的プログラムへ変換する．目的プログラムは**バイナリコード**とも呼ばれる．

コンパイラの標準的な構造を図 2.1 に示す．

字句解析 (lexical analysis)　**字句解析**は，アルファベットの系列としての原始プログラムを入力として，プログラミング言語で定められる単語に切り分ける．プログラミング言語の多くは基本的な記述として，1 バイトの文字コードを用いる．一般的には ASCII コード (ANSI X3.4-1986) を用いる．プログラミング言語ごとに定められた予約語 (keyword)，識別子 (identifier)，定数 (constant)，文字列リテラル (string literal)，区切り記号 (delimiter) などを**トークン**として認識し，原始プログラムをトークン列に変換する．トークンは有限オートマトンによって定義される．

構文解析 (parsing)　**構文解析**は，字句解析の結果出力をプログラミング言語を定義する文脈自由文法に従ってプログラムの基本的意味を表現する**解析木** (parse tree) を構成する．解析木は構文木 (syntax tree) ともいう．解析木は，プログラムが構文的に持つ情報を表現する．文法に従わない入力を，文脈自由文法が生成しない文字列として検出する．文脈自由文法が生成する文字列い対して解析木を構成する．

中間コード生成 (intermediate code generation)　構文木はプログラミング言語が記述するプログラムの意味を表し，構文木をトラバースすることによってプログラムが記述する操作を抽出することができる．計算機ハードウェアに依存しない制御構造とデータフローを中間コード言語のプログラムで表す．中間コードとして，スタックマシンの命令であるバイトコードや 3 番地コードが用いられる．構文木から意味を表現する抽象構文木 (Abstract Syntax Tree) を構成して中間言語とする場合もある．

コード最適化 (code optimization)　解析木から生成される中間コードはプログラムの持つ意味を直接的に表す．しかし，重複した操作が繰り返し実行されるなどの冗長な操作を含む．計算の意味を保存しながらより効率的なコードを生成することを**コード最適化** (code optimization) という．コード最適化は，計算の意味を保存することが要求される．コード最適化は計算の意味を保たれる範囲でより効率的なコードを発見的に生成する．このため，最適化されたコードが必ずしも最も効率的であることは保証されない．最も効率的なコードを得ることは計算理論の結果から不可能であ

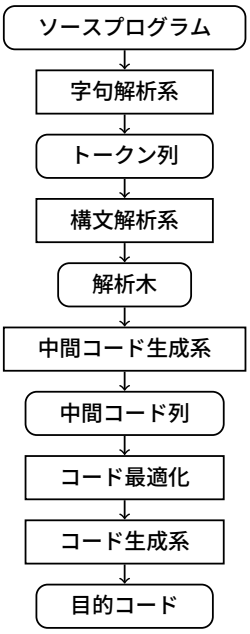


図 2.1: コンパイラの構造

ることが示される．コード最適化では，プログラムの振舞いをプログラムから決定することによってより高速な命令コードを生成する．ここでは，中間コードのおいて，変数が使われ始めるてどこまで用いられるかというったデータフロー解析や，静的単一代入 (SSA;Static Single Assignment) という解析が容易な形式のコードに変換することによってプログラムの実行の流れを追跡することで，より効率的なコードを生成する．このような解析はプログラムの意味を表現する上でも非常に有用であり，最適化のみならずプログラムの正しさを検証するためにも重要であり，非常に興味深い技法である．

目的コード生成 (object code generation)　目的コード生成では，最適化された中間コード言語のプログラムを機械語のプログラムに変換する．このフェーズでは，中間コードが表す制御フローとデータフローに従って目的コードを実行する計算機の資源を効率的に利用する機械語コードを生成する．計算機アーキテクチャに従って，演算を実施するためのデータの主記憶からの転送をできるだけ少なくする必要がある．演算のためのレジスタの数はプロうグラムで利用できるメモリの数は非常に少ないため，プログラムを解析することで効率駅にレジスタを利用する機械語プログラムを生成する必要がある．プログラムの操作に対してハードウェアで用意されている効率的な命令を選択する必要がある．

2.1.1　コンパイラにおけるパス

原始プログラムに含まれる表現を一度だけ読んで処理を行う部分を**１パス** (pass) という．コンパイラにおけるいくつかのフェーズは一つのパスにまとめることができ，例えば，字句解析と構文

解析は一つのフェーズにまとめることができる．目的コードを原始プログラムを一度だけ読んで生成するコンパイラのことを 1 パスコンパイラ (one-pass compiler) という．

2.2 字句解析の概要

字句解析は原始プログラムの文字列をプログラムとして意味のある単語にまとめてトークンの系列として出力する．プログラミング言語で用いられるトークンの種類は以下のようなものがある．

- **予約語 (keyword)**: プログラミング言語によって特別な意味を持つ単語．条件分岐を構成する，`if`, `then`, `else` や，繰返し構造を構成する `while`, `do` などのほかに，変数に対する組み込みのデータ型を表す `int`, `char` がある．これらの語はプログラミング言語が決まった意味で用いるために特に区別して認識する．
- **識別子 (identifier)**: プログラムで用いる変数名，関数名などに用いる文字系列である．通常，英数字と一部の記号の系列で構成される．
- **定数 (constant)**: 先頭の符号と数字および小数点で構成される数 (例：`-123.4`) や浮動小数点表現 (例：`3.14E1`)．一つの文字`'A'` も定数として認識する．
- **文字列リテラル (string literal)**: ダブル引用`"`で囲った文字の並び．`"Hello World"`のように空白も含めることも可能．
- **区切り子 (delimiter)**: プログラムの構文要素を区切る記号．セミコロン `;` は逐次に実行する命令の区切りを表す．このほか，並びの区切りを表すカンマ `,` や括弧 `()`, {}, `[]` がある．数式，論理式において数字や記号を区切る記号`+`, `-`, `&&`, `=`, `>=`がある．

トークンは，正規表現を用いて定義することができる．このため，字句解析では，トークンを受理する有限オートマトンを構成し，有限オートマトンの受理系列をトークンとして出力する．

2.3 構文解析の概要

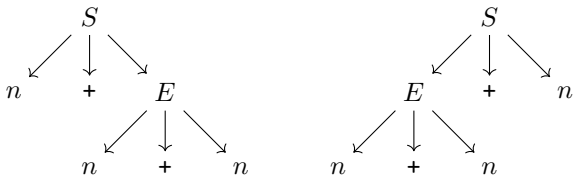
一般にプログラミング言語は文脈自由言語として定義される．プログラミング言語では文脈自由文法 G が生成する文字列の集合として定義される．入力したソースプログラムはトークン列に変換され，その語 w が G の生成する言語に属するかどうか $w \in L(G)$ を判定する．

ソースプログラムに対する導出の過程は生成規則の左辺を親，右辺を子とした木構造で表すことができる．この木構造は，プログラムの基本的な意味を表現する．例えば，以下のように文脈自由文法 G を与える．

$$G = \langle \{+, n\}, \{S, E\}, \left\{ \begin{array}{l} S \to E \\ E \to E{+}E \\ E \to n \end{array} \right\}, S \rangle$$

このとき，n+n+$n \in L(G)$ である．これは，$S \Rightarrow E$+$E \Rightarrow n$+$E \Rightarrow n$+E+$E \Rightarrow n$+n+$E \Rightarrow n$+n+n という導出系列が存在するためである．ここで n は自然数に対するトークンであり，実際には 1,12,30 などの数字を字句解析した結果である．

この導出に対する解析木は図 2.2(a) のようになる．



(a) 解析木 1　　(b) 解析木 2

図 2.2: 導出に対する解析木

n+n+n には，別の導出 $S \Rightarrow E + E \Rightarrow E$+$n \Rightarrow E$+$E$+$n \Rightarrow E$+$n$+$n \Rightarrow n$+$n$+$n$ も存在する．この導出に対応する解析木は (b) のようになる．この導出は E+E において，右の E を n に置き換えてから，左の E を $E + E$ に置き換えている．(a) では $E + E$ の左の E を n に置き換えてから，右の E を $E + E$ に置き換えている．(a) の解釈では，+ を加算とすると，右の 2 つの n を足してから左の n を加算する．(b) の解釈では，左の 2 つの n を足してから右の n を加算する．

$w \notin L(G)$ であれば，w は文法エラーを含む．通常の言語処理系の構文解析では，単にエラーであることを通知するだけでなく，部分的な構文木を作成して，エラーを起こしている理由をプログラムに提示する．例えば，n++$n \notin L(G)$ である．最初の + に対して木を構成しようとすると E+E の右側の E を + に置き換えることができないため，エラーが生じ，+ の右辺が間違っているといったエラーメッセージが出力される．

コンパイラ構成の観点からプログラミング言語の構文解析に対してはより実用的な要求がある．解析木はプログラムの意味に対応するため，曖昧でない文法が必要になる．曖昧な文法で定義する場合は，どの解析木を導出の結果とするかについてあらかじめ決めておく必要がある．上の文法 G はあいまいで n+n+n に対して図 2.2 の 2 つの解析木が存在する．加算演算は結合的であるので，どちらの木であっても答えは変わらない．しかし，+ を - に置き換えて減算とすると，減算は結合的でないので答えが定まらない．このため，通常は解析木 (b) のように解釈することが決められている．`3-2-1` は通常 0 と評価される．解析木 (a) のように解釈すると 2 と評価される．

文脈自由文法 G に対して，$w \in L(G)$ は決定可能な問題であり，任意の文脈自由文法において入力の長さに対して $\mathcal{O}(n^3)$ で決定することができるアルゴリズムが知られている[1]．コンパイラの構成上，より早い構文解析が必要であり，文脈自由文法のクラスを限定してより高速な構文解析アルゴリズムに合わせてプログラミング言語を定義する文脈自由文法を設計するのが普通である．決定的に構文解析するための文法のクラスが定義される．

[1] CYK(Cook-Young-Kasami) のアルゴリズムとして知られている

2.4 中間コード生成の概要

中間コードは原始プログラムが持つ基本的な振舞いを命令の系列として表現する．中間コードは原始プログラムの手順を表す実行制御フローとデータに対する演算およびメモリの更新を表すデータフローで構成される．中間言語として，**逆ポーランド記法** (postfix Polish notation あるいは reverse Polish notation)，**抽象構文木** (abstract syntax tree)，**３番地コード** (three-address code)，バイトコードなどが用いられる．

中間言語が表す振舞いは，プログラムが表す計算手順を標準的な命令単位で並べた系列であり，計算機ハードウェアから独立している．中間言語に一旦変換することで，目的プログラムが持つ本質的な動作意味を単純に表現することができる．このことから異なるアーキテクチャの目的コードは中間コードを変換して生成することができ，移植性が高くなる．

中間言語プログラムでは，変数は**記号表**を作成して実際に割り当てるメモリ番地に変換する．中間言語プログラムの実行環境では，**駆動レコード**とよばれるスタック構造によって，変数の値を格納する番地を決定する．逆ポーランド記法やバイトコードではデータスタックによってメモリの格納番地を決定する．

このフェーズでは変数の格納番地の決定を行うとともに，データ型など解析木に対してより詳細な制約をチェックする**意味解析**を行う．例えば，`x+y` という式に対して，x と y が同じ型であるかどうか，違う型であっても加算演算が可能かどうかといった**型チェック**を行う．

2.5 コード最適化の概要

コード最適化 (code optimization) は計算の意味を保持しながらより高速で省メモリの命令系列を導くことである．コード最適化は中間言語プログラムに対して行われるものと目的プログラムに対して行われるものがある．コード最適化は命令を制御が分割したり合流したりしない**基本ブロック**に対して行う**局所最適化** (local optimization)(あるいは**覗き穴最適化** (peep hole optimization) とも呼ばれる)，基本ブロックより広い範囲で行う**大域最適化**がある．その他，数値計算などにおいて最も効果が高い繰り返しループに着目して行う**ループ最適化**がある．一つの手続き範囲 (intra-procedure) で行う最適化，さらに，手続きをまたいで行う (inter-procedure) 最適化がある．最適化は範囲が広がるほど，意味を変えずに効率化することが難しくなる．最適化によって，原始プログラムと目的プログラムの対応をとることが難しくなるので，原始プログラムをデバッグする場合は，最適化を行わない場合が多い．

例えば，図 2.3(a) の中間言語プログラムの断片に対して，同じ計算が繰り返されるているため，(b) のように最適化することができる．(a), (b) は変数に同じ値を与えると各変数に同じ値を返すので，計算の意味を保存する．

```
x=1
z=x+y
w=x+y
u=z*4
v=w*4
```

(a) 最適化前

```
x=1
z=1+y
w=z
u=z*4
v=u
```

(b) 最適化後

図 2.3: 簡単な最適化の例

大域的な最適化においては**データフロー解析**あるいは**静的単一代入形式 (SSA, Static Single Assignment)** などの方法が用いられる．静的単一代入は，変数への代入が 1 回に制限される．このため，変数の上書きがなく，変数名[2]が同じであれば，代入されている値が同じであることが保証される．この性質を用いて柔軟にデータフロー解析をおなうことが可能になる．SSA は最近のコンパイラではよく用いられる中間言語形式である．

2.6　目的コード生成の概要

中間言語コードから実際に実行対象となる計算機ハードウェアが直接実行できる機械語を生成する．目的コード生成では中間言語プログラムの操作をハードウェアの適切な命令に置き換えることと中間コードのメモリをデータ演算を行う際に使用するレジスタに対応させる．ここでは，レジスタの値をできるだけ変更せず，メモリとレジスタの転送を少なくする**レジスタ割り当て**を行う必要がある．

3 番地コードの並び

```
t1=i2-i3
t2=i1+t1
```

を機械語に対応するアセンブリコードにそのまま変換すると以下のようになる．

```
Load   i2   } t1=i2-i3
Sub    i3   }
Store  t1   }
Load   i1   } t2=i2+t1
Add    t1   }
Store  t2   }
```

ここで，2 つめの 3 番地コードを `t2=t1+i1` とおきかえてもよい．すると，最初の命令で計算した `t1` をすぐにメモリに格納する必要はなく，以下のように命令を一つ省略できる．

[2]例えば変数 x への代入先が複数あれば，x_1, x_2 のように名前に添字をつける．

```
Load   i2
Sub    i3
Store  t1
Add    i1
Store  t2
```

さらに t1 がこのあと参照されないことがわかればさらに命令を省略できる．

```
Load   i2
Sub    i3
Add    i1
Store  t2
```

変数の参照と更新に着目すれば，ハードウェアで使用するレジスタの値とメモリとの転送命令 Load, Store を減らして高速化することが可能である．与えられたプログラムに対してレジスタ割り当てを最適にする問題は，有限の組み合わせに対してプログラムを解析することで可解である．しかし，そのアルゴリズムの時間計算量は NP 完全であることが知られているため，レジスタ割り当てによって実行時間を短縮できる効果よりも割当の組み合わせを計算する時間の方が長くなってしまうことが考えられる．このため，実際のコンパイラではより計算量の小さな近似アルゴリズムが用いられる．

SSA 形式では，支配木という木構造としてデータのフロー関係を表すことができる．各変数が有向な範囲は支配木の部分木として表すことによって，その部分木の交差グラフによって各時点において同時に存在しうる変数の組み合わせが示される．このグラフ構造から効率的なレジスタ割り当てが可能である．

第3章　字句解析

字句解析は，文字記号の系列である入力ファイルから，プログラムの構文を解析するために不要な文字を取り除いて，意味のある単語 (トークン,token) の系列に変換する．単語は，正規言語として定義され，切り分けは有限状態オートマトンを用いる．

3.1　正規表現

正規表現は指定された語の集合（言語）を指定する．コンパイラにおいては，トークンの種類を正確に指定するために用いる．予約語，識別子，定数，文字列リテラル，区切り子を指定する正規表現を決める．

識別子 (identifier) として認められる文字系列は,「非数字または被数字の後に非数字か数字を複数個連接した系列」として定義される．数字文字 Dec, 非数字文字 NDec, 識別子は ID は次の正規表現で定義することができる．

$$
\begin{aligned}
\mathsf{Dec} &::= \mathtt{0}|\mathtt{1}\cdots|\mathtt{9} \\
\mathsf{NDec} &::= \mathtt{_}|\mathtt{a}|\cdots|\mathtt{z}|\mathtt{A}|\cdots|\mathtt{Z} \\
\mathsf{ID} &::= \mathsf{NDec}(\mathsf{NDec}\,|\,\mathsf{Dec})^*
\end{aligned}
$$

10 進数定数 Decimal は以下のように定義できる．

$$
\begin{aligned}
\mathsf{NZDec} &::= \mathtt{1}|\cdots|\mathtt{9} \\
\mathsf{Decimal} &::= \mathsf{NZDec}\,\mathsf{Dec}^*
\end{aligned}
$$

正規表現は次のように定義される．

定義 3.1. 集合 Σ 上の**正規表現**は以下の再帰的定義によって与えられる表現である．

(1) ε, $\varnothing$ は正規表現である．

(2) Σ の要素 $a \in \Sigma$ は正規表現である．

(3) R_1, R_2 が正規表現であるとき，$R_1|R_2$, $R_1\,R_2$ は正規表現である．

(4) R が正規表現であるとき，R^* は正規表現である．

「再帰的定義」は，自らの定義の中に自身が含まれている定義である．上の定義では，(3),(4) において，正規表現の定義に正規表現自体が使われている．ここでは，定義しようとしている表現よりも短い表現が定義の中で使われているため，うまく定義されていることに注意しよう．例え

ば，$\Sigma - \{a, b\}$ に対して $(ab)|(b^*)$ が Σ 上の正規表現であるためには，(3) から ab と b^* が正規表現でなければならない．ab が正規表現であるためには，(3) から a,b が正規表現でなければならない．$a, b \in \Sigma$ なので，a, b は (2) により正規表現である．b^* が正規表現であるためには b が正規表現でなければならないが，$b \in \Sigma$ なので正規表現である．したがって，$a|(b^*)$ は正規表現である．括弧はまとまりを表すために適宜用いる．演算の結合の順番は強い順に $*$, 連接,$|$ とする．$ab|b^*$ は $(ab)|(b^*)$ を表す．

定義 3.2. Σ 上の正規表現 R に対して，R が表す系列の集合 $[\![R]\!] \subseteq \Sigma^*$ を以下のように再帰的に定める．

(1) $[\![\varepsilon]\!] = \{\varepsilon\}$, $[\![\varnothing]\!] = \varnothing$

(2) $a \in \Sigma$ に対して $[\![a]\!] = \{a\}$

(3) $[\![R_1\, R_2]\!] = [\![R_1]\!] \cdot [\![R_2]\!]$, $[\![R_1 \mid R_2]\!] = [\![R_1]\!] \cup [\![R_2]\!]$

(4) $[\![R^*]\!] = [\![R]\!]^*$

ここで右辺に現れる $\cdot$,$*$ は *1.4* 節で定義した集合に対する連接と閉包を表す．

従って，$ab|b^*$ という正規表現が表す Σ の系列の集合は以下のようになる．

$$\begin{aligned} [\![ab|b^*]\!] &= [\![ab]\!] \cup [\![b^*]\!] \\ &= [\![a]\!][\![b]\!] \cup [\![b]\!]^* \\ &= (\{a\} \cdot \{b\}) \cup \{b\}^* \\ &= \{ab\} \cup \{\varepsilon, b, bb, bbb, \cdots\} \end{aligned}$$

プログラミング言語のトークンは正規表現で定義される．

3.2　有限オートマトン

正規表現によって表現される文字列を認識するメカニズムは有限オートマトンである．有限オートマトンは，有限の状態を持つ状態遷移機械であり，ある状態 q_1 から次の状態 q_2 へ遷移するときに Σ の要素 a を読む．この遷移を $q_1 \xrightarrow{a} q_2$ と書く．初期状態から，受理状態に至る遷移の系列で受理する文字系列がオートマトンが受理する語である．q_0 が初期状態で，$q_0 \xrightarrow{a_1} q_1$, $q_1 \xrightarrow{a_2} q_2, \cdots$, $q_{n_1} \xrightarrow{a_n} q_n$ という遷移があり，q_n が受理状態であるとき，語 $w = a_1 a_2 \cdots a_n$ をオートマトンが受理するという．

定義 3.3. 有限オートマトンは以下の 5 項組で定義される．$\langle Q, \Sigma, \delta, q_0, F \rangle$ ここで，Q は状態の有限集合，δ は遷移関係，$q_0 \in Q$ は初期状態，$F \subseteq Q$ は受理状態集合である．

δ は 3 つ組 (q, a, q') の集合であり，q から a を読んで q' に遷移することを示す．遷移関係は $(q, a, q') \in \delta$ を $q \xrightarrow{a} q'$ のように表す．遷移関係 δ を Σ^* に以下のように拡張した関係 $q \overset{s}{\Rightarrow} q'$ に拡張する．

$$\begin{array}{l} q \overset{\varepsilon}{\Rightarrow} q \\ q \overset{as}{\Rightarrow} q' \quad \text{ここで，} q \xrightarrow{a} q'', \; q'' \overset{s}{\Rightarrow} q' \end{array}$$

定義 3.4. 有限オートマトン $A = \langle Q, \Sigma, \delta, q_0, F \rangle$ に対して $L(A) = \{w \mid q_0 \overset{w}{\Rightarrow} q_f, q_f \in F\}$ を A の受理言語という.

有限オートマトンの受理言語と正規表現が表す言語には以下のようなよく知られた関係がある.

定理 3.1.
- 正規表現 R に対して $[\![R]\!]$ を受理する有限オートマトン A_R が存在する.
- 有限オートマトン A に対して $L(A) = [\![R]\!]$ となる正規表現 R が存在する.

このことから，正規表現と有限オートマトンは表現能力が等しいことがわかる．また，有限オートマトンは，受理言語の積，和，否定に関して閉じていることが知られている.

定理 3.2.
- 有限オートマトン A_1, A_2 に対して，$L(A_3) = L(A_1) \cap L(A_2)$，$L(A_4) = L(A_1) \cup L(A_2)$ を満たす有限オートマトン A_3, A_4 が存在する.
- 有限オートマトン A に対して，$L(A_c) = \Sigma^* \backslash L(A)$ を満たす有限オートマトン A_c が存在する.

上記定理の証明については，オートマトンの教科書[1]を参照してほしい.

3.3 字句解析の仕組み

字句解析では，原始プログラムを Σ の系列として受取り，トークンごとに切り分けて出力する．トークンの種類に応じてオートマトンを構成する．それぞれのオートマトンを A_{kw}, A_{id}, A_{cnst}, A_{str}. A_{del} とする.

字句解析はこれらのオートマトンを図**??**のように組み合わせて構成される.

[1] J. ホップクロフト/J. ウルマン：「オートマトン言語理論 計算論 1」訳：野崎昭弘ら　2.5 および 3.2

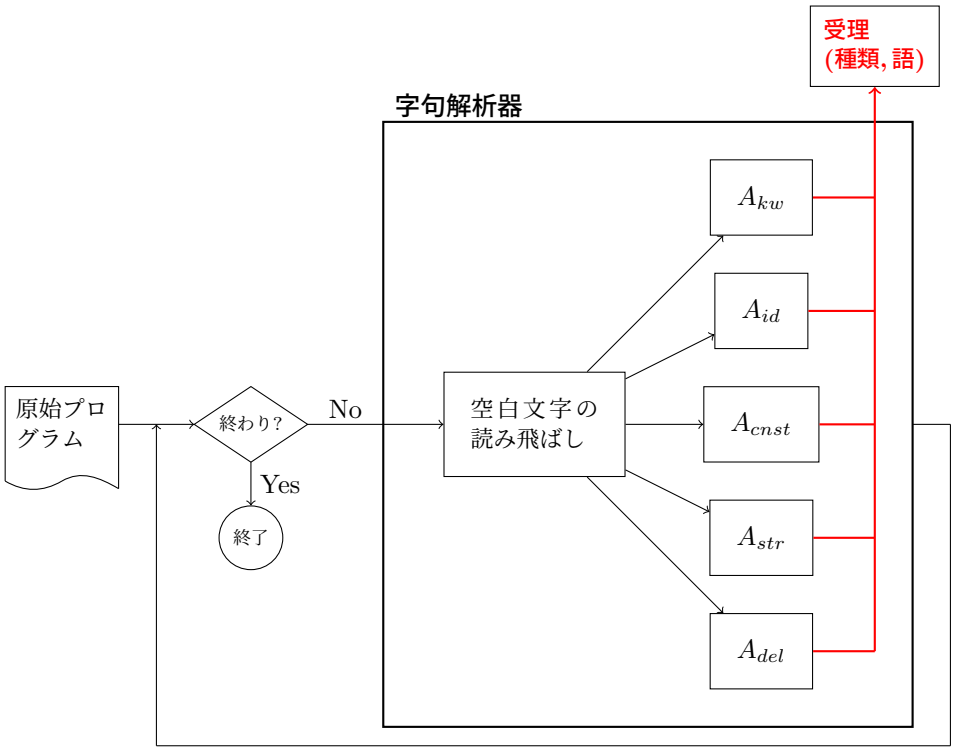


図 3.1: 字句解析器の構成

ステップ 1: 空白，タブコード，改行などの空白文字を読み飛ばす．

ステップ 2: 次の文字を A_{kw}, A_{id}, A_{cnst}, A_{str}, A_{del} に入力する．

ステップ 3: いずれかのオートマトンが受理状態に達するまで，ステップ 2 を繰り返す．

ステップ 4: いずれかのオートマトンが受理状態に達したらそのオートマトンの種類と受理した系列を出力し，すべてのオートマトンの状態を初期状態に戻す．

ステップ 5: 入力の最後に達していなければ，ステップ 1 に戻る．入力の最後に達したら終了

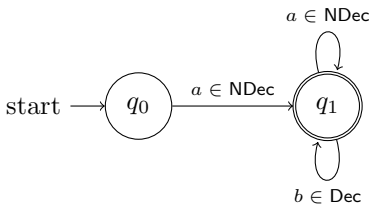
字句解析の実現においては，トークンの全体の和集合を受理するオートマトンを作成し，受理状態がどのトークンによるものかを区別しておくことで字句解析が実現ができる．有限オートマトンが和集合について閉じていることからこのような構成はいつも可能であることが保証されている．

受理状態が 2 つ以上のオートマトンによる場合は，受理の優先順位を決めておくことで字句解析を行う．予約語と識別子は一般に同じ文字列を受理する可能性がある．その場合は，予約語を優先させる．

字句解析器のプログラミング　字句解析を行うオートマトンを構成するために，正規表現を用いる．正規表現 R を $[\![R]\!]$ を受理する有限オートマトンに変換して，字句解析器に組み込む．有限オート

NDec(NDec|Dec)*

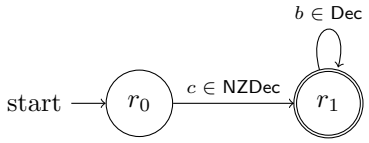
A_{id}:



(a) 識別子トークンを受理するオートマトン

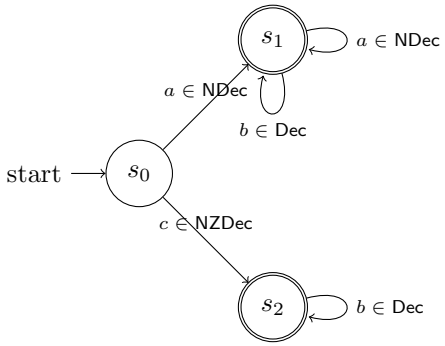
NZDec Dec*

A_{cnst}:



(b) 自然数定数トークンを受理するオートマトン

$A_{id} \cup A_{cnst}$:



(c) 両方のトークンを受理するオートマトン

図 3.2: 字句解析の正規表現とオートマトン

マトンからプログラムへの自動変換は比較的容易に実現できる．字句解析器は，トークンの集合を正規表現で記述するだけで，自動生成することが可能である．

図 3.2 に識別子と自然数定数を受理するオートマトンを示す．いずれかを受理するオートマトン $A_{id \cup cnst}$ は (c) のようになる．s_1 が受理状態であった場合は識別子，s_2 が受理状態であった場合は自然数定数である．

3.4 字句解析器の生成

3.4.1 正規表現の記法

オートマトンに対応する動作をさせるために計算機上で正規表現は以下のように記述される．

- ε, $\varnothing$ を直接表現する記法はない．
- $a \in \Sigma$ については，`"a"` のように引用符で囲む．$s \in \Sigma^+$ についてはまとめて `"abc"` のように引用符で囲む．
- $R_1|R_2$ については，R_1 の記法 | R_2 の記法．連続した文字コードを持つ場合には，`[X-Y]` のように途中を省略できる．$s =$ `a1a2`$\cdots$`an` $\in \Sigma^+$ に対して `[`s`]` は `a1|a2|`$\cdots$`|an` を表す．例えば `[0-9]` は `0|1|2|3|4|5|6|7|8|9` を表す．

- Kleene 閉包は * を後ろにつける．[ab]* など
- ラベルの後ろに正規表現を書いて名前をつけることができる．NZDEC [1-9] や DEC [0-9] など．ラベルの参照時は{}をつける．

ラベルの機能を使うと自然数定数は {NZDEC}{DEC}*と記述できる．
NDEC [_a-zA-Z] とすると，識別子名を表す系列は{NDEC}({NDEC}|{DEC})*となる．

3.4.2　字句解析器生成ツール

lex

lex では正規表現とアクション (C 言語) の対を並べて記述し，入力文字列から正規表現で構成されるオートマトンが受理する系列を抽出して，対応するアクションを実行するような C 言語プログラムを生成する．受理した入力系列は yytext という文字列に格納される．
例えば，以下のような仕様ファイル lex1.l を用意したとする．

```
%%
[A-Za-z]+ {printf("String found:%s\n",yytext);}
[0-9]+    {printf("Digits found:%s\n",yytext);}
.         {printf("Other character:%d\n",yytext[0]);}
%%
```

lex では，ピリオド"."は Σ の任意の文字のいずれかであることを示す．受理の優先度は上に書かれるほど高い．入力の”A”は 1 番めと 3 番めの両方に受理されるが，アクションは 1 番めが実行される．lex は，上のような仕様を受け取って，正規表現にマッチする系列が入力されたとき，アクションを実行する C 言語プログラムを生成する．

```
% lex -o lex1.c lex1.l
% cc -o lex1 lex1.c -ll
```

1 行目で仕様 lex1.l から lex1.c を生成し，これを 2 行目で実行形式に変換している．
より複雑なプログラムを書くために最初の%%の前に C 言語の include 文や define マクロまたはプロトタイプ宣言，最後の%%の後に C 言語の関数を記述することができる．

```
%{
#include <stdio.h>
void pp(char*);
%}
%%
[A-Za-z]+ {pp("String");}
[0-9]+    {pp("Digits");}
```

```
.         {printf("Other character:%d\n",yytext[0]);}
%%
int main(void)
{
    yylex();
    return 0;
  }
void pp(char *s) {
    printf("%s found:%s\n",s,yytext);
  }
```

sly(python)

Python では sly パッケージ[2]で字句解析器を生成できる．正規表現は，r に続けて記載する．正規表現で制御に使う文字はバックスラッシュでエスケープする．python における正規表現については https://docs.python.org/ja/3/howto/regex.html などを参照せよ．

```
int x;
int f(int y){
    r=3; return x+y;
  }
int main(void) {
int x,y; x=6;
{int x; x=7;
y=f(x);}
printf("%02d",x+y);
}
```

上記の C 言語風のプログラムを字句解析をすると以下のようになる．

```
from sly import Lexer

class CalcLexer(Lexer):
# Set of token names.   This is always required
tokens = { ID, STRING, KWORD, NUMBER, PLUS, MINUS, TIMES,
DIVIDE, ASSIGN, LPAREN, RPAREN, LBRACE, RBRACE,SEMICOLON, COMMA }

# String containing ignored characters between tokens
ignore = ' \t\n'
```

[2]https://sly.readthedocs.io/en/latest/sly.html#writing-a-lexer

```
# Regular expression rules for tokens
ID      = r'[a-zA-Z_][a-zA-Z0-9_]*'
STRING  = r'\".*?\"'
KWORD   = r'int|return|printf|void'
NUMBER  = r'\d+'
PLUS    = r'\+'
MINUS   = r'-'
TIMES   = r'\*'
DIVIDE  = r'/'
ASSIGN  = r'='
LPAREN  = r'\('
RPAREN  = r'\)'
LBRACE  = r'{'
RBRACE  = r'}'
SEMICOLON = r';'
COMMA = r','

if __name__ == '__main__':
data = '''
int x;
int f(int y){
    r=3; return x+y;
  }
int main(void) {
int x,y; x=6;
{int x; x=7;
y=f(x);}
printf("%02d",x+y);
}
'''
lexer = CalcLexer()
for tok in lexer.tokenize(data):
print('type=%r, value=%r' % (tok.type, tok.value))
```

第4章　構文解析

4.1　構文解析の基礎

4.1.1　BNF 定義と CF 文法

プログラミング言語の構文は CF 文法 (文脈自由文法, context-free grammar) によって定義される.

定義 4.1. *CF* 文法 G は $G = \{N, \Sigma, P, S\}$ の四つ組で定義される．ここで，N は非終端記号の集合，Σ は終端記号の集合，P は生成規則の集合，$S \in N$ は開始記号である．P の各生成規則は $A \to \alpha$ の形をしている．ここで，$A \in N$, $\alpha \in (N \cup \Sigma)^*$ である.

プログラミング言語の構文解析における CF 文法では，Σ は字句解析によって得られるトークンの集合とするのが一般的である.

PL2[1]のプログラムにおける手続きの定義は以下の文法で定めらている．'procedure' は予約語，'IDENT' は識別子のトークン，'(', ')', ':' および ',' は区切り記号である.

⟨proc_decl⟩ → 'procedure' ⟨proc_name⟩ '(' ')' ':' ⟨inblock⟩
⟨proc_decl⟩ → 'procedure' ⟨proc_name⟩ '(' ⟨id_list⟩ ':' ⟨inblock⟩
⟨proc_name⟩ → 'IDENT'

⟨id_list⟩ → 'IDENT'
⟨id_list⟩ → ⟨id_list⟩ ',' 'IDENT'

⟨proc_decl⟩ は識別子のトークンである名前 proc_name を持つ手続きを定義する．本体は ⟨inblock⟩ で記述されるプログラムで定義される.

$\beta, \gamma \in (N \cup \Sigma)^*$, $A \in N$, $A \to \alpha$ に対して，$\beta A \gamma \Rightarrow \beta \alpha \gamma$ を P による**導出**という．導出は終端記号と非終端記号の系列に対して，生成規則の左辺の非終端記号を右辺で置き換える操作である．$\alpha_1, \alpha_2, \cdots, \alpha_n \in (N \cup \Sigma)^*$ において，$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$ のとき，$\alpha_1 \Rightarrow^* \alpha_n$ と書くことにする.

G が定義する言語は $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ で定義される．すなわち開始記号 S から P の生成規則を用いて導出し，すべての非終端記号がおきかえられた終端記号の系列である.

G によってプログラミング言語が定義されているとき，原始プログラムを字句解析した系列 w が $w \in L(G)$ であるとき，構文としてそのプログラムは正しいといえる．$w \notin L(G)$ であれば，構文エラーがあると判断し，目的コードの生成は行わない.

例 4.1. 以下の *PL2* プログラムは ⟨proc_decl⟩ から生成される系列である．ここで，⟨inblock⟩ $\Rightarrow^*$ *'IDENT' ':=' 'IDENT' '+' 'NUMBER'* とする.

[1]学生実験 c において対象とする原始プログラムを記述するプログラミング言語

```
procedure main(x,y):
x:=y+1
```

BNF 定義

形式的にはプログラミング言語は文脈自由文法で定義されるが，文脈自由文法では生成規則の集合を記述すれば十分であることが多い．このため，生成規則の集合でプログラミング言語を定義する．その際に一般に BNF(Backus-Nauer Form) 定義とと呼ばれる記法が用いられる．BNF は生成規則とほぼ同じであるが，→ の代わりに ::= を用いる．また，同じ左辺をもつ規則をまとめて，右辺を | で並べて記載する．⟨proc_decl⟩ についての BNF 定義は以下のようになる．

$$
\begin{aligned}
\langle \text{proc_decl} \rangle \quad &::= \quad \text{`}\texttt{procedure}\text{'}\ \langle \text{proc\_name} \rangle\ \text{`}(\text{'}\ \text{`})\text{'}\ \text{`};\text{'}\ \langle \text{inblock} \rangle \\
&\quad\ \mid \quad \text{`}\texttt{procedure}\text{'}\ \langle \text{proc\_name} \rangle\ \text{'}(\text{'}\ \langle \text{id\_list} \rangle\ \text{`};\text{'}\ \langle \text{inblock} \rangle \\
\langle \text{proc_name} \rangle \quad &::= \quad \text{`}\texttt{IDENT}\text{'} \\
\langle \text{id_list} \rangle \quad &::= \quad \text{`}\texttt{IDENT}\text{'} \\
&\quad\ \mid \quad \langle \text{id_list} \rangle\ \text{`,'}\ \text{`}\texttt{IDENT}\text{'}
\end{aligned}
$$

最左導出と最右導出

導出の途中に現れる系列 $\alpha \in (N \cup \Sigma)^*$ において，α が複数の非終端記号を含むときどの非終端記号を置き換えて導出するかによって，複数の導出が存在する．例えば，A, B を非終端記号に含み，$\{a, b, c\} \subseteq \Sigma$, $A \to a$, $B \to b$ という生成規則を持つとき，ABc に対する導出は $ABc \Rightarrow aBc$ と $ABc \Rightarrow Abc$ がある．$\alpha \in (N \cup \Sigma)^*$ に対する導出のうち，最も左にある非終端記号を置き換える導出を**最左導出**，最も右にある非終端記号を置き換える導出を**最右導出**といい，それぞれ $\alpha \Rightarrow_{lm} \beta$, $\alpha \Rightarrow_{rm} \beta'$ のように記述する．

α の $A \to \gamma$ による最左導出 $\alpha \Rightarrow_{lm} \beta$ が存在するとき，α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_1 \in \Sigma^*$ である．すなわち，α の A より左の α_1 は終端記号だけの系列か，ε であり，$\beta = \alpha_1 \gamma \alpha_2$ である．

α の $A \to \gamma$ による最右導出 $\alpha \Rightarrow_{rm} \beta$ が存在するとき，α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_2 \in \Sigma^*$ である．すなわち，α の A より右の α_2 は終端記号だけの系列か，ε であり，$\beta = \alpha_1 \gamma \alpha_2$ である．

最左導出，最右導出の 0 回以上の繰り返しをそれぞれ $\Rightarrow^*_{lm}$, $\Rightarrow^*_{rm}$ で表す．

例 4.2. *PL2* の ⟨proc_decl⟩ からの最左導出は以下のようになる．

$$
\begin{aligned}
\langle \text{proc_decl} \rangle \quad &\Rightarrow_{lm} \quad \text{`}\texttt{procedure}\text{'}\ \langle \text{proc\_name} \rangle\ \text{`}(\text{'}\ \langle \text{id_list} \rangle\ \text{`})\text{'}\ \text{`:'}\ \langle \text{inblock} \rangle \\
&\Rightarrow_{lm} \quad \text{`}\texttt{procedure}\text{'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`}(\text{'}\ \langle \text{id\_list} \rangle\ \text{`})\text{'}\ \text{`:'}\ \langle \text{inblock} \rangle \\
&\Rightarrow_{lm} \quad \text{`}\texttt{procedure}\text{'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`}(\text{'}\ \langle \text{id\_list} \rangle\ \text{`,'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`})\text{'}\ \text{`:'}\ \langle \text{inblock} \rangle \\
&\Rightarrow_{lm} \quad \text{`}\texttt{procedure}\text{'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`}(\text{'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`,'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`})\text{'}\ \text{`:'}\ \langle \text{inblock} \rangle
\end{aligned}
$$

さらに $\langle \text{inblock} \rangle \Rightarrow^*_{lm}$ `IDENT` `:=` `IDENT` `+` `NUMBER` であるとき

$$
\Rightarrow^*_{lm} \quad \text{`}\texttt{procedure}\text{'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`}(\text{'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`,'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`})\text{'}\ \text{`:'}\ \text{`}\texttt{IDENT}\text{'}\ \text{`:='}\ \text{`}\texttt{IDENT}\text{'}\ \text{`+'}\ \text{`}\texttt{NUMBER}\text{'}
$$

PL2 の ⟨proc_decl⟩ からの最右導出は以下のようになる．

⟨proc_decl⟩ $\Rightarrow_{rm}$ ‘procedure’ ⟨proc_name⟩ ‘(’ ⟨id_list⟩ ‘)’ ‘:’ ⟨inblock⟩
ここで ⟨inblock⟩ $\Rightarrow^*_{rm}$ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ‘NUMBER’ であるとき
$\Rightarrow^*_{rm}$ ‘procedure’ ⟨proc_name⟩ ‘(’ ⟨id_list⟩ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ‘NUMBER’
$\Rightarrow_{rm}$ ‘procedure’ ⟨proc_name⟩ ‘(’ ⟨id_list⟩ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ‘NUMBER
$\Rightarrow_{rm}$ ‘procedure’ ⟨proc_name⟩ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ‘NUMBER
$\Rightarrow_{rm}$ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ‘NUMBER’

開始記号 S からの 0 回以上の導出によって得られる $\alpha \in (N \cup \Sigma)^*$ (ここで，$S \Rightarrow^* \alpha$) のことを**文形式** (sential form) という．さらに，$S \Rightarrow^*_{lm} \beta$, $S \Rightarrow^a_{rm} st\gamma$ となる $\beta, \gamma \in (N \cup \Sigma)^*$ のことをそれぞれ左文形式，右文形式という．

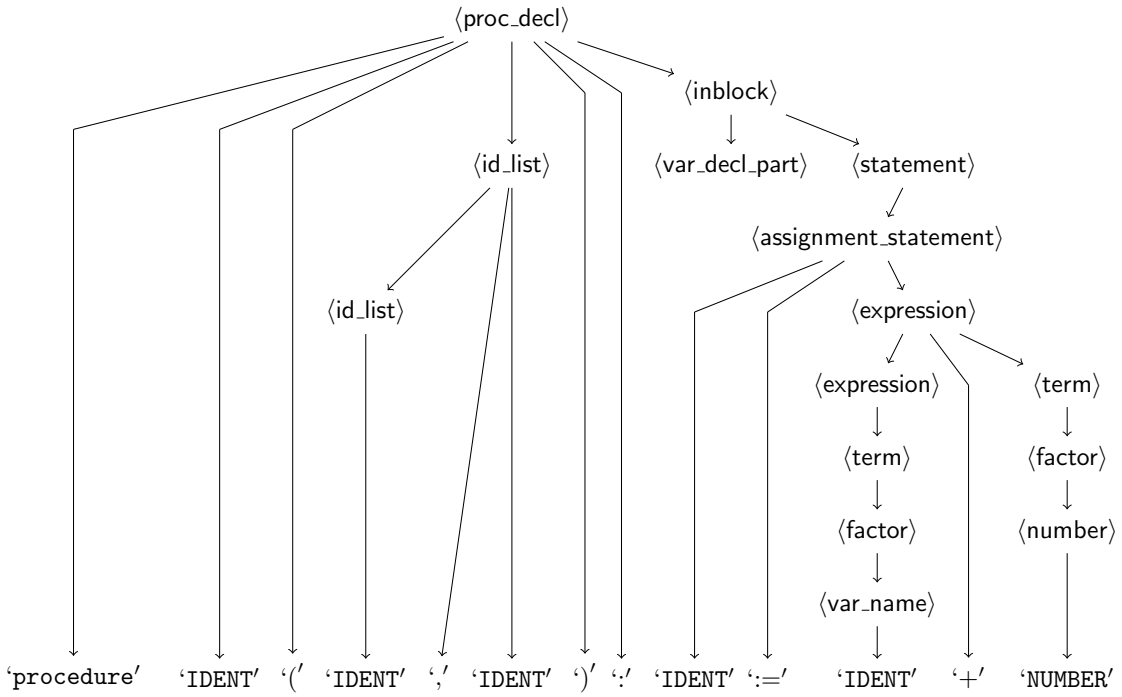
4.1.2 解析木

導出の過程は木構造で表現することができる．非終端記号，終端記号を頂点として表し，$A \to \alpha$ で導出したとき，A の頂点から $\alpha = \alpha_1\alpha_2\cdots\alpha_n(\alpha_i \in N \cup \Sigma)$ の a_i に有向辺を追加する．S からの導出をこのようにして表すと，S の頂点を根，$a \in \Sigma$ の頂点を葉とする木構造が構成される．この木構造のことを**解析木** (parse tree)(あるいは**構文木** (syntax tree)) という．

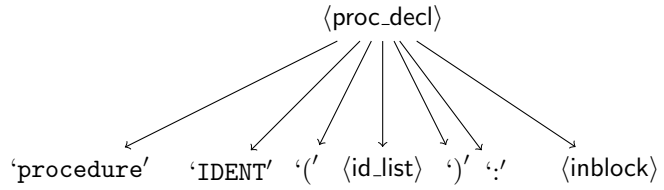
例 4.1 の例の (最左) 導出は以下のようになる．

⟨proc_decl⟩ ⇒ ‘procedure’ ⟨proc_name⟩ ‘(’ ⟨id_list⟩ ‘)’ ‘:’ ⟨inblock⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ⟨id_list⟩ ‘)’ ‘:’ ⟨inblock⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ⟨id_list⟩ ‘,’ ‘IDENT’ ‘)’ ‘:’ ⟨inblock⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ⟨inblock⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ⟨var_decl_part⟩ ⟨statement⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ⟨statement⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ⟨assignment_statement⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ⟨expression⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ⟨expression⟩ ‘+’ ⟨term⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ⟨term⟩ ‘+’ ⟨term⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ⟨factor⟩ ‘+’ ⟨term⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ⟨var_name⟩ ‘+’ ⟨term⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ⟨term⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ⟨factor⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ⟨number⟩
⇒ ‘procedure’ ‘IDENT’ ‘(’ ‘IDENT’ ‘,’ ‘IDENT’ ‘)’ ‘:’ ‘IDENT’ ‘:=’ ‘IDENT’ ‘+’ ‘NUMBER’

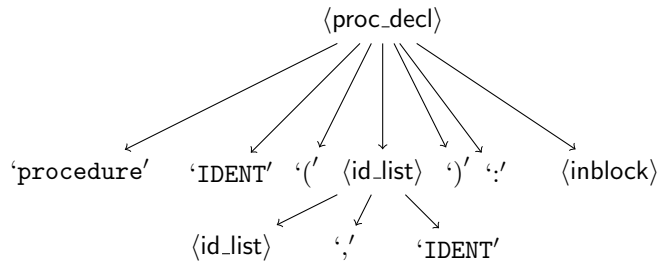
この導出から構成される木構造は以下のようになる．



解析木を構成するにあたって，最左導出では，開始記号は単一の記号なので，必ず最左の非終端記号であり，以下のような木を構成する．

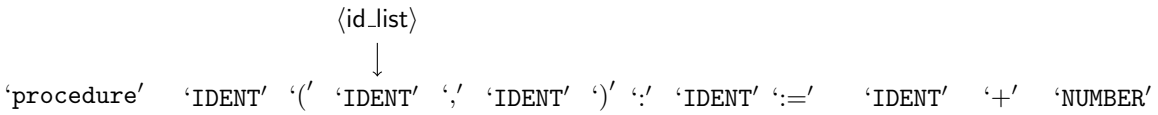


次に最左の非終端記号である ⟨id_list⟩ を導出すると以下の木が得られる．

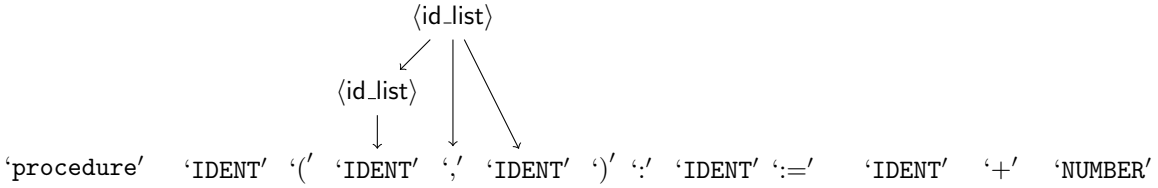


このように最左導出によって構文木は根から下向きに構成される．このような構文解析を**下降型構文解析**という．

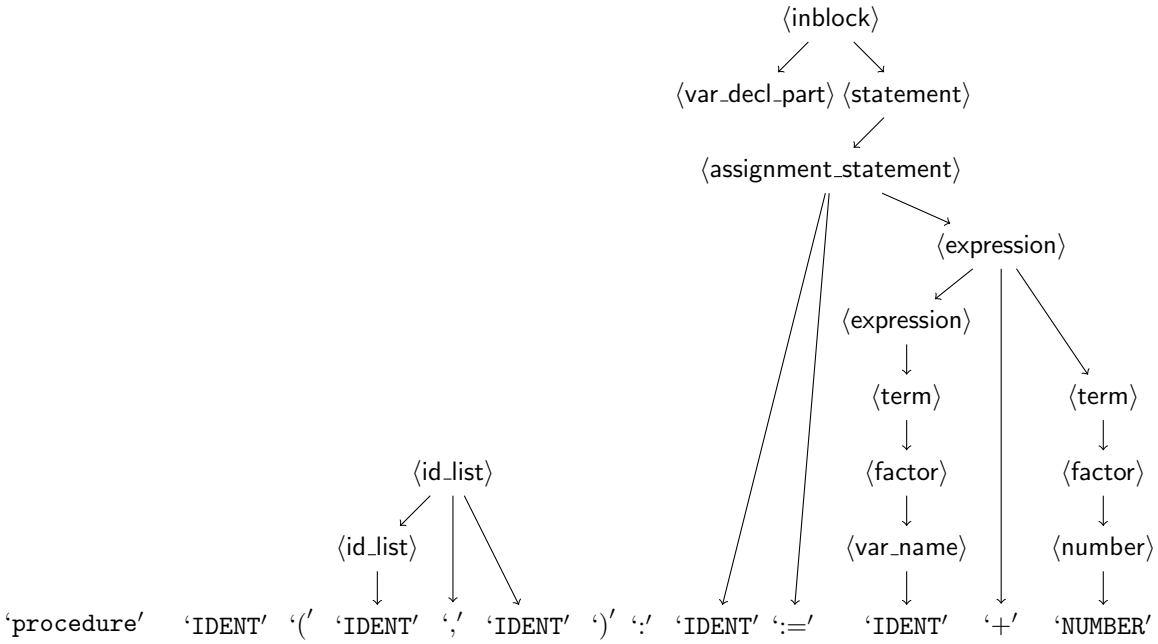
入力系列に対して引数リスト ⟨id_list⟩ を構成する ‘IDENT′ に ⟨id_list⟩ → ‘IDENT′ を逆向きに適用し，木を構成する．



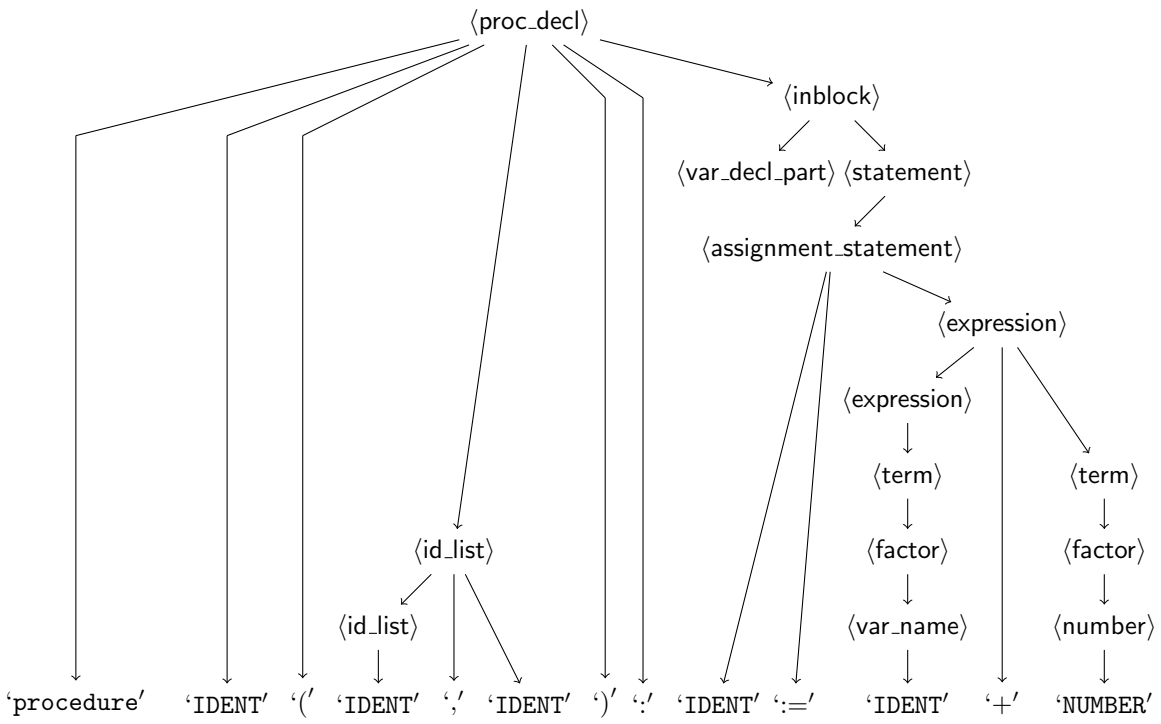
次に，⟨id_list⟩ → ⟨id_list⟩‘,′‘IDENT′ を逆向きに適用する．



以下，右辺にマッチする生成規則を適用し，⟨inblock⟩ の部分木を生成する．



最後に ⟨proc_decl⟩ を左辺に持つ規則によって木が構成される．



このように構文解析する系列に対して生成規則の右辺をマッチさせて，解析木を構成すると木は葉から根の方向に構成される．このような構文解析を**上昇型構文解析**という．

4.1.3　文法の曖昧性

⟨proc_decl⟩ の例においては，どのような順番で生成規則を適用しても構成される解析木は同じである．文法によっては，生成規則の適用の順番によって同じ終端記号系列に対して複数の解析木が構成できることがある．

CF 文法 $G_1 = \langle \{E\}, \{\mathtt{or}, \mathtt{and}, eqt\}, P_1, E \rangle$ を考える．ここで，

$$P_1 = \left\{ \begin{array}{l} E \to E\ \mathtt{or}\ E \\ E \to E\ \mathtt{and}\ E \\ E \to eqt \end{array} \right\}$$

とする．eqt は等式のトークンであるとする．

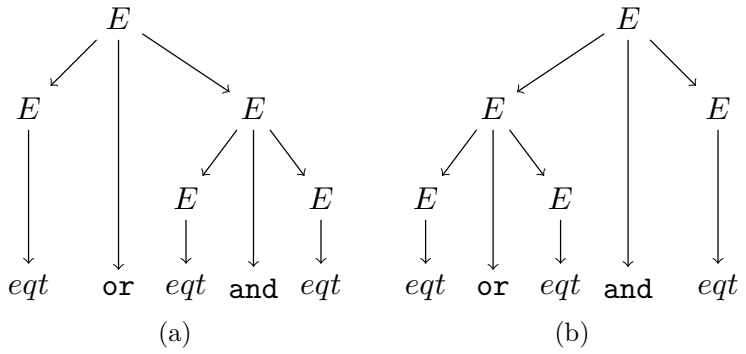


図 4.1: 複数の解析木

$L(G_1)$ に属する eqt or eqt and eqt という系列に対して図 4.1 のような 2 つの解析木が存在する．導出において，$\underline{E}$ を置き換えの対象となった E とすると，(a) は $\underline{E} \Rightarrow \underline{E}\,\mathtt{or}\,E \Rightarrow eqt\,\mathtt{or}\,\underline{E} \Rightarrow eqt\,\mathtt{or}\,\underline{E}\,\mathtt{and}\,E \Rightarrow eqt\,\mathtt{or}\,eqt\,\mathtt{and}\,\underline{E} \Rightarrow eqt\mathtt{or}eqt\mathtt{and}eqt$, (b) は $\underline{E} \Rightarrow \underline{E}\,\mathtt{and}\,E \Rightarrow \underline{E}\,\mathtt{or}\,E\,\mathtt{and}\,E \Rightarrow eqt\,\mathtt{or}\,\underline{E}\mathtt{and}\,E \Rightarrow eqt\,\mathtt{or}\,eqt\,\mathtt{and}\,\underline{E} \Rightarrow eqt\,\mathtt{or}\,eqt\,\mathtt{and}\,eqt$, という導出に対する解析木となる．

このように 1 つのトークン系列に対して 2 つ以上の異なる解析木が存在するとき文法は曖昧であるという．

3 つの eqt のトークンが具体的にそれぞれ $x = 1$, $y = 2$, $z = 3$ であるとする．x, y, z がすべて 1 であるとすると，$x = 1$ は真，$y = 2$, $z = 3$ は偽となる．and, or がそれぞれ論理積，論理和を表すとすると，(a) の解釈では，最初の $\mathrm{eqt}(x = 1)$ が真となるので，全体の論理式は真であるが，(b) の解釈では最後の $\mathrm{eqt}(z = 3)$ が偽であるので論理式は偽となる．

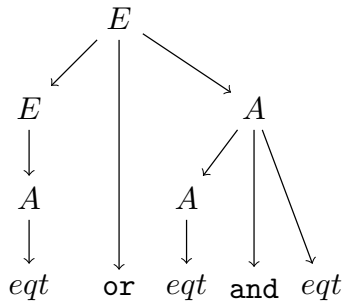
曖昧な文法では入力トークンの系列の意味が定まらないため，プログラミング言語の文法は曖昧でないことが望まれる．

論理和と論理積の場合，一般には和が積が優先されると解釈されるため，(a) の解釈をとる．これにそって G_1 の曖昧さを除去した文法 G_2 を以下に示す．

$$G_2 = \langle \{E, A\}, \{\mathtt{or}, \mathtt{and}, \mathtt{eqt}\}, P_2, E \rangle$$

$$\text{ここで，}\ P_2 = \left\{ \begin{array}{l} E \to E\,\mathtt{or}\,A \\ E \to A \\ A \to A\,\mathtt{and}\,\mathtt{eqt} \\ A \to \mathtt{eqt} \end{array} \right\}$$

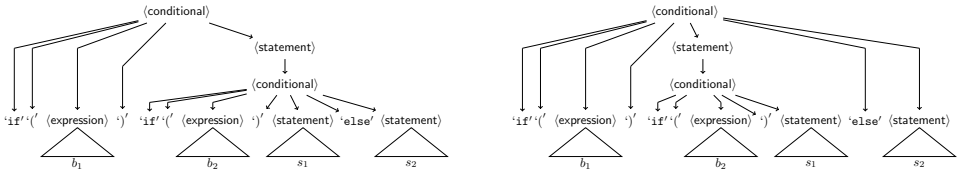
$L(G_1) = L(G_2)$ であり，G_2 は曖昧ではない．これは，and を or より優先するために，新たな非終端記号 A を導入して，and が解析木で or よりも葉に近いように解釈されるようにしたためである．G_2 では，**単一生成規則** (single production)“$E \to A$” を導入したため解析木のサイズが増大する．n レベルの優先度の導入のために n 個の非終端記号が導入され，n 個の単一生成規則が必要になる．曖昧性を除去するトレードオフとして，解析木が大きくなり，解析により長い時間がかかることになる．



このため，生成規則の適用順序を決めて解析の曖昧性を除去する場合もある．C 言語などの条件分岐は `else` つきの場合と `if` ～ `then` ～ `else` ～，`else` なしの場合 `if` ～ `then` ～ の場合がある．

$$
\begin{aligned}
\langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \cdots \\
\langle \text{conditional} \rangle &::= \text{`}\texttt{if}\text{'}\,\text{`}(\text{'}\,\langle \text{expression} \rangle\,\text{`})\text{'}\,\langle \text{statement} \rangle \\
&\quad \mid \quad \text{`}\texttt{if}\text{'}\,\text{`}(\text{'}\,\langle \text{expression} \rangle\,\text{`})\text{'}\,\langle \text{statement} \rangle\,\text{`}\texttt{else}\text{'}\,\langle \text{statement} \rangle
\end{aligned}
$$

b_1, b_2 を ⟨expression⟩ から導出されるトークン列, s_1, s_2 を ⟨statement⟩ から導出されるトークン列とする．`if` (b_1) `if` (, b_2) s_1 `else` s_2 に対して図**??**のような 2 つの解析木が存在する．



(a)　　(b)

C 言語では，`else` は最も近い条件に対するものであると解釈するので，解析木として (b) であると決め，曖昧さを除去している．したがって，b_1 が真で，b_2 が偽であると評価された場合に実行される．構文解析における曖昧性は望ましいものとは言えないので，プログラム言語を定義する場合は無曖昧な CF 文法で定義することを基本とする．

上記の場合は，新たな予約語 `fi` を導入することで，曖昧さを解消できる．

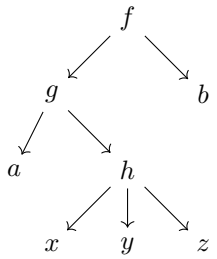
$$
\begin{aligned}
\langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \cdots \\
\langle \text{conditional} \rangle &::= \text{`}\texttt{if}\text{'}\,\text{`}(\text{'}\,\langle \text{expression} \rangle\,\text{`})\text{'}\,\langle \text{statement} \rangle\,\text{`}\texttt{fi}\text{'} \\
&\quad \mid \quad \text{`}\texttt{if}\text{'}\,\text{`}(\text{'}\,\langle \text{expression} \rangle\,\text{`})\text{'}\,\langle \text{statement} \rangle\,\text{`}\texttt{else}\text{'}\,\langle \text{statement} \rangle\,\text{`}\texttt{fi}\text{'}
\end{aligned}
$$

4.1.4　解析木の表現

木の頂点の出力次数がわかっているとき解析木をトラバースする方法に基づいて，木構造を 1 次元的に表現することができる．木 T の根頂点を $root(T)$ と書くことにする．$root(T)$ の次数が n であるとき $root(T)$ からの子を根頂点とする木を左から $T_1, T_2, \cdots, T_n$ とする．

- **プリオーダー** $root(T)$ の次に T_1 のプリオーダー系列 $\mathsf{pre}(T_1)$ を連結し，その次に $\mathsf{pre}(T_2)$ を連結し，というのを再帰的に T_n まで繰り返す.
- **インオーダー** T_1 のインオーダーの系列 $\mathsf{in}(T_1)$ に $root(T)$ を連結し，その次に $\mathsf{in}(T_2), \cdots, \mathsf{in}(T_n)$ を連結する.
- **ポストオーダー** T_1 のポストオーダー $\mathsf{post}(T_1)$, $\cdots \mathsf{post}(T_n)$ を連結し，最後に $root(T)$ を連結する.

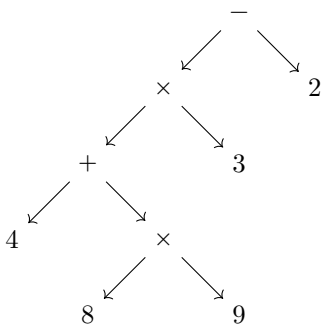
関数を表す式 $f(g(a, h(x, y, z)), b)$ に対して，関数名とその引数を親子関係として木構造で表すと，以下のようになる.



この木構造はプリオーダーで表すと $fgahxyzb$ となる.

f が 2 引数，g が 2 引数，h が 3 引数であることが既知であれば，この系列から関数の形を再現できる.

四則演算の算術式 $(4 + 8 \times 9) \times 3 - 2$ は以下のような構造を持つ.



これをポストオーダー系列で表すと $489 \times +3 \times 2-$ となる.

この記法は**逆ポーランド記法**と呼ばれる．それぞれの演算の引数が決まっていればスタックに対するルールに従って計算すれば式の評価が行われる.

逆ポーランド記法における評価の方法は以下のとおりである.

1. 数字がきたらスタックに積む
2. 演算がきたら引数の数だけスタックからポップして演算を施し，その結果をスタックにプッシュする.
3. 入力を全部読んだあとにスタックトップのあるのが評価値

$489 \times +3 \times 2-$ をスタックと対にしてこの方法で評価すると以下のようになる．

$$\begin{aligned}
(489 \times +3 \times 2-, []) &\rightarrow (89 \times +3 \times 2-, [4]) \\
&\rightarrow (9 \times +3 \times 2-, [4, 8]) \\
&\rightarrow (\times + 3 \times 2-, [4, 8, 9]) \\
&\rightarrow (+3 \times 2-, [4, 72]) \\
&\rightarrow (3 \times 2-, [76]) \\
&\rightarrow (\times 2-, [76, 3]) \\
&\rightarrow (2-, [228]) \\
&\rightarrow (-, [228, 2]) \\
&\rightarrow (\varepsilon, [226])
\end{aligned}$$

プリオーダーは，木の根を開始記号とする最左導出の規則適用に相当し，ポストオーダーは，木の根を開始記号とする最右導出の逆順の規則適用に相当する．

4.1.5　First と Follow

入力系列 w が与えられたとき，できるだけ効率的に CF 文法に従って $w \in L(G)$ (あるいは $w \notin L(G)$) を示したい．すなわち，$S \Rightarrow^* w$ となる導出を示す必要がある．この際に導出に使う生成規則を選択する必要がある．例えば，$A \rightarrow aB, A \rightarrow bC$ という 2 つの規則があり，$\beta A \gamma$ という文形式から次の導出を行う．このとき，両方の生成規則が適用可能である．簡単のために最左導出であると考えると，$\beta \in \Sigma^*$ であり，入力文字列 w の接頭語が βa であるならば $A \rightarrow aB$ を選択すべきであり，接頭語が βb であるならば，$A \rightarrow bC$ を選択すべきである．

このように次に処理すべきトークン (上の場合は β の次のトークン) によって生成規則の選択を決めることができる場合がある．

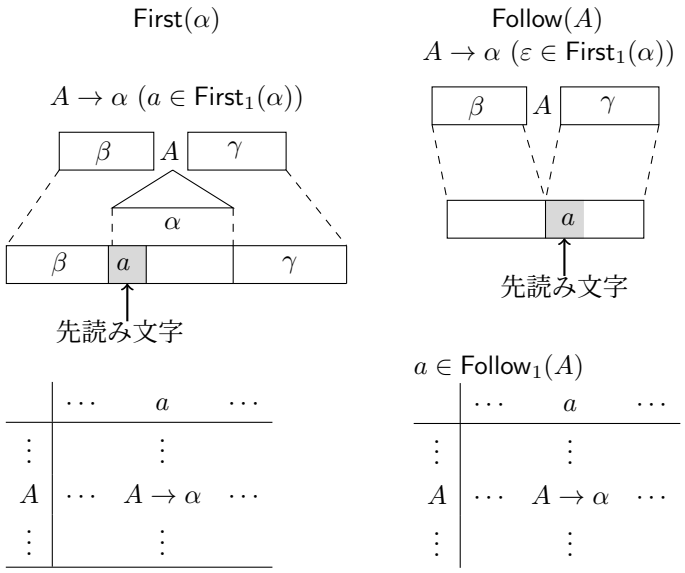


図 4.2: 導出の際の先読みによる決定

さらに，$A \to \varepsilon$ の場合は，A に続く文字が何かによってこの生成規則を適用できるかどうかを決めることができる場合がある．

コンパイラの構文解析においては，このようなまだ読み込んでいない**先読み文字**の情報を用いて，生成規則の選択を決めることが行われる．もしこのような先読み文字の情報を用いない場合は，すべての選択を試してみることになる．生成規則の選択がまちがっていた場合は，まちがいをやりなおす**バックトラック**によって規則を選び直さないといけなくなる．βb が接頭語であるにもかかわらず，$A \to aB$ で書き換えた場合は，書き換えをやりなおして，$A \to bC$ に書き換える．このとき一旦読んでしまったトークンをまた入力系列に戻す．このような操作には時間がかかるため，一般的なプログラミング言語の設計においては生成規則の解析と先読み文字によって，適用できる生成規則が一意的になるような CF 文法で設計される．

生成規則が適用されるときに生成される接頭語を First，生成規則が適用されて生成される文字列の直後の接頭語を Follow といい，これらは生成規則をあらかじめ解析することによって得られる．

定義 4.2. *CF* 文法 $G = \langle N, \Sigma, P, S \rangle$ が与えられたとき，系列 $\alpha \in (N \cup \Sigma)^*$ に対する First_1 集合を以下のように定義する．

$\alpha \Rightarrow^* \varepsilon$ のとき，$\mathsf{First}_1(\alpha) = \{a \in \Sigma \,|\, \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\} \cup \{\varepsilon\}$

$\alpha \not\Rightarrow^* \varepsilon$ のとき，$\mathsf{First}_1(\alpha) = \{a \in \Sigma \,|\, \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\}$

$\mathsf{First}_1(\alpha)$ は，α から導出によって生成される記号列で先頭に出現する可能性のある終端記号の集合を表す．α から ε が生成される，つまり，α が導出によって消失しうる場合には，$\mathsf{First}_1(\alpha)$ には ε という特別な要素を含めることにする．明らかに $\mathsf{First}_1(\varepsilon) = \{\varepsilon\}$ である．

$\alpha \in (N \cup \Sigma)^+$ に対して，α の最初の記号を $\mathsf{fst}(\alpha)$，最初の記号を除いた系列を $\mathsf{fol}(\alpha)$ と書くことにする．($\mathsf{fst}(\alpha) \in N \cup \Sigma$, $\mathsf{fol}(\alpha) \in (N \cup \Sigma)^*$ となる．) CF 文法 $\langle N, \Sigma, P, S \rangle$ において，PO_P を

以下のように定める．$PO_P = \{\gamma \in (N \cup \Sigma)^* \mid \beta\gamma = \alpha, \beta \in (N \cup \Sigma)^*, A \to \alpha \in P\}$　PO_P は，生成規則 $A \to \alpha$ の右辺 α の接尾語の集合を表す．

$\alpha \in PO_P \cup N$ に対して $\mathsf{First}_1(\alpha)$ は，以下のようにして計算することができる．

アルゴリズム 4.1. (First_1 を求めるアルゴリズム):

ステップ 1: $\mathsf{First}_1(\varepsilon) \leftarrow \{\varepsilon\}$

ステップ 2: $\alpha \in (PO_P \backslash \{\varepsilon\}) \cup N$ に対して

(a) $\mathsf{fst}(\alpha) \in \Sigma$ ならば，$\mathsf{First}_1(\alpha) \leftarrow \{\mathsf{fst}(\alpha)\}$

(b) $\mathsf{fst}(\alpha) \in N$ ならば，$\mathsf{First}_1(\alpha) \leftarrow \varnothing$

ステップ 3: すべての $\mathsf{First}_1(\alpha)$ *(*$\alpha \in PO_P \cup N$*)* に新たな要素が追加されなくなるまで以下を繰り返す．

(a) $\mathsf{fst}(\alpha) = A \in N$ である α に対して $\mathsf{First}_1(\alpha)$ に以下の要素を追加する．

$\varepsilon \notin \mathsf{First}_1(A)$ のとき，$\mathsf{First}_1(A)$

$\varepsilon \in \mathsf{First}_1(A)$ のとき，$\mathsf{First}_(A) \backslash \{\varepsilon\} \cup \mathsf{First}_1(\mathsf{fol}(\alpha))$

(b) $A \to \beta \in P$ に対して $\mathsf{First}_1(\beta)$

定義 4.3. *CF* 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して，$A \in N$ に対する $\mathsf{Follow}_1(A)$ は以下のように定義される．

$$\begin{aligned} \mathsf{Follow}_1(A) \quad = \quad & \{a \in \Sigma \mid S \Rightarrow^* \beta A a \gamma, \beta, \gamma \in (N \cup \Sigma)^*\} \\ & \cup \{\# \mid S \Rightarrow^* \beta A, \beta \in (N \cup \Sigma)^*\} \end{aligned}$$

ここで，# は入力の終わりを示す記号である．

アルゴリズム 4.2.

ステップ 1: $\mathsf{Follow}_1(S) \leftarrow \{\#\}$

ステップ 2: $A \in N \backslash \{S\}$ に対して $\mathsf{Follow}_1(A) \leftarrow \varnothing$

ステップ 3: $B \to \beta C \gamma \in P$ を満たす $B, C \in N$, $\beta, \gamma \in (N \cup \Sigma)^*$ に対してすべての $\mathsf{Follow}_1(A)$*(*$A \in N$*)* に新たな追加がなくなるまで以下を追加する．

(a) $\varepsilon \notin \mathsf{First}_1(\gamma)$ のとき，$\mathsf{First}_1(\gamma)$

(b) $\varepsilon \in \mathsf{First}_1\gamma$ のとき，$\mathsf{First}_1(\gamma) \backslash \{\varepsilon\} \cup \mathsf{Follow}_1(B)$

4.2　下降型構文解析

4.2.1　左再帰性

手続き宣言の BNF の一部を以下に再掲する．

$$
\begin{array}{rcl}
\langle \mathsf{proc_decl} \rangle & ::= & \text{`}\texttt{procedure}'\ \text{`}\texttt{IDENT}'\ \text{`}(\text{'}\ \langle \mathsf{id\_list} \rangle\ \text{`})\text{'}\ \text{`}:\text{'}\ \langle \mathsf{inblock} \rangle \\
\langle \mathsf{id_list} \rangle & ::= & \langle \mathsf{id_list} \rangle\ \text{`},\text{'}\ \text{`}\texttt{IDENT}' \\
& | & \text{`}\texttt{IDENT}'
\end{array}
$$

$\langle \mathsf{proc_decl} \rangle$からの最左導出の第一段階は$\langle \mathsf{proc_decl} \rangle \Rightarrow$ `procedure' `IDENT' `(' $\langle \mathsf{id\_list} \rangle$ `)' `:' $\langle \mathsf{inblock} \rangle$ であり，次に$\langle \mathsf{id\_list} \rangle$に対して導出が適用される．ここで，$\langle \mathsf{id\_list} \rangle ::= \langle \mathsf{id\_list} \rangle$`,'`IDENT' は左再帰的 (left-recursive) と呼ばれる．右辺の最初に左辺と同じ非終端記号を含むので

$$
\begin{array}{rcl}
\langle \mathsf{id_list} \rangle & \Rightarrow_{lm} & \langle \mathsf{id_list} \rangle\text{`},\text{'}\text{`}\texttt{IDENT}' \\
& \Rightarrow_{lm} & \langle \mathsf{id_list} \rangle\text{`},\text{'}\text{`}\texttt{IDENT}'\text{`},\text{'}\text{`}\texttt{IDENT}' \\
& \Rightarrow_{lm} & \langle \mathsf{id_list} \rangle\text{`},\text{'}\text{`}\texttt{IDENT}'\text{`},\text{'}\text{`}\texttt{IDENT}'\text{`},\text{'}\text{`}\texttt{IDENT}' \\
& & \vdots
\end{array}
$$

のように同じルールが際限なく適用されて$\langle \mathsf{id_list} \rangle$が消えない可能性がある．$\langle \mathsf{id_list} \rangle \rightarrow$ `IDENT' を適用すればこの導出を止めることができるが，`IDENT' $\in \mathsf{First}_1(\langle \mathsf{id_list} \rangle)$ であるので，文形式の左端によっては別の規則を選択することができない．

左再帰性除去

$\{A \rightarrow A\alpha, A \rightarrow \beta\}$ という左再帰の規則を含む生成規則に対して，以下の生成規則集合は A から同じ文形式への導出を持ち，左再帰性を持つ規則を含まない．

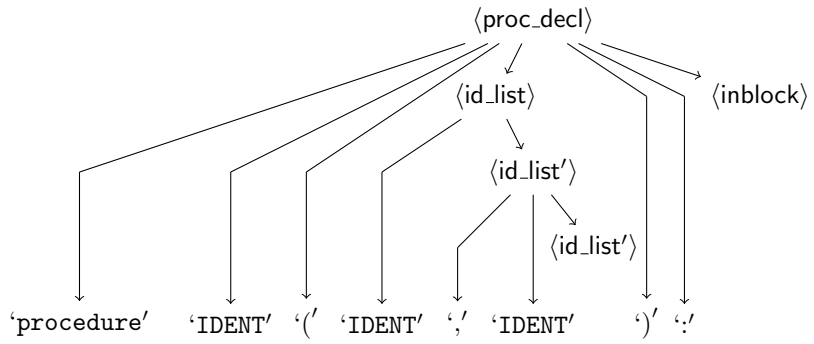
$$
\begin{array}{rcl}
A & \rightarrow & \beta A' \\
A' & \rightarrow & \alpha A' \\
A' & \rightarrow & \varepsilon
\end{array}
$$

A' は新たな非終端記号である．

これに従って$\langle \mathsf{id_list} \rangle$に対して左再帰性を除去すると以下のようになる．

$$
\begin{array}{rcl}
\langle \mathsf{id_list} \rangle & ::= & \text{`}\texttt{IDENT}'\ \langle \mathsf{id_list}' \rangle \\
\langle \mathsf{id_list}' \rangle & ::= & \text{`},\text{'}\ \text{`}\texttt{IDENT}'\ \langle \mathsf{id_list}' \rangle \\
& | & \varepsilon
\end{array}
$$

この場合の解析木は以下のようになる．



例 4.3. 加算+と乗算*を持つ式に対して，左再帰性をもつ生成規則と E からの導出が等しい左再帰性を除去しｈた生成規則は以下のようになる．

$$\begin{aligned} E &\to E\,\texttt{+}\,T \mid T \\ T &\to T\,\texttt{*}\,F \mid F \\ F &\to \texttt{(}\,E\,\texttt{)} \mid \text{`}\texttt{IDENT}' \end{aligned}$$

(a) 左再帰性を持つ生成規則

$$\begin{aligned} E &\to TE' \\ E' &\to \texttt{+}TE' \mid \varepsilon \\ T &\to FT' \\ T' &\to \texttt{*}FT' \mid \varepsilon \\ F &\to \texttt{(}\,E\,\texttt{)} \mid \text{`}\texttt{IDENT}' \end{aligned}$$

(b) 左再帰性を除去した生成規則

4.2.2　左括り出し

以下の BNF 定義において

$$\begin{aligned} \langle \text{conditional} \rangle \;::=\; & \text{`}\texttt{if}' \langle \text{expression} \rangle \text{`}\texttt{then}' \langle \text{statement} \rangle \text{`}\texttt{else}' \langle \text{statement} \rangle \text{`}\texttt{fi}' \\ \mid\; & \text{`}\texttt{if}' \langle \text{expression} \rangle \text{`}\texttt{then}' \langle \text{statement} \rangle \text{`}\texttt{fi}' \end{aligned}$$

予約語 ‘if′ を見ただけではどちらの規則を適用すればよいかわからない．さらに新たな非終端記号 ⟨conditional′⟩ を導入し，

$$\begin{aligned} \langle \text{conditional} \rangle \;::=\; & \text{`}\texttt{if}' \langle \text{expression} \rangle \text{`}\texttt{then}' \langle \text{statement} \rangle \langle \text{conditional}' \rangle \\ \langle \text{conditional}' \rangle \;::=\; & \text{`}\texttt{else}' \langle \text{statement} \rangle \text{`}\texttt{fi}' \\ \mid\; & \text{`}\texttt{fi}' \end{aligned}$$

とすれば，⟨conditional⟩ の規則は単一になり，⟨conditional′⟩ についても先読みトークンが ‘else′ か ‘fi′ かで決定的に規則を選択できる．

同じ左辺を持つ生成規則において，右辺の共通な左端をまとめることを**左括り出し** (left-factoring) という．一般には $A \to \alpha\beta_1 \mid \alpha\beta_2$ に対して，新たな非終端記号 A' を導入して $A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$ に変換することである．

4.2.3　LL 構文解析

必要に応じて左再帰性の除去，左括りだしを施した後，最左導出によって構文解析を行う．入力系列を先頭から読みながら最左導出を適用する構文解析は **LL(Left-to-right leftmost) 構文解析** と呼ばれる．

下向き構文解析表

最左導出において，開始記号から導出される文形式で最左に非終端記号 A を含む文形式は $\beta A\gamma$ の形であり，$\beta \in \Sigma^*$, $\gamma \in (N \cup \Sigma)^*$ である．CF 文法 G の生成規則のうち，左辺に A を持つ生成規則が以下のようになっているとする．

$$A \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$$

入力系列 w に対する解析木を構成する場合，$w = \beta\beta'$ であるとき $A \to \alpha_i$ による $\beta A\gamma$ の導出が w に到達するためには，以下の条件が成り立つ必要がある．

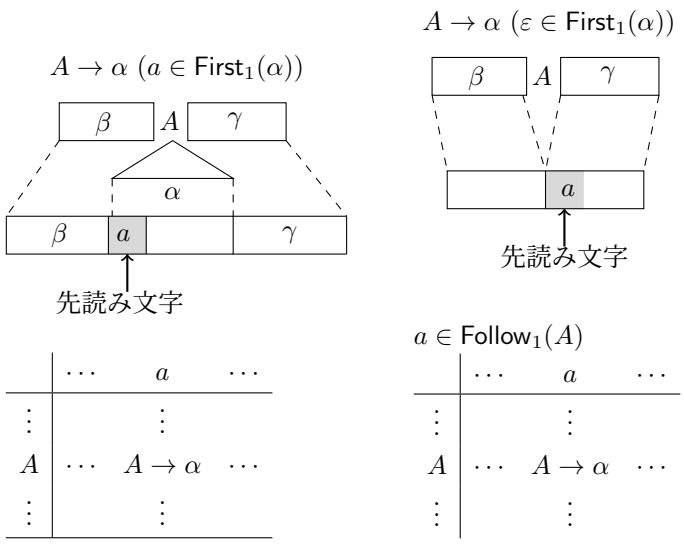
- $\beta' \neq \varepsilon$ かつ $\varepsilon \notin \mathsf{First}_1(\alpha_i)$ のとき $\mathsf{fst}(\beta') \in \mathsf{First}_1(\alpha_i)$
- $\beta' \neq \varepsilon$ かつ $\varepsilon \in \mathsf{First}_1(\alpha_i)$ のとき，$\mathsf{fst}(\beta') \in \mathsf{Follow}_1(A)$
- $\beta' = \varepsilon$ のとき，$\alpha_i = \varepsilon$ かつ $\# \in \mathsf{Follow}_1(A)$

最左導出に対する下向き構文解析表は，縦向きに非終端記号ごとの行，横向きに $\Sigma \cup \{\#\}$ ごとの列を持つ表であり、以下のように構成される．

$$A \to \alpha_1 \,|\, \alpha_2 \,|\, \cdots \,|\, \alpha_n$$

であるとき、

- $a \in \mathsf{First}_1(\alpha_i) \cap \Sigma$ ならば、A の行、a の列に $A \to \alpha_i$ を追加する。
- $\varepsilon \in \mathsf{First}_1(\alpha_i)$ ならば、A の行、$a \in \mathsf{Follow}_1(A)$ の列に $A \to \alpha_i$ を追加



LL(1) 構文解析

定義 4.4. *CF* 文法 G に対する下向き構文解析表において、表のエントリの生成規則が高々 1 つであるとき、G は **LL(1) 構文解析可能**である。このとき、G は **LL(1) 文法**である。

下向き構文解析表は，最左導出可能なすべての生成規則を数え上げる．したがって，CF 文法 G が LL(1) 構文解析可能であるとき、$w \in L(G)$ であるとき、かつ、そのときに限り、開始記号から w への決定的な生成規則の適用によって導出が存在する．明らかに下向き構文解析表以外の生成規則を適用しても導出を構成することはできない．

LL 構文解析は，構文木を最左導出で根方向から葉方向に構成する．先読み文字（トークン）以外の入力文字を直接参照することなく，適用可能な生成規則を正確に予測しながら構成するため**予測的構文解析**とも呼ばれる．

先読み文字を k 文字読んで導出の生成規則を決定できる CF 文法を LL(k) 文法という．2 文字以上先読みすると一般にはより適用する生成規則を限定することができる．しかし，一般的には 2 以上の k が使われることは少ない．

4.2.4　拡張 CF 文法と ELL(1) 文法

拡張 CF 文法は，生成規則の右辺に正規表現の選択と Kleene 閉包を導入して拡張した CF 文法である．この拡張によって CF 文法の表現力が増加することはない．この拡張によってプログラミング言語をよりコンパクトに定義することができるようになる．

アルゴリズム 4.3. 拡張 *CF* 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して等価な *CF* 文法 $G' = \langle N', \Sigma', P', S' \rangle$ を以下のようにして構成できる．P のすべての右辺が $(N \cup \Sigma)^*$ の要素になるまでステップ *1* を繰り返す．

ステップ *1:* $A \to e \in P$ のうち $e \notin (N \cup \Sigma)^*$ である生成規則を P から取り除く．

(a) $e = e_1 \,|\, e_2$ の場合：P に $A \to e_1$ と $A \to e_2$ に加える．

(b) $e = e_1 e_2$ の場合：新たな非終端記号 B を導入し，$A \to e_1 B$ と $B \to e_2$ を P に加え，N に B を追加する．

(c) $e = e_1^*$ の場合：新たな非終端記号 B を導入し，$A \to B$, $B \to e_1 B$, $B \to \varepsilon$ を P に追加し，N に B を追加する．

最終的に得られた N を N' とし，P を P' とする．

変換に基づいて得られる CF 文法に対して $\mathsf{First}_1(\alpha)$ を求めることができる．この First_1 を用いて拡張 CF 文法に対する First_1 を構成できる．

定義 4.5. 拡張 *CF* 文法の生成規則 $A \to e$ に対して

- $e = \varepsilon$ のとき，$\mathsf{First}_1(e) = \{\varepsilon\}$
- $e \in \Sigma$ のとき, $\mathsf{First}_1(e) = \{e\}$
- $e = e_1 \,|\, e_2$ のとき，$\mathsf{First}_1(e) = \mathsf{First}_1(e_1) \cup \mathsf{First}_1(e_2)$
- $e = e_1 e_2$ のとき，

  $\varepsilon \in \mathsf{First}_1(e_1)$ のとき，$\mathsf{First}_1(e) = \mathsf{First}_1(e_1) \backslash \{\varepsilon\} \cup \mathsf{First}_1(e_2)$

  $\varepsilon \notin \mathsf{First}_1(e_1)$ のとき，$\mathsf{First}_1(e) = \mathsf{First}_1(e_1)$
- $e = e_1^*$ のとき，$\{\varepsilon\} \cup \mathsf{First}_1(e_1)$

定義 4.6. $A \to e \in P$ のとき，$\mathsf{Follow}_1(e) = \mathsf{Follow}_1(A)$

$e = e_1 \,|\, e_2$ のとき，$\mathsf{Follow}_1(e_1) = \mathsf{Follow}_1(e)$, $\mathsf{Follow}_1(e_2) = \mathsf{Follow}_1(e)$

$e = e_1 e_2$ のとき，$\mathsf{Follow}_1(e_2) = \mathsf{Follow}_1(e)$

$\varepsilon \in \mathsf{First}_1(e_2)$ のとき，$\mathsf{Follow}_1(e_1) = \mathsf{First}_1(e_2) \backslash \{\varepsilon\} \cup \mathsf{Follow}_1(e)$

$\varepsilon \notin \mathsf{First}_1(e_2)$ のとき，$\mathsf{Follow}_1(e_1) = \mathsf{First}_1(e_2)$

$e = e_1^*$ のとき，$\mathsf{Follow}_1(e_1) = \mathsf{First}_1(e_1) \backslash \{\varepsilon\} \cup \mathsf{Follow}_1(e)$

Follow_1 は大きな項からその部分項へ計算される．

拡張 CF 文法に対しても CF 文法と同様に下向き構文解析表を構成することができる．

定義 4.7. 拡張 *CF* 文法 G に対する下向き構文解析表を構成したとき，表の各エントリーの生成規則が高々 1 つであるとき，*ELL(1)* 構文解析可能である．このとき，G は *ELL(1)* 文法である．

4.2.5 Javacc(JJTree) による降下型構文解析プログラムの生成

JJTree は，JavaCC のプリプロセッサである．jjt ファイルには，生成規則と字句解析の仕様を記述する．

JJTree のプログラムには，字句解析に用いるトークンの定義と生成規則を表すクラスメソッドを記述する．例えば，$E \to T(\texttt{+}T)^*$ といった生成規則は以下のようなクラスメソッドとして記述する．

```
void E() :
{}
{
T()
( <ADDOP> T())*
}
```

ここで，<ADDOP> は+のトークンである．生成規則の左辺をクラスメソッドとし，本体を右辺として記述する．JJTree は，下降型構文解析プログラムを生成する．

JJTree によるプログラム生成の流れを図 4.3 に示す．jjt ファイルを JJTree プロセッサに入力し，javacc ファイル (.jj ファイル) といくつかの java ファイルが生成される．さらに javacc で変換し，すべてのファイルを java ファイルに変換する．

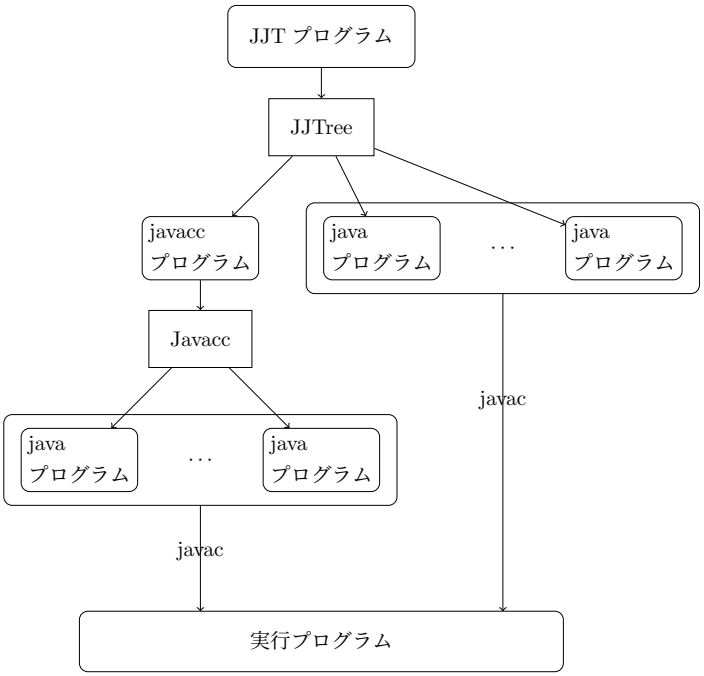


図 4.3: jjtree-config

4.2.6　例

（本内容の javacc コードは，早乙女健治 (著)：Javacc : コンパイラ・コンパイラ for Java (テクノプレス刊 2003) からの引用によるものです.）

以下の拡張 CF 文法で定義された四則演算式を構文解析する.

$$
\begin{aligned}
\langle \mathsf{AddExpr} \rangle &::= \langle \mathsf{MulExpr} \rangle ((+|-) \langle \mathsf{MulExpr} \rangle)* \\
\langle \mathsf{MulExpr} \rangle &::= \langle \mathsf{PrimExpr} \rangle ((*|/) \langle \mathsf{PrimExpr} \rangle)* \\
\langle \mathsf{PrimExpr} \rangle &::= \text{`(' } \langle \mathsf{AddExpr} \rangle \text{ `)' } | \text{ `}\mathtt{Literal}\text{'}
\end{aligned}
$$

```
/** simple calculator **/
options {
    STATIC=false;
    MULTI=true;
  }
PARSER_BEGIN(Calc)
public class Calc {
    public static void main(String args[]) throws ParseException {
        while (true) {
            Calc parser = new Calc(System.in);
```

```
            switch (parser.CalcUnit()) {
                case -1:
                return;
                case 0:
                continue;
                default:
                //       ((SimpleNode)parser.jjtree.rootNode()).dump("");
                parser.jjtree.rootNode().calculate();
              }
          }
      }
  }
PARSER_END(Calc)
SKIP:
{
" "|"\r"
}

TOKEN:
{
< ADDOP: "+" >
| < SUBOP: "-" >
| < MULOP: "*" >
| < DIVOP: "/" >
| < LP: "(" >
| < RP: ")" >
| < LITERAL: (["0"-"9"])* >
| < NL: "\n" >
}

int CalcUnit() :
{}
{
( AddExpr() <NL>
  { return 1; }
| <NL>
  { return 0; }
| <EOF>
  { return -1; }
)
```

```
}
void AddExpr() #void :
{}
{
MulExpr()
(
<ADDOP> MulExpr() #AddNode(2)
| <SUBOP> MulExpr() #SubNode(2)
)*
}
void MulExpr() #void :
{}
{
PrimExpr()
(
<MULOP> PrimExpr() #MulNode(2)
| <DIVOP> PrimExpr() #DivNode(2)
)*
}
void PrimExpr() #void :
{}
{
Literal()
| <LP> AddExpr() <RP>
}
void Literal() :
{ Token t; }
{
t=<LITERAL>
  { jjtThis.litValue = Integer.parseInt(t.image); }
}
```

AddExpr() 内の，#AddNode(2)，#SubNode(2) MulExpr() 内の，#MulNode(2)，#DivNode(2) はそれぞれ，四則演算子が 2 項演算なので，2 つノードが生成されれば，新たにノードを生成する効果を持つ.

4.3　上昇型構文解析

下降型構文解析は単純で効率的な構文解析手法ではあるが，CF 文法が実用的には LL(1) 文法である必要がある，LL(1) 言語に用いることのできる CF 文法の生成規則の適用と意図する構文木の

対応が必ずしも明確ではないなどの特徴があるため，比較的シンプルで構文と意味の間に柔軟性がある言語の解析に向いている．下降型では，入力系列を読み取ることなく，構文木の根の方向から適用する生成規則を限られた先読み文字によって決定することができなければならない．

これに対して，生成規則の右辺を入力および入力から導出の逆動作 (**還元**) で得られた文形式に対してマッチングすることで，左辺に置き換える還元を行って，開始記号まで還元することによって，逆方向から導出を構成する構文解析手法がある．これは，構文木の葉の方向から根を構成するので**上昇型構文解析**と呼ばれる．ここでは，上昇型構文解析の計算機構であるプッシュダウン変換機械 (pushdown transducer) を示し，この上での上昇型構文解析として LR 構文解析を示す．一般に用いられる LR(1) 構文解析は LL(1) 構文解析よりも適用できる CF 文法の範囲が大きい．Yacc などの構文解析プログラム生成ツールが対象としている LALR(1) 構文解析は LL(1) 構文解析とは適用できる CF 文法の範囲は比較できないが，左括りだしや左再帰性の除去などが必要なく，構文木を直接的に得られるためよく用いられる．LR 構文解析として，SLR(1) 構文解析手法を示したのち，その拡張として LR(1) 構文解析手法を示す．

4.3.1 プッシュダウン変換機械

プッシュダウン変換機械 (pushdown transducer, PDT) は，プッシュダウンオートマトンに出力テープを追加した計算モデルである．プッシュダウンオートマトン (PDA) はスタックへのプッシュ，ポップ動作によってスタック記号を容量制限のない記憶に格納することができる．プログラミング言語は文脈自由文法で定義されることから，その受理にはプッシュダウンオートマトンを用いる必要がある．さらに構文解析の際にはどの生成規則が使われたかを出力する必要があるので，出力テープを持つ PDT を用いる．下降型構文解析においては，入れ子構造をもつ手続き呼び出しがスタックの働きをしている．上昇型構文解析では，構文解析表に従って動作する PDT を構成する．

定義 4.8. *PDT* を $T = \langle \Gamma, \Sigma, O, \Delta, I_0, F \rangle$ として定義する．ここで

Γ	スタック記号の有限集合
Σ	入力記号の有限集合
O	出力記号の有限集合
Δ	遷移関数 $\Gamma^+ \times \Sigma \to \Gamma^* \times O^* \times \{0,1\}$
$I_0 \in \Gamma$	初期スタック記号
$F \subseteq \Gamma \cup \{\varepsilon\}$	最終スタック記号の集合

Σ には特別の記号 # を含む．# は入力テープの右端を示す希望である．(s, i, o) の 3 項組を時点表示 (configuration) という．スタック $s \in \Gamma^*$ はスタック記号の有限系列，入力テープ $I \in \Sigma^*$，出力テープ $o \in O^*$ とする．s は左側がスタックの底，右側がスタックトップを表す．遷移関数 $\Delta : \Gamma^+ \times \Sigma \to \Gamma^* \times O^* \times \{0,1\}$ によって時点表示を更新する．

- $\Delta(u, a) = (w, \pi, 1)$ のとき，遷移を $u \xrightarrow{a} (w, \pi)$ と書き，PDT T の時点表示の更新を $(vu, ax, o) \vdash (vw, x, o\pi)$ で表す．
- $\Delta(u, a) = (w, \pi, 0)$ のとき，遷移を $u \xrightarrow{(a)} (w, \pi)$ と書き，PDT T の時点表示の更新を $(vu, ax, o) \vdash (vw, ax, o\pi)$ で表す．

$u \xrightarrow{a} (w, \pi)$ を時点表示 (vu, ax, o) に適用するときには，T のスタックの上部 u を w に置き換え，出力テープに π を追加して，$o\pi$ とする．$u \xrightarrow{(a)} (w, \pi)$ を時点表示 (vu, ax, o) に適用するときには，スタックの上部 u を v に置き換えて，出力テープに π を追加して $o\pi$ とする．このとき，入力 a は読み込まない．遷移を行うためには入力テープの先頭は a である必要があるが，入力テープのポインタは動かさない．この動作を**先読み**といい，a を**先読み記号**という．

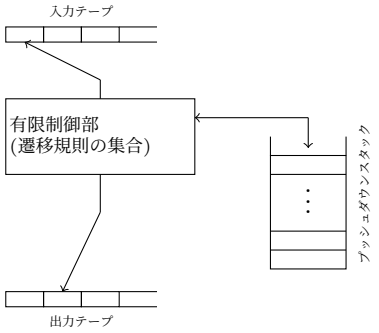


図 4.4: プッシュダウン変換機械 (PDT)

4.3.2　LR 構文解析

CF 文法の上昇型構文解析器 (bottom-up parser) は，入力として $x\# \in \Sigma^*$ を受取り，$x\#$ から導出の逆関係 $\Leftarrow$ を用いて，入力の左側から**還元** (reduction) により開始記号 S を得る．

$$w = \alpha_n \Leftarrow \alpha_{n-1} \Leftarrow \cdots \Leftarrow \alpha_1 \Leftarrow \alpha_0 = S$$

ここで，$\beta\alpha\gamma \Leftarrow \beta A\gamma$ となる生成規則 $A \to \alpha$ が存在する．α_0 においては，$\beta = \varepsilon$ である．上記の還元の系列は，逆方向から見ると最右導出になっている．例えば $\alpha_1 = AB \Leftarrow S$ であるとき，$\alpha_2 = A\beta \Leftarrow AB$ である．$A \to \alpha$ による還元が $\alpha_{i+1} = \beta\alpha\gamma \Leftarrow \beta A\gamma = \alpha_i$ であるとき，α を α_{i+1} の**ハンドル** (handle) という．α_i から決定的にハンドルを見つけて還元することが効率的な構文解析を構成するために必要である．α_i におけるハンドルが α である場合，$\alpha_i = \beta\alpha\gamma$, $\beta \in \Sigma^*$ であり，β は入力 w の接頭語になっている．

上昇型構文解析器を構成するとき，CF 文法 $G = \langle N, \Sigma, P, S\rangle$ から**拡大文法** (augmented grammar) $G' = \langle N \cup \{S'\}, \Sigma \cup \{\#\}, P \cup \{S' \to S\#\}, S'\rangle$ を構成する．ここで，S' は新たな非終端記号であり，$\#$ は終端記号である．拡大文法における上昇型構文解析では $S' \to S\#$ による還元か，ハンドルが見つからない場合に終了する．前者の場合，最右導出が構成できて $w \in L(G)$ が示され，後者の場合は，$w \notin L(G)$ であり，構文エラーである．

例 4.4. 文法 $\langle\{i, n, +, *, (,)\}, \{E\}, P, E\}$ に対する拡大文法は $\langle\{i, n, +, *, \#, (,)\}, \{E, S'\}, P \cup \{S' \to E\#\}, S'\rangle$ となる．ここで，

$$P = \left\{ \begin{array}{ll} 0: \quad S' \to E\# & 3: \quad E \to (\,E\,) \\ 1: \quad E \to E + E & 4: \quad E \to i \\ 2: \quad E \to E * E & 5: \quad E \to n \end{array} \right\}$$

入力系列を *(i+n)+i*i#* とする．

$$
\begin{aligned}
S' &\Rightarrow_{rm} E\# \\
&\Rightarrow_{rm} E+E\# \\
&\Rightarrow_{rm} E+E*E\# \\
&\Rightarrow_{rm} E+E*i\# \\
&\Rightarrow_{rm} E+i*i\# \\
&\Rightarrow_{rm} (E)+i*i\# \\
&\Rightarrow_{rm} (E+E)+i*i\# \\
&\Rightarrow_{rm} (E+n)+i*i\# \\
&\Rightarrow_{rm} (i+n)+i*i\#
\end{aligned}
$$

これを還元によって受理する *PDT* の動作は以下のようになる．

$$
\begin{aligned}
&(I_0, (i+n)+i*i\#, \varepsilon) \\
\vdash\ &(I_0(, \quad i+n)+i*i\#, \quad \varepsilon) \\
\vdash\ &(I_0(i, \quad +n)+i*i\#, \quad \varepsilon) \\
\vdash\ &(I_0(E, \quad +n)+i*i\#, \quad 4) \\
\vdash\ &(I_0(E+, \quad n)+i*i\#, \quad 4) \\
\vdash\ &(I_0(E+n, \quad)+i*i\#, \quad 4) \\
\vdash\ &(I_0(E+E, \quad)+i*i\#, \quad 45) \\
\vdash\ &(I_0(E+E), \quad +i*i\#, \quad 45) \\
\vdash\ &(I_0(E), \quad +i*i\#, \quad 451) \\
\vdash\ &(I_0E, \quad +i*i\#, \quad 4513) \\
\vdash\ &(I_0E+, \quad i*i\#, \quad 4513) \\
\vdash\ &(I_0E+i, \quad *i\#, \quad 4513) \\
\vdash\ &(I_0E+E, \quad *i\#, \quad 45134) \\
\vdash\ &(I_0E+E*, \quad i\#, \quad 45134) \\
\vdash\ &(I_0E+E*i, \quad \#, \quad 45134) \\
\vdash\ &(I_0E+E*E, \quad \#, \quad 451344) \\
\vdash\ &(I_0E+E, \quad \#, \quad 4513442) \\
\vdash\ &(I_0E, \quad \#, \quad 45134421) \\
\vdash\ &(I_0E\#, \quad \varepsilon, \quad 45134421) \\
\vdash\ &(acc, \quad \varepsilon, \quad 451344210)
\end{aligned}
$$

定義 4.9. 最右導出 $S' \Rightarrow^*_{rm} \alpha Aw \Rightarrow_{rm} \alpha\beta w$ *(*ここで $w \in \Sigma\{\#\}$*)* において，αA の接頭語および $\alpha\beta$ の接頭語を G' の**可延頭部** *(viable prefix)* という．

可延頭部は，上昇型構文解析において構文解析が進んでいる状態を示している．

SLR(1) 構文解析

定義 4.10. *CF* 文法 $G = \langle N, \Sigma, P, S\rangle$ とする．$A \to \alpha \in P$ *(*$\alpha \in (N \cup \Sigma)^*$*)* に対して，$\alpha$ の i 番目の記号を $\alpha(i) \in N \cup \Sigma$ で表す．また，α の接頭語で長さが $k(\geqslant 0)$ の系列を $\alpha(:k)$, α の接尾語で

長さ $\ell(\geqslant 0)$ の系列を $\alpha(\ell:)$ で表すこととする．$k=0, \ell=0$ のときはそれぞれ ε とする．各生成規則 $A \to \alpha$ に対する *LR(0)* 項を以下のような形を持つ項として定義する．

$$[\, A \to \alpha(:k) \bullet \alpha(\ell:) \,]$$

ここで，$|\alpha| = k + \ell$ である．

G から得られる *LR(0)* 項の集合を $LR(0)_G$ と書く．

$A \to \alpha \in P$ に対して，$|\alpha| = m$ のとき，$m+1$ 個の LR(0) 項が存在する．

例 4.5. 文法 $G_{Arith} = \langle \{E, T, F\}, \{i, +, *, (,)\}, P, E \rangle$ とする．ここで，

$P = \left\{ \begin{array}{l} E \to E+T \mid T, \\ T \to T*F \mid F, \\ F \to i \mid (\, E \,) \end{array} \right\}$ とする．P から得られる *LR(0)* は以下のようになる．

$$\begin{array}{llll}
[E \to \bullet E+T] & [E \to E \bullet +T] & [E \to E+ \bullet T] & [E \to E+T \bullet] \\
[E \to \bullet T] & [E \to T \bullet] & & \\
[T \to \bullet T*F] & [T \to T \bullet *F] & [T \to T* \bullet F] & [T \to T*F \bullet] \\
[T \to \bullet F] & [T \to F \bullet] & & \\
[F \to \bullet (E)] & [F \to (\bullet E)] & [F \to (E \bullet)] & [F \to (E) \bullet] \\
[F \to \bullet i] & [F \to i \bullet] & &
\end{array}$$

定義 4.11. 拡大文法の開始記号を S' とする．$A \to \alpha_1\alpha_2$ を生成規則に含む．$S' \Rightarrow_{rm}^{*} \beta A w \Rightarrow_{rm} \beta\alpha_1\alpha_2 w$ となる導出が存在するとき，$[A \to \alpha_1 \bullet \alpha_2]$ を可延頭部 $\beta\alpha_1$ に対する正準 *LR(0)* 項と呼ぶ．可延頭部 $\beta\alpha_1$ に対する正準 *LR(0)* 項の集合を $I_0(\beta\alpha_1)$ と書く．

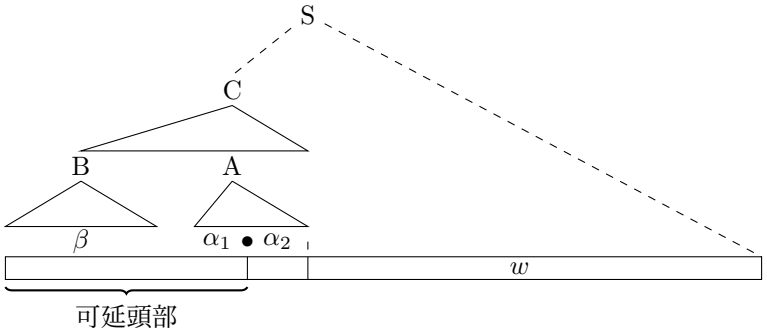


図 4.5: 導出と可延頭部

LR 構文解析は，生成規則の右辺のマッチングによって構文木を構成する．LR(0) 項は • までその生成規則の右辺にマッチングしている状態を示す (図 4.5)．可延頭部 $\gamma \in (N \cup \Sigma)^*$ に対する正準 LR(0) 項は，以下のような遷移系を定義することによって計算することができる．

定義 4.12. *CF* 文法 G に対して $LR(0)_G$ 上に以下の遷移関係 $\xrightarrow{M}$ を定義する．*($M \in N \cup \Sigma \cup \{\varepsilon\}$)*
$A \to \alpha \in P$ において

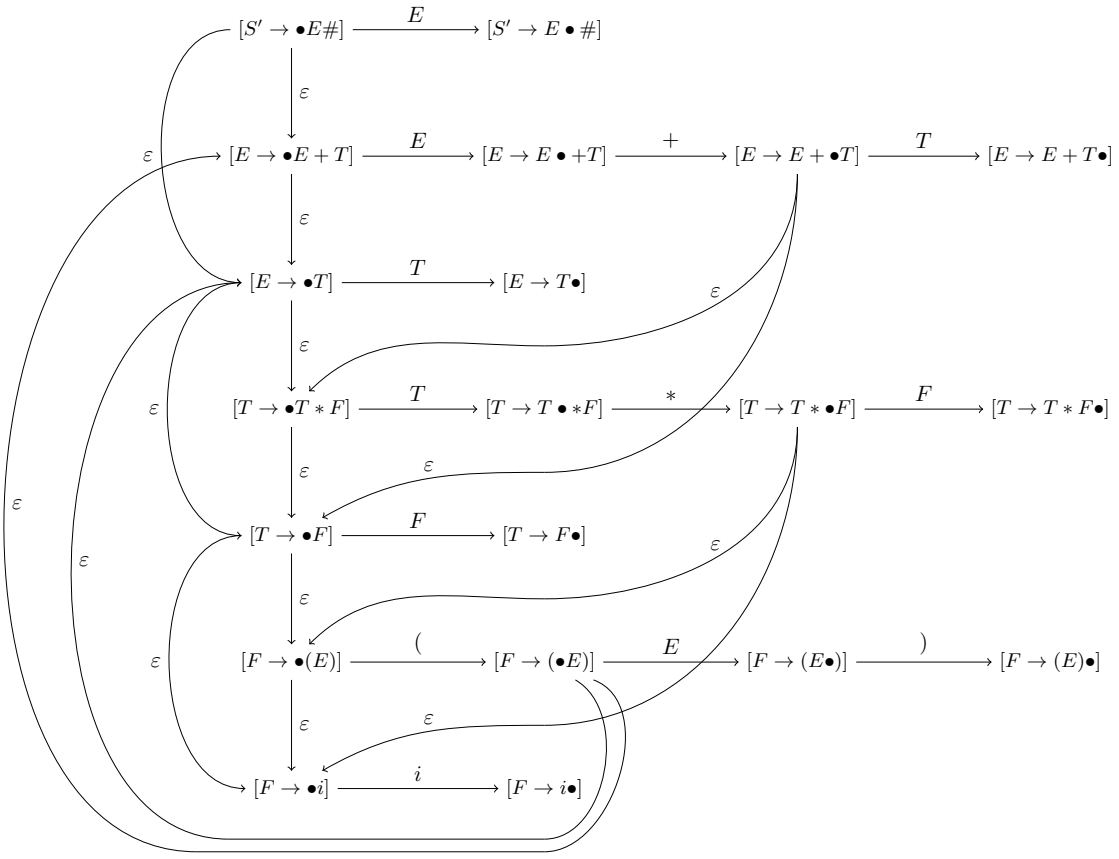


図 4.6: G_{Arith} に対する LR(0) 遷移

$a \in \Sigma$ のとき $[A \to \beta \bullet a\beta'] \xrightarrow{a} [A \to \beta a \bullet \beta']$

$X \in N$ のとき $[A \to \beta \bullet X\beta'] \xrightarrow{X} [A \to \beta X \bullet \beta']$ および
$X \to \gamma \in P$ に対して $[A \to \beta \bullet X\beta'] \xrightarrow{\varepsilon} [X \to \bullet\gamma]$

$\theta \in LR(0)_G$ に対して $\mathsf{act}(\theta) = \{M \mid \theta \xrightarrow{\varepsilon}{}^{*} \xrightarrow{M} \theta', M \neq \varepsilon$となる θ' が存在する $\}$
$I \subseteq LR(0)_G$ に対して $\mathsf{act}(I) = \cup_{\theta \in I}\mathsf{act}(\theta)$ とする.

例 4.6. 例 *4.5* の文法 G_{Arith} に対する遷移は図 *4.6* のようになる.

LR(0) 項を状態として, $\bullet$ が次のシンボルに移動する遷移系によって, 可延頭部が計算できる. 拡大文法を考えると $[S' \to \bullet S\#]$ から $[A \to \alpha_1 \bullet \alpha_2]$ への遷移

$$[S' \to \bullet S\#] \xrightarrow{M_1} \cdots \xrightarrow{M_n} [A \to \alpha_1 \bullet \alpha_2]$$

が存在するとき, 連接 $M_1 M_2 \cdots M_n$ が可延頭部となる. (M_i には ε も含まれる.)

定義 4.13. I を *LR(0)* 項の集合とする. I と $\alpha \in (N \cup \Sigma)^*$ に対して Goto 関数を以下のように定

義する．

$$\mathsf{Goto}(I,\alpha)=\begin{cases}\{\theta' \mid \theta \xrightarrow{\varepsilon}{}^{*}\ \theta\text{となる } \theta\in I \text{ が存在する }\} & \alpha=\varepsilon \text{ のとき}\\ \{\theta' \mid \theta \xrightarrow{M}\ \theta'\text{となる } \theta\in \mathsf{Goto}(I,\beta) \text{ が存在する }\} & \alpha=\beta M,\beta\in(N\cup\Sigma)^{*},\\ & M\in N\cup\Sigma \text{ のとき}\end{cases}$$

可延頭部に対する正準 LR(0) 項の集合は，LR(0) 遷移系を NFA とみなして，部分集合構成法を用いて決定的な遷移をもつ遷移系に変換することによって求めることができる．

アルゴリズム 4.4. 以下の手順に沿って構成される遷移系 $\Theta_0=(\mathcal{S}_0,\mathcal{E}_0)$ を**正準 LR(0) 遷移系**とよぶ．$\mathcal{S}_0$ は Θ_0 の状態，$\mathcal{E}_0$ は Θ_0 の辺である．

ステップ 1*:* $I_0=\mathsf{Goto}([S'\to\bullet S\#],\varepsilon)$ とする．

ステップ 2*:* $\Theta_0\leftarrow(\{I_0\},\varnothing)$*;*$T\leftarrow\{I_0\}$

ステップ 3*:* $T=\varnothing$ ならば終了する．Θ_0 に正準 *LR(0)* 遷移系が得られる．

ステップ 4*:* $I\in T$ を選び，T から削除する．

ステップ 5*:* すべての $M\in\mathsf{act}(I)$ に対して，$I'=\{\theta' \mid \theta\in I,\theta \xrightarrow{\varepsilon}{}^{*}\xrightarrow{M}\xrightarrow{\varepsilon}{}^{*}\theta'\}$ を構成する．

$I'\notin\mathcal{S}$ ならば，T に I' を $\mathcal{S}$ に I' を追加し，$\mathcal{E}$ に $I\xrightarrow{M}I'$ を追加する．

$I'\in\mathcal{S}$ ならば，$\mathcal{E}$ に $I\xrightarrow{M}I'$ を追加する．

ステップ 6*:* ステップ *3* へもどる．

正準 LR(0) 遷移系は，(正準)LR(0) オートマトンと言われることもある．

命題 4.1. θ が可延頭部 α に対する正準 *LR(0)* 項であるとき，かつそのときに限り $\theta\in\mathsf{Goto}(I_0,\alpha)$ である．

例 4.7. G_{Arith} の正準 *LR(0)* 遷移系は図 *4.7* のようになる．

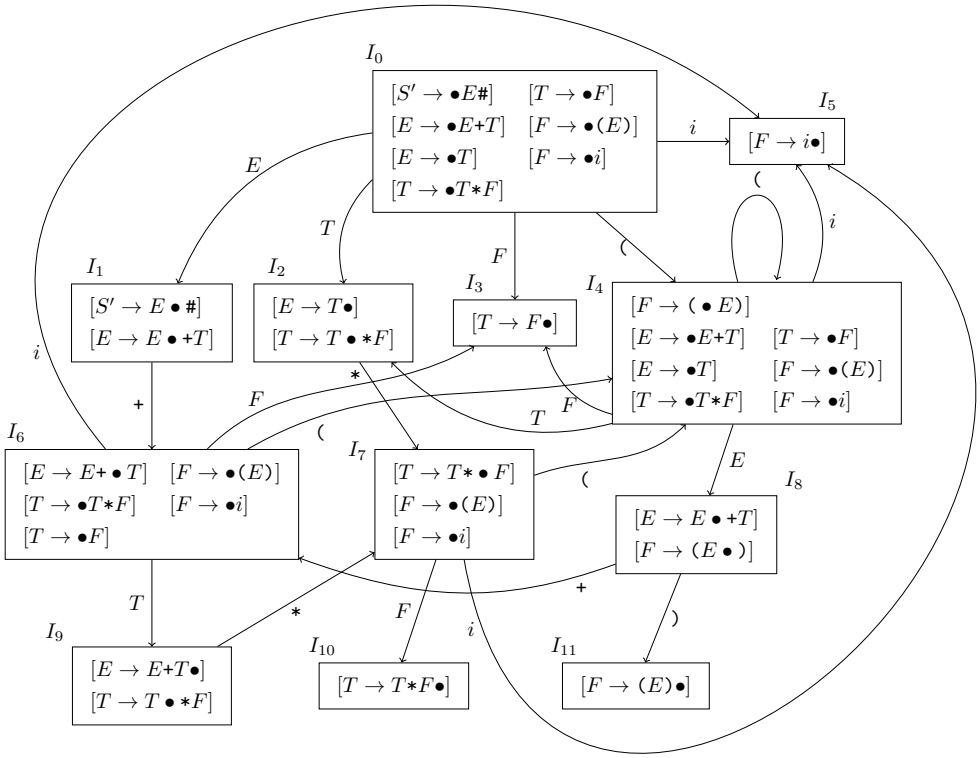


図 4.7: G_{Arith} の LR(0) 遷移系

LR(0) 構文解析　正準 LR(0) 遷移系を PDT 上で動作させることによって LR(0) 構文解析を行う．PDT のスタック上には常に次の形で記号を書き込む．ここで，I_i は LR(0) 遷移系の状態であり，M_i は $N \cup \Sigma$ の要素である．

$$I_0 M_1 I_1 M_2 \cdots M_n I_n$$

スタックトップから $M_{i+1}, \cdots, M_n$ を抽出して，生成規則の右辺とマッチさせる．マッチングが成功している場合，LR(0) 項は，$[A \to M_{i+1} \cdots M_n \bullet]$ の形の LR(0) 項によって**還元**され，スタックの内容を A でおきかえる．これは A の遷移が可能になったことを示すので，M_1 の前にある状態から A の遷移で移動可能な状態 I をスタックにプッシュし，使われた生成規則の番号を出力テープに出力する $(*)$．これに対して，$[A \to \alpha\bullet]$ という状態に達していない場合は，M をスタックにプッシュして，移動した先の状態 I_i をプッシュする $(\dagger)$．$(*)$ の振舞いを**還元動作**，$(\dagger)$ の振舞いを**シフト動作**と呼ぶ．

還元　スタックが $I_0 X_1 I_1 \cdots X_n I_n$ であり，$[A \to X_{i+1} X_{i+2} \cdots X_n \bullet] \in I_n$ となる場合，スタックトップ (系列の右側) から $X_{i+1} I_{i+1} \cdots X_n I_n$ をポップして A をプッシュして，$I_0 X_1 \cdots I_i A I'_{i+1}$ とする．ここで，$I'_{i+1} = \mathsf{Goto}(I_i, A)$，$A \to X_{i+1} \cdots X_n$ の生成規則を r とする．

$$\begin{aligned} & (I_0 X_1 I_1 \cdots I_i X_{i+1} I_{i+1} \cdots X_n I_n, aw, \pi) \\ \vdash \quad & (I_0 X_1 I_1 \cdots I_i A I'_{i+1}, aw, \pi r) \end{aligned}$$

シフト $a \in \Sigma$, $[A \to X_{i+1} \cdots X_n \bullet a X_{n+1} \cdots X_m] \in I_n$ であるとき，スタックに aI_{n+1} ($I_{n+1} = \mathsf{Goto}(I_n, a)$) をプッシュする．

$$\begin{aligned} & (I_0 X_1 I_1 \cdots X_n I_n, aw, \pi) \\ \vdash \quad & (I_0 X_1 I_1 \cdots X_n I_n a I_{n+1}, w, \pi) \end{aligned}$$

受理 $[S' \to X\# \in I_n]$ かつ入力系列が # であるとき，受理する．

$$\begin{aligned} & (I_0 S I_1, \#, \pi) \\ \vdash \quad & (acc, \varepsilon, \pi 0) \end{aligned}$$

受理動作は，規則 $0(S' \to S\#)$ による還元動作である．この場合，スタックに特別な記号 acc を書き込む．

エラー 還元，シフト，受理のいずれもが不可能である場合，構文エラーとなり停止する．

I_0 のスタックから開始して，$I_0 X_1 I_1 \cdots X_i I_i$ となっている場合，$I_i = \mathsf{Goto}(I_0, X_1 \cdots X_i)$ であり，$\theta \in I_i$ は，可延頭部 $X_1 \cdots X_i$ に対する LR(0) 項である．同じ可延頭部による正準 LR(0) 項の集合の全体を**正準 LR(0) 集成** (cannonical collection of LR(0) items) とよぶ．

これらの動作は以下のような PDT で実現される．

アルゴリズム 4.5.

ステップ 1: 正準 *LR(0)* 遷移系 $(\mathcal{S}, \mathcal{E})$ を構成し，*LR(0)* 項 $[S' \to \bullet S\#]$ を含む $\mathcal{S}$ の要素を初期状態 I_0 とする．

ステップ 2: $[A \to X_1 \cdots X_n \bullet] \in I \in \mathcal{S}$, $a \in \Sigma$ に対して次の遷移規則を追加する．

$$(I X_1 I_1 \cdots X_n I_n, a) \xrightarrow{(a)} (I A I', r)$$

ここで $I \xrightarrow{X_1} I_1, I_i \xrightarrow{X_i} I_{i+1}, I \xrightarrow{A} I' \in \mathcal{E}$ であり，r は $A \to X_1 \cdots X_n$ の生成規則の番号である．

ステップ 3: $I \xrightarrow{M} I' \in \mathcal{E}$ に対して $\theta = [A \to X_1 \cdots X_i \bullet a X_{i+1} \cdots X_n] \in I$, $\theta' = [A \to X_1 \cdots X_i a \bullet X_{i+1} \cdots X_n] \in I'$ のとき次の遷移規則を追加する．*($a \in \Sigma$)*

$$(I X_1 I_1 \cdots X_i I, a) \xrightarrow{a} (I X_1 I_1 \cdots X_i I a I', \varepsilon)$$

ここで $I \xrightarrow{X_1} I_1, I_i \xrightarrow{X_{i+1}} I_{i+1}, I \xrightarrow{a} I' \in \mathcal{E}$ である．

ステップ 4: $[S' \to S \bullet \#] \in I \in \mathcal{S}$ に対して次の遷移規則を追加する．

$$(I' S I, \#) \xrightarrow{\#} (acc, 0)$$

この PDT を $PDT_{LR0(G)}$ と書くことにする．

命題 4.2. *CF* 文法 G に対して，拡大文法を G' とする．$w \in L(G)$ であるとき，かつ，そのときに限り，$PDT_{LR0(G')}$ は $w\#$ を受理する．

定義 4.14. *CF* 文法 G の拡大文法を G' とする．*CF* 文法 G による $w \in \Sigma^*$ に対する *LR(0)* 構文解析は，$PDT_{LR0(G')}$ において，$(I, w\#, \varepsilon)$ からの $\vdash$ による計算である．$PDT_{LR0(G')}$ の遷移関数 Δ において，$\Delta(u, a)$ の数が任意の u, a に対してたかだか *1* であるとき，**LR(0) 構文解析可能**である．*LR(0)* 構文解析可能な *CF* 文法を **LR(0) 文法**という．

LR(0) 文法はそのままではあまり実用的ではない．G_{Arith} の LR(0) 遷移系において，I_2 の状態は $[E \to T\bullet]$ を含み，$[E \to T \bullet *F]$ も含むことから，PDT は入力記号 $*$ に対して，還元とシフトに相当する遷移が可能である．(すなわち G_{Arith} は LR(0) 文法ではない．)

$(u, a) \xrightarrow{(a)} (AI, \pi)$ となる (AI, π) が複数ある場合，**Reduce/Reduce コンフリクト**，$(u, a) \xrightarrow{(a)} (AI, \pi)$ と $(u, a) \xrightarrow{a} (uaI, a)$ が両方存在する場合 **Shift/Reduce コンフリクト**があるという．

先読み文字の機能を導入すれば，I_2 において $[E \to T\bullet]$ からの還元動作で受理に至る可能性があるのは，$\mathsf{Follow}_1(E)$ に先読み文字が含まれている場合である．先読み文字が $*$ であれば，スタックに $+I_7$ を追加するシフト動作を行う．G_{Arith} において，$\mathsf{Follow}_1(E) = \{\#, +,)\}$ であるので，PDT の動作が定まる．

アルゴリズム 4.5 の PDT の構成を以下のように修正する．

> **ステップ 2':** $[A \to X_1 \cdots X_n\bullet] \in I \in \mathcal{S}$ に対して $a \in \mathsf{Follow}_1(A)$ に対して次の遷移規則を追加する．
>
> $$(IX_1I_1 \cdots X_nI_n, a) \xrightarrow{(a)} (IAI', r)$$
>
> ここで $I \xrightarrow{X_1} I_1, I_i \xrightarrow{X_i} I_{i+1}, I \xrightarrow{A} I' \in \mathcal{E}$ であり，r は $A \to X_1 \cdots X_n$ の生成規則の番号である．

この修正を適用して得られる PDT を $PDT_{SLR(1)}$ と書く．

定義 4.15. *CF* 文法 G の拡大文法を G' とする．*CF* 文法 G による $w \in \Sigma^*$ に対する *SLR(1)* 構文解析は，$PDT_{SLR1(G')}$ において，$(I, w\#, \varepsilon)$ からの $\vdash$ による計算である．$PDT_{SLR1(G')}$ の遷移関数 Δ において，$\Delta(u, a)$ の数が任意の u, a に対してたかだか *1* であるとき，**SLR(1) 構文解析可能**である．*SLR(1)* 構文解析可能な *CF* 文法を **SLR(1) 文法**という．

例 4.8. G_{Arith} に対する *PDT* は図 *4.8* のように定義される．

Shift			Reduce		
			I_0EI_1	$\rightarrow^{(\#)}$	(Acc, ε)
			$I_0EI_1 + I_6TI_9$	$\rightarrow^{(\#,+,))}$	$(I_0EI_1, 1)$
			I_0TI_2	$\rightarrow^{(\#,+,))}$	$(I_0EI_1, 2)$
			$I_0TI_2 * I_7FI_{10}$	$\rightarrow^{(\#,*,+,))}$	$(I_0TI_2, 3)$
I_0	$\rightarrow^{(}$	$(I_0(I_4, \varepsilon)$	I_0FI_3	$\rightarrow^{(\#,*,+,))}$	$(I_0TI_2, 4)$
I_0	$\rightarrow^{i}$	(I_0iI_5, ε)	$I_0(I_4EI_8)I_{11}$	$\rightarrow^{(\#,*,+,))}$	$(I_0FI_3, 5)$
I_1	$\rightarrow^{+}$	$(I_1 + I_6, \varepsilon)$	I_0iI_5	$\rightarrow^{(\#,*,+,))}$	$(I_0FI_3, 6)$
I_2	$\rightarrow^{*}$	$(I_2 * I_7, \varepsilon)$	$I_4EI_8 + I_6TI_9$	$\rightarrow^{(\#,+,))}$	$(I_4EI_8, 1)$
I_4	$\rightarrow^{(}$	$(I_4(I_4, \varepsilon)$	I_4TI_2	$\rightarrow^{(\#,+,))}$	$(I_4EI_8, 2)$
I_6	$\rightarrow^{i}$	(I_6iI_5, ε)	$I_4TI_2 * I_7FI_{10}$	$\rightarrow^{(\#,*,+,))}$	$(I_4TI_2, 3)$
I_7	$\rightarrow^{i}$	(I_7iI_5, ε)	I_4FI_3	$\rightarrow^{(\#,*,+,))}$	$(I_4TI_2, 4)$
I_7	$\rightarrow^{(}$	$(I_7(I_4, \varepsilon)$	$I_4(I_4EI_8)I_{11}$	$\rightarrow^{(\#,*,+,))}$	$(I_4FI_3, 5)$
I_8	$\rightarrow^{+}$	$(I_8 + I_6, \varepsilon)$	I_4iI_5	$\rightarrow^{(\#,*,+,))}$	$(I_4FI_3, 6)$
I_8	$\rightarrow^{)}$	$(I_8)I_{11}, \varepsilon)$	$I_6TI_9 * I_7FI_{10}$	$\rightarrow^{(\#,*,+,))}$	$(I_6TI_9, 3)$
I_9	$\rightarrow^{*}$	$(I_9 * I_7, \varepsilon)$	I_6FI_3	$\rightarrow^{(\#,*,+,))}$	$(I_6TI_9, 4)$
			$I_6(I_4EI_8)I_{11}$	$\rightarrow^{(\#,*,+,))}$	$(I_6FI_3, 5)$
			I_6iI_5	$\rightarrow^{(\#,)*,+,))}$	$(I_6FI_3, 6)$
			$I_7(I_4EI_8)I_{11}$	$\rightarrow^{(\#,*,+,))}$	$(I_7FI_{10}, 5)$
			I_7iI_5	$\rightarrow^{(\#,*,+,))}$	$(I_7FI_{10}, 6)$

図 4.8: $PDT_{SLR1(G_{Arith})}$

SLR(1) 構文解析表　$PDT_{SLR1(G')}$ の遷移関数を構文解析表の形で書くことが多い．SLR(1) 構文解析表では，LR(0) 遷移系の状態と先読み文字による動作の対応 (action 表) と，非終端記号 A を還元した後の PDT の振舞い (GOTO 表) を示す．

	action				*GOTO*		
	$\cdots$	a	$\cdots$	#	$\cdots$	A	$\cdots$
I_0							
$\vdots$							
I_n							

図 4.9: SLR(1) 構文解析表の形

action 表では，スタックトップにある LR(0) 遷移系の状態 I で先読み文字 $a \in \Sigma \cup \{\#\}$ となったとき，

$(uI, a) \xrightarrow{(a)} (A, \pi)$ が Δ で定義されていれば，$r\pi$ を I の行の a の欄に追加する．

$(uI, a) \xrightarrow{a} (uIaI', \varepsilon)$ が Δ で定義されていれば，sI' を I の行の a の欄に追加する．

$r\pi$ は還元動作，sI はシフト動作を意味する．

GOTO 表では，$(uI, a) \xrightarrow{(a)} (AI', \pi)$ が Δ で定義されていれば，I' を I の行の A の欄に追加する．

構文解析表を構成して action 表にたかだか 1 つの動作しか記入されていなければ，SLR(1) 構文解析可能であり，その文法は SLR(1) 文法である．LR(0) 遷移系の構成から GOTO 表の欄にはたかだか 1 つの状態しか記入されない．

例 4.9. G_{Arith} に対する *SLR(1)* 構文解析表を図 *4.10* に示す.

	action						goto		
	i	$+$	$*$	$($	$)$	$\#$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

図 4.10: G_{Arith} の SLR(1) 構文解析表

LR(1) 構文解析

SLR(1) 構文解析は，上昇型構文解析として最も単純な解析手法である[2]．しかし，適用範囲はあまり大きくない．以下の CF 文法 $G_=$ は，SLR(1) 構文解析可能ではない.

$$G_= = \langle \{S, E, T\}, \{=, +, i\}, P_=, S \rangle$$

開始記号を S' とした拡大文法 $G'_=$ の生成規則は以下のようになる.

$$P'_= = \left\{ \begin{array}{llllll} 0: & S' \to S\# & 1: & S \to E = E & 2: & S \to i \\ 3: & E \to E + T & 4: & E \to T & 5: & T \to i \end{array} \right\}$$

$G'_=$ の正準 LR(0) 遷移系と SLR(1) 構文解析表は以下のようになる.

[2]SLR=Simple LR の意味

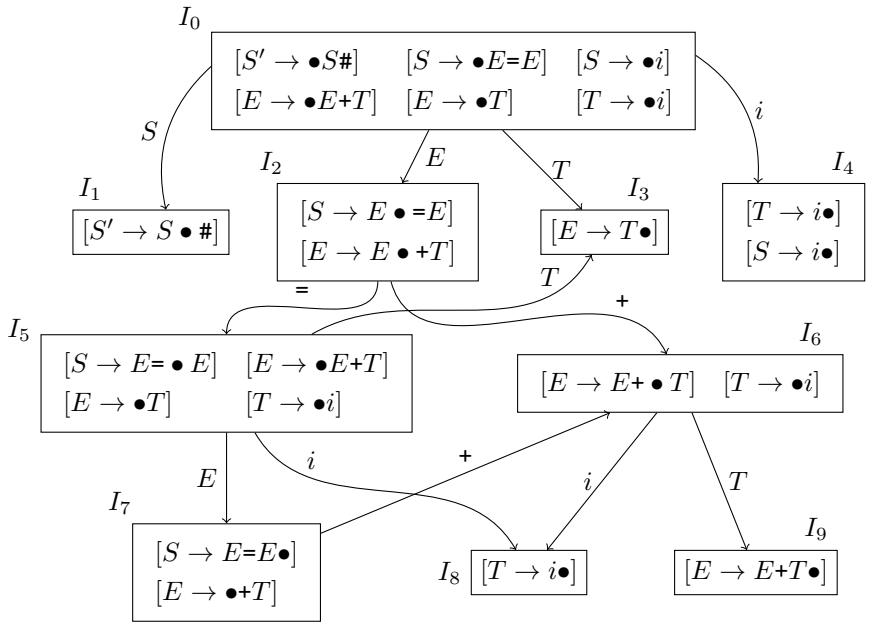


図 4.11: $G_=$ の正準 LR(0) 遷移系

	action				Goto		
	i	$=$	$+$	$\#$	S	E	T
0	$s4$				1	2	3
1				acc			
2		$s5$	$s6$				
3		$r4$	$r4$	$r4$			
4		$r5$	$r5$	$r5$			
				$r2$			
5	$s8$					7	3
6	$s8$						9
7			$s6$	$r1$			
8		$r5$	$r5$	$r5$			
9		$r3$	$r3$	$r3$			

図 4.12: $G_=$ の SRL(1) 構文解析表

状態 4 において reduce/reduce conflict が発生している．#が先読み文字のときに S に還元するか T に還元するかが定まらない．これを定めるためには，次に=があるのかどうかでどちらの規則で還元するかを定める必要がある．

このため，マッチング状態に含まれる文脈情報を増やした LR(1) 構文解析を考える．LR(0) 項

を構成する生成規則で還元されたときの次の文字を含めた LR(1) 項を考える．LR(1) 項に加える文字は，生成規則の左辺 A の $\mathsf{Follow}_1(A)$ のみで十分である．他の LR(1) 項は受理計算には現れない．(すべてエラー状態である．)

定義 4.16. *CF* 文法 G およびその拡大 *CF* 文法を G' とする．S' 以外の非終端記号 A に対する生成規則 $A \to \alpha$ に対する *LR(0)* 項 $[A \to \beta_1 \bullet \beta_2]$ および $a \in \Sigma \cup \{\#\}$ の組 $[A \to \beta_1 \bullet \beta_2, a]$ および，$[S' \to \beta_1 \bullet \beta_2, \varepsilon]$ を **LR(1) 項**という．ここで，S' は G'，S は G の開始記号である．

定義 4.17. 最右導出 $S' \Rightarrow_{rm}^{*} \alpha A w \Rightarrow \alpha\beta_1\beta_2 w$ が存在し，$a = \mathsf{First}_1(w)$ であるとき，$[A \to \beta_1 \bullet \beta_2, a]$ を可延頭部 $\alpha\beta_1$ に対する正準 *LR(1)* 項と呼ぶ．可延頭部 $\alpha\beta_1$ の集合を $I_1(\alpha\beta_1)$ と記述することにする．

定義 4.18. *(1)* $A \neq S'$ である $LR(1)$ 項 $[A \to \beta_1 \bullet a\beta_2, a]$ $(a \in \Sigma)$ に対して

$$[A \to \beta_1 \bullet a\beta_2, b] \xrightarrow{a} [A \to \beta_1 a \bullet \beta_2, b]$$

(2) $A \neq S'$ である $LR(1)$ 項 $[A \to \beta_1 \bullet X\beta_2, a]$ $(X \in N)$ に対して

$$[A \to \beta_1 \bullet X\beta_2, b] \xrightarrow{X} [A \to \beta_1 X \bullet \beta_2, b]$$
$$[A \to \beta_1 \bullet X\beta_2, b] \xrightarrow{\varepsilon} [X \to \bullet\gamma, c]$$

ここで，$X \to \gamma \in P$, $c \in \mathsf{First}(\beta_2 b)$

(3) $[S' \to \bullet S\#]$ に対して，

$$[S' \to \bullet S\#, \varepsilon] \xrightarrow{S} [S' \to S \bullet \#, \varepsilon]$$
$$[S' \to \bullet S\#, \varepsilon] \xrightarrow{\varepsilon} [S \to \bullet\gamma, \#]$$

アルゴリズム 4.4 と同様に正準 LR(1) 遷移系を構成する．

アルゴリズム 4.6. 以下の手順に沿って構成される遷移系 $\Theta_1 = (\mathcal{S}_1, \mathcal{E}_1)$ を**正準 LR(1) 遷移系**とよぶ．$\mathcal{S}_1$ は Θ_1 の状態，$\mathcal{E}_1$ は Θ_1 の辺である．

ステップ 1*:* $I_1 = \mathsf{Goto}([S' \to \bullet S\#, \varepsilon], \varepsilon)$ とする．

ステップ 2*:* $\Theta \leftarrow (\{I_1\}, \varnothing)$;$T \leftarrow \{I_1\}$

ステップ 3*:* $T = \varnothing$ ならば終了する．Θ_1 に正準 *LR(1)* 遷移系が得られる．

ステップ 4*:* $I \in T$ を選び，T から削除する．

ステップ 5*:* すべての $M \in \mathsf{act}(I)$ に対して，$I' = \{\theta' \mid \theta \in I, \theta \xrightarrow{\varepsilon}{}^{*} \xrightarrow{M} \xrightarrow{\varepsilon}{}^{*} \theta'\}$ を構成する．

$I' \notin \mathcal{S}_1$ ならば，T に I' を $\mathcal{S}_1$ に I' を追加し，$\mathcal{E}$ に $I \xrightarrow{M} I'$ を追加する．

$I' \in \mathcal{S}$ ならば，$\mathcal{E}$ に $I \xrightarrow{M} I'$ を追加する．

ステップ 6*:* ステップ *3* へもどる．

LR(1) 項に対する Goto 関数は定義 4.13 と同様に定義できる．

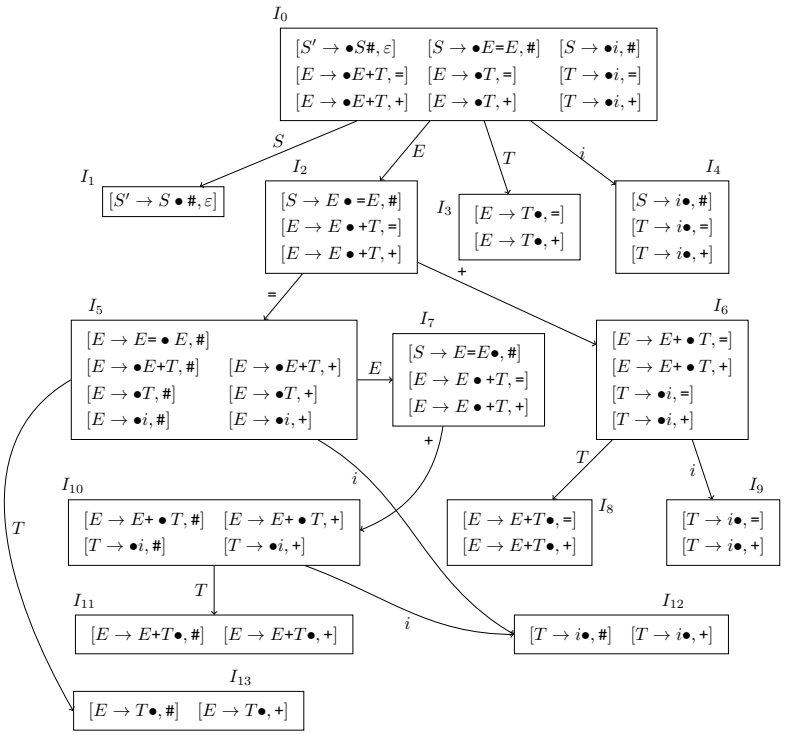


図 4.13: $G_=$ の正準 LR(1) 遷移系

定義 4.19. I を *LR(1)* 項の集合とする．I と $\alpha \in (N \cup \Sigma)^*$ に対して Goto 関数を以下のように定義する．

$$\mathsf{Goto}(I, \alpha) = \begin{cases} \{\theta' \mid \theta \xrightarrow{\varepsilon}{}^{*} \theta\text{となる } \theta \in I \text{ が存在する } \} & \alpha = \varepsilon \text{ のとき} \\ \{\theta' \mid \theta \xrightarrow{M} \theta'\text{となる } \theta \in \mathsf{Goto}(I, \beta) \text{ が存在する } \} & \alpha = \beta M, \beta \in (N \cup \Sigma)^*, \\ & M \in N \cup \Sigma \text{ のとき} \end{cases}$$

正準 LR(1) 遷移系 $(\mathcal{S}_1, \mathcal{E}_1)$ において，$I \in \mathcal{S}_1$ に対して，$I \xrightarrow{M} I' \in \mathcal{E}_1$ ならば，$\mathsf{Goto}(I, M) = I'$ である．

例 4.10. $G_=$ に対する正準 *LR(1)* 遷移系は図 *4.13* のようになる．
さらに *LR(1)* 構文解析表は図 *4.14* のようになる．

	i	$=$	$+$	$\#$	S	E	T
0	$s4$				1	2	3
1				acc			
2		$s5$	$s6$				
3		$r4$	$r4$				
4		$r5$	$r5$	$r2$			
5	$s12$					7	13
6	$s9$						8
7			$s10$	$r1$			
8		$r3$	$r3$				
9		$r5$	$r5$				
10	$s12$						11
11			$r3$	$r3$			
12			$r5$	$r5$			
13			$r4$	$r4$			

図 4.14: $G_=$ の LR(1) 構文解析表

$G_=$ の表エントリに操作の重複はなく，$G_=$ は *LR(1)* 文法である．これは，正準 *LR(1)* 遷移系の I_4 において，$S \to i$ で還元するのは先読み文字が $\#$ の場合のみであり，それ以外の先読み文字の場合は $T \to i$ で還元することが *LR(1)* 項によって区別できるためである．

LALR(1) 構文解析

正準 LR(1) 遷移系は，Σ の数が増えると状態数が増える．LR(1) 項に含まれる LR(0) 項は共通の形をしている事があり，可延頭部までの構文解析の状態としては共通である．LALR(1) 構文解析は，含まれる LR(0) 項が全く同じで先読み文字だけ違いがあるときに，構文解析としては同じ状態にあると考えて LR 遷移系の状態数を減らす構文解析である．

$G_=$ の場合は，I_6 と I_{10}, I_3 と I_{13}, I_9 と I_{12} は同じ状態にあるとして，図 4.15 のように LALR(1) 遷移系を構成する．

この遷移系に基づいて図 4.16 のように構文解析表を構成する．

図 4.16 のエントリーにはたかだか 1 つの操作のみが書かれているため，$G_=$ は，LALR(1) 構文解析である．

正準 LR(1) 遷移系から正準 LALR(1) 遷移系を構成する場合，最大で LR(1) と同じ状態の遷移系を扱わなければならない．LALR(1) 遷移系は LR(1) 遷移系を作成しなくても構成できる．LALR(1) 遷移系のより直接的な作成アルゴリズムについては参考書を参照のこと．

LALR(1) 構文解析は，可延頭部に着目する LR(0) 項と同じくらいの状態数で，構文解析を行うことができる．ここで，先読み文字は状態に埋め込んでいることから SLR(1) よりもより広い範囲の CF 文法に適用可能である．LALR(1) は構文解析生成プログラムの yacc が採用している構文解

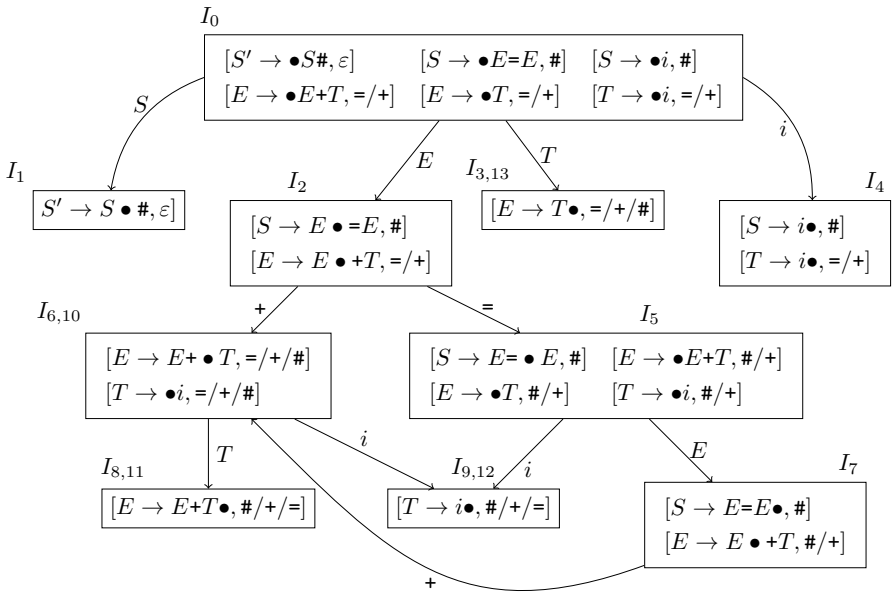


図 4.15: $G_=$ の LALR(1) 遷移系

析である．Σ の数が増えても，LR(1) 遷移系と比較して一定程度状態数を減らすことができるので，効率的な構文解析手法として非常によく用いられる．

4.3.3　構文解析手法の能力比較

図 4.17 に無曖昧な CF 文法の解析手法の比較についてしめす [**?**].

	action				goto		
	i	$=$	$+$	$\#$	S	E	T
0	s4				1	2	(3,13)
1				acc			
2		s5	s(6,10)				
(3,13)		r4	r4	r4			
4		r5	r5	r2			
5						7	(3,13)
(6,10)	s(9,12)						(8,11)
7			s(6,10)	r1			
(8,11)		r3	r3	r3			
(9,12)		r5	r5	r5			

図 4.16: $G_=$ の LALR(1) 構文解析表

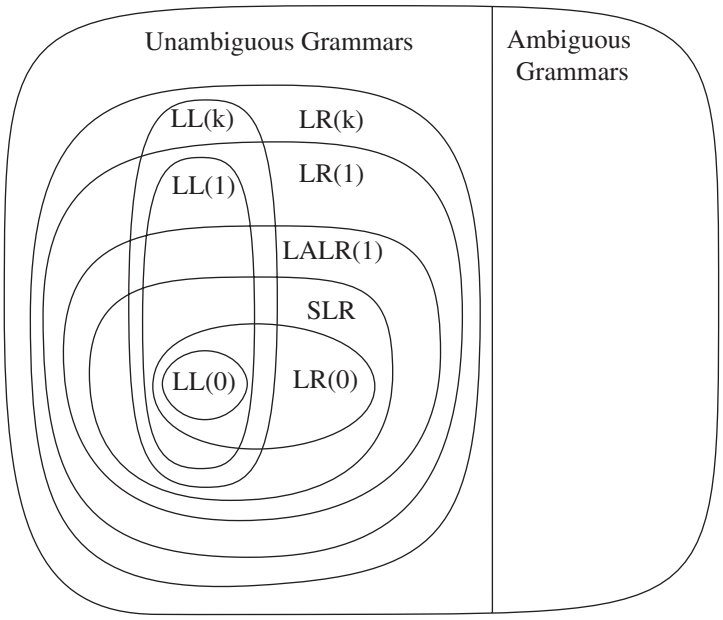


図 4.17: CF 文法の解析能力の比較

4.3.4 演算子順位法

演算を持つ式を構文解析する方法として，演算子順位 (operator precedence) 法がある．この構文解析手法は演算子の優先度と結合の規則を直接的に構成できることから実現が容易で，効率的な構文解析が可能である．しかし，文法から生成できない文字列にも適用が可能であり，CF 文法と

	i	n	$+$	$-$	$*$	$/$	$($	$)$	$\#$
i			$\gg$	$\gg$	$\gg$	$\gg$		$\gg$	$\gg$
n			$\gg$	$\gg$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$+$	$\ll$	$\ll$	$\gg$	$\gg$	$\ll$	$\ll$	$\ll$	$\gg$	$\gg$
$-$	$\ll$	$\ll$	$\gg$	$\gg$	$\ll$	$\ll$	$\ll$	$\gg$	$\gg$
$*$	$\ll$	$\ll$	$\gg$	$\gg$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$/$	$\ll$	$\ll$	$\gg$	$\gg$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$($	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$==$	
$)$			$\gg$	$\gg$	$\gg$	$\gg$		$\gg$	$\gg$
$\#$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	acc

表 4.1: 演算子順位表

の対応は不明確であり，LL, LR の構文解析のような CF 文法に基づいて柔軟に構文解析プログラムを生成することはできない．

$$G_{Exp} = \langle \{E\}, \{+, -, *, /, (,), i, n\}, P, E \rangle$$

$$P = \left\{ \begin{array}{llll} 1: & E \to E{+}E, & 2: & E \to E{-}E, \\ 3: & E \to E{*}E, & 4: & E \to E/E, \\ 5: & E \to (E), & 6: & E \to i, \\ 7: & E \to n & & \end{array} \right\}$$

G_{Exp} で定義される式の表現について考える．ここでの演算子は左結合であることから，左にある演算子が優先されることから，$+ \gg +$, $+ \gg -$, $- \gg +$, $- \gg -$ および $* \gg *$, $* \gg /$, $/ \gg *$, $/ \gg /$ という順序をつける．ここでは通常の演算式のように $*$, $/$ は $+$，$-$ の演算よりも優先されることから，$* \gg +$, $* \gg -$, $/ \gg +$, $/ \gg -$ という優先度を付加する．(については，中の式を先に解析するため (の優先度は低くなる．) との対応をとるために同じ優先度 $==$ を導入している．) は，括弧の中の解析を優先するため高い優先度を持つ．その他の演算子の間にも表 4.1 というような順序を定義する．

定義 4.20. $\alpha \in (N \cup \Sigma)^*$ に対して $\mathsf{term}(\alpha)$ を α からすべての非終端記号を取り除いて得られる記号列とする．

演算子順位法による構文解析

演算子順位法で構文解析を行う PDT を定義する．

PDT のスタックを $a_1 \cdots a_n$ とする．ここでスタックの底 a_1 は $\#$ であるとする．スタックトップは a_n であり，入力でポインタのある次に読み込む記号を $b \in \Sigma \cup \{\#\}$ とする．

還元 $a_n \gg b$ ならば，$a_i \gg a_{i+1} == \cdots == a_n$ となる $a_{i+1}, \cdots, a_n$ をスタックから取り除く．$\mathsf{term}(\alpha) = a_1 i + 1, \cdots, a_n$ となる生成規則 $A \to \alpha$ の番号 o を出力テープに書き込む．Δ に以下の規則を含む．

$$(a_{i+1} \cdots a_n, b) \xrightarrow{(b)} (\varepsilon, o)$$

シフト $a_n == b$ または $a_n \ll b$ ならば，入力記号 b をスタックトップに格納する．Δ に以下の規則を含む．

$$(a_n, b) \xrightarrow{b} (a_n b, \varepsilon)$$

受理 $a_n = b = \#$ ならば，受理状態となる．Δ に以下の規則を含む．

$$(\#, \#) \xrightarrow{\#} (acc, \varepsilon)$$

エラー それ以外の場合，構文エラーとなる．

演算子順位法による構文解析を行うには，まず，スタックに # をセットして，PDT を動作させ acc に到達すれば，構文としては正しい．

例 4.11. `i+i*i#`に対して表 *4.1* の演算子順位表を使った構文解析は以下のようになる．

	$(\#, \boldsymbol{i+i*n\#}, \varepsilon)$	
$\vdash$	$(\# i, \boldsymbol{+i*n\#}, \varepsilon)$	シフト $(\# \ll i)$
$\vdash$	$(\#, \boldsymbol{+i*n\#}, 6)$	$E \to i$ で還元 $(i \gg +)$
$\vdash$	$(\# +, \boldsymbol{i*n\#}, 6)$	シフト $(\# \ll +)$
$\vdash$	$(\# +i, \boldsymbol{*n\#}, 6)$	シフト $(+ \ll i)$
$\vdash$	$(\# +, \boldsymbol{*n\#}, 66)$	$E \to i$ で還元 $(i \gg *)$
$\vdash$	$(\# +*, \boldsymbol{n\#}, 66)$	シフト $(+ \ll *)$
$\vdash$	$(\# +*\boldsymbol{n}, \#, 66)$	シフト $(* \ll i)$
$\vdash$	$(\# +*, \#, 667)$	$E \to n$ で還元 $(n \gg \#)$
$\vdash$	$(\# +, \#, 6673)$	$E \to E*E$ で還元 $(* \gg \#)$
$\vdash$	$(\#, \#, 66731)$	$E \to E+E$ で還元 $(+ \gg \#)$
$\vdash$	$(acc, 66731)$	受理

G_{Exp} のように生成規則が演算子を特定できるような場合は，曖昧な文法に対しても比較的簡単に構文解析が可能である．簡単な数式に対してはこのような条件が成立するため簡易的に構文解析プログラムを構成できる．

4.4 構文エラー表示と誤り回復

構文解析表に従って解析を進める際に対応する表 (下向き構文解析表，LR 構文解析表の action 表) に生成規則，操作がない場合，入力記号列は構文エラーを含む．コンパイラは構文エラーを含む入力を変換する必要はないが，入力に対して何がエラーの原因かを表示して，適切な修正を促すことが望ましい．このような誤り回復の方法としては，プログラムに記号の挿入，削除を適用する方法，あるいはパニックモードを用いる方法がある．プログラムに挿入，削除の修正を自動的に適用する方法は，適用が難しく時間がかかることに加えて，修正が必ずしもプログラマーが意図したものになるかがわからないので，修正が決定できる必要があるため適用できる言語は限定的である．

パニックモードを用いる方法では，構文エラーに到達したときは，残りのプログラム中に分離記号 (デリミター) を見つけるまで入力を読み捨てて誤り回復を試みる．しかし，得られた結果が信

頼できるものではなく，複数の構文エラーを一回の構文解析で指摘することができるようにすることが目的である．一般に，最初のエラーが発生した場所を詳しく表示することが重要である．

4.5　YACC による構文解析プログラム生成

YACC(yet another compiler compiler) は構文解析器 (パーザー) を CF 文法から自動生成するツールである[3]適用できる CF 文法のクラスは LALR(1) である．

yacc で記述するファイルは %%で区切られた 3 つの部分で記述される．

ヘッダー，データ構造，トークン定義

%%

CF 文法

%%

C プログラム（使用する関数の定義リスト)

yacc は lex とともに使われる．lex によって用いる単語を定義して字句解析を行い，その結果を yacc で生成されたトークン列を送って構文解析を行う．

以下に四則演算を含む数式の yacc 記述を示す．%token は lex で切り分けるトークンを定義している．% left は左結合の演算子のトークンを表し上の方のトークンの還元の優先度が低いものを上に記述する．ここでは，加減算の方が乗除算よりも優先度が低いのこの順番で記述する．

CF 文法の生成規則は下にあるような形で記述する．生成規則の後ろに $\{\cdots\}$ で記述された動作をその生成規則が還元されたときに実行する．$\$i$ は右辺の i 番目の記号の値を表し，$$ は左辺の非終端記号の値を示す．

C 言語プログラムの場所はこのファイル程度書いておけば変換時のエラーは発生しないｓ．

```
%{
#include <stdio.h>
#include <stdlib.h>
#define        YYDEBUG 1
int        yylex();
int        yyerror(char const *str);
extern  int yylval;
%}
%token        NL
%token        NUMBER
%token        LPAR
%token        RPAR
```

[3]yacc は．AT&T がもともと権利をもつツールであるため，GNU では bison として実現されている．

```
%left        ADDOP SUBOP
%left        MULOP DIVOP
%%
s        : list
;
list        : /* empty */
| list expr NL                {printf("%d\n",$2);}
;
expr        : expr ADDOP expr        {$$ = $1 + $3; }
| expr SUBOP expr        {$$ = $1 - $3; }
| expr MULOP expr        {$$ = $1 * $3; }
| expr DIVOP expr        {$$ = $1 / $3; }
| LPAR expr RPAR        {$$ = $2; }
| NUMBER                {$$ = $1; }
;
%%
int yyerror(char const *str) {
    extern char *yytext;
    fprintf(stderr, "parser error near %s\n", yytext);
    return 0;
  }

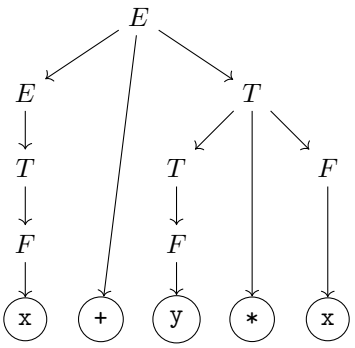
int main(void) {
    extern int yyparse(void);
    extern FILE *yyin;

    yyin = stdin;
    if (yyparse()) {
        fprintf(stderr, "Error Occurred!\n");
        exit(1);
      }
  }
int yywrap() {
    return(1);
  }
```

演算子による優先度を定義しない場合は，誤った構文木が得られるため計算結果が異なる可能性がある．(Web サイトに記載．) Shift/Reduce コンフリクトが発生した場合，Shift 動作を優先する．

第5章　意味解析

構文解析で得られた構文木はプログラムの構造を形式的に表現している．構文木のノードにはプログラムの意味に関わる情報が含まれている．例えば，x+y*x という式の構文木は以下のようになる．



ここで，終端記号は文法上では，トークンとして扱われていたが，実際には，変数，演算子などプログラム実行上の意味を持つ．さらに，トークンが同じ変数である場合（例えば，x) は，同じ式の中では割り当てられた同じ値を持つ．一般に+は加算，*は乗算を表す．プログラミング言語の多くは値に「型」という概念を持つ．これは，計算を実行するときにどれだけ記憶領域を確保すればよいかを示すとともに，式やプログラムの操作の記述に不整合がないかを実行前にチェックするために用いられる．プログラムが定める意味に従った構文の整合性を確認することを**意味解析**と呼ぶ．

5.1　記号表

記号表は構文木のノードが表す記号が持つ意味的情報を格納する．構文木において，終端記号としてのトークンは記号情報を持つ．字句解析によって得られたトークンは，識別子 (identifier)，演算子 (operator)，区切り記号 (delimiter) などの種類を持つ．それぞれのトークンに応じてプログラム中での意味が与えられる．記号表は構文に現れる記号の情報を格納する．したがって，記号表の構成はプログラミング言語の定義に依存する．

多くのプログラミング言語では，同じ識別子を異なった文脈で使用することができる．プログラミング言語においてはスコープルールが定義されている．最も基本的なスコープルールは局所と大域に対するルールであり，局所記号は一定のブロック内のみに有効な記号であり，大域記号はプログラム中のすべての場所に有効な記号である．スコープルールとしてはまず局所に有効な記号が存在するかどうかを調べて，局所になければ大域記号を参照するという規則である．

```
1  int x;
2  int y;
3  int f(int y) {
4      x = 2;
5      return x+y;
6  }

7  int main() {
8      int x;
9      x = 1;
10     y = 3;
11     x = f(x);
12     printf("%d,%d\n",x,y);
13 }
```

図 5.1: 局所変数と大域変数の例

例えば上記のプログラムにおいて，3 行目から 6 行目において y は引数からの局所変数であり，7 行目から 13 行目において，x も局所変数となる．その他の x および y は大域変数として参照される．したがって，このプログラムが C 言語と同様に main から実行されるとすると，9 行目は，8 行目で宣言した局所変数への代入であり，10 行目は 2 行目で宣言された y を参照する．関数の引数は 1 であり，この結果として，関数 f は 3 を返す．よって 13 行目の結果として，3,3 が表示される．

コンパイラが実行コードを生成する場合，また，インタプリタがプログラムを実行する際には，適切なメモリ領域を参照する必要がある．大域変数はプログラムの実行前に，局所変数は関数の呼出しごとにメモリ領域を確保してその番地が決定する．記号表は変数（記号）とメモリ番地の対応を管理する．

コンパイラが図 5.1 からコード生成を行う場合，２行目において大域変数 x，y を割り当て，３行目で関数記号 f，局所変数 y を割り当てる．４，５行目の x は１行目で割り当てた大域変数 x を参照し，５行目の y は３行目で割り当てた局所変数 y を参照する．6 行目で y の参照を破棄する．７行目で関数記号 main を割り当て，８行目で局所変数 x を割り当てる．９行目の x は８行目で割り当てた局所変数 x を参照し，10 行目の y は２行目で割り当てた大域変数 y を参照する．11 行目で局所変数 x に関数 f を呼び出し，その結果を局所変数 x に代入する．12 行目における x は局所変数，y は大域変数を参照する．このような参照を可能にするために，変数記号から記号表のどのエントリを参照するかを決める必要がある．図 5.1 において，x と y はともに大域変数の場合と局所変数の場合がある．各変数は宣言された時点で記号表に登録され，局所変数は，有効なブロックの中でのみ有効であり，有効なブロックから出ると参照されることがないので，記号表から削除する．大域変数はプログラムの最初から最後まで有効なので記号表から削除されることはない．

構文の構造に従って参照を決めるスコープルールを**静的スコープ**，呼び出しによって直前の局所変数を参照するスコープルールを**動的スコープ**と呼ぶ．上にあげた参照ルールは静的スコープであ

り，図 5.1 が静的スコープを持つとすると最も最後に追加された記号を参照することによって正しい参照を実現することができる．静的スコープは，構文解析の段階で決定されるため，コンパイル時に参照を決定することができる．一方で，動的スコープは，プログラムの実行時に参照を決定するため，実行時に参照を決定する必要がある．プログラミング言語としては静的スコープがよく用いられるが，動的スコープを持つ言語も存在する．例えば，Lisp 言語は動的スコープを持つ．

プログラミング言語によっては，関数やメソッドのスコープルールが決められている場合がある．例えば，Java 言語において，private なメソッドはそのクラス内でのも参照できるメソッド (操作) であり，オブジェクト指向言語においてよく用いられる．オブジェクト指向言語では，クラスから生成されるインスタンスに固有の局所変数を用いてオブジェクトの独立性を保つことが多く，スコープを指定するディレクティブが多く用意されていることが多い．

5.1.1 記号表の実現

記号表は，構文要素の記号をもとにスコープルールを守りながら，その構文要素が持つ意味的な情報を管理するデータ構造である．例えば，図 5.1 のプログラムでは，9 行目のコンパイルにおいては以下のような属性をもつ表構造として実現される．

	名前	種類	値	スコープ	番地	型	引数個数	引数型
1	`x`	var		大域 (G)	0	`int`		
2	`y`	var		大域 (G)	1	`int`		
3	`f`	func		大域 (G)	100	`int`	1	0
4	`x`	var		局所 (L)	0	`int`		

値フィールドは，定数記号の宣言時に用いられることを想定している．

記号表はコンパイルの過程でスコープルールに従って，記号の登録，参照および削除を行ってプログラムで決められた変数や関数記号が実際にメモリ上のどの領域を意味するか，どの命令系列に対応する関数か，ということを表として管理する．ブロック構造によって文脈が変化する場合，ブロック内で局所変数が宣言された場合は記号表に追加し，ブロックから脱出する場合は局所変数を記号表から削除する．このため，記号表は柔軟に登録，削除ができ，高速に参照できるデータ構造である必要がある．記号データを一列に並べた線形リストは，参照は線形時間時間で可能であるが，スコープルールを実現するためのデータの追加，削除も線形で可能であるが，データ量が大きくなると実際の操作時間は増大する．バランス木などの木構造によって整列した場合は参照は log 時間で可能になり，高速にアクセス可能であるが，追加・削除における木構造の操作に時間がかかる．この目的のため，次に述べるハッシュによるインデックス付けの手法がよく用いられる．

例えば以下のような C 言語風のプログラミング言語において，各関数が呼び出された直後と各関数の環境を出る直前の記号表の様子を示す．番地については，関数についてはコンパイルされた命令の最初の位置，帯域変数では，プロセス内で割り当てられた帯域変数領域の先頭からの相対位置 $(G + n)$，局所変数，仮引数については，スタックフレームにおける局所変数領域の先頭からの相対位置 $(FP + n)$ が記入されている．

```
int g;

func f(int a, real b) {
    int x;
    x = a + 1;
    g = x;
}

func h() {
    int x;
    g = 0;
}

func main() {
    int x;
    x = 3;
    f(x + 1, (2.5 + 3.5) * 2);
    h();
}
```

関数 f の局所文脈に入るとき（仮引数・局所変数の登録前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	—	—

関数 f の局所文脈を出るとき（局所項目の消去直前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
a	仮引数	呼出し時に決定	local	FP+0	int	—	—
b	仮引数	呼出し時に決定	local	FP+8	real	—	—
x	変数	未定	local	FP+16	int	—	—

関数 h の局所文脈に入るとき（仮引数・局所変数の登録前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	—	—

関数 h の局所文脈を出るとき（局所項目の消去直前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	0	()
x	変数	未定	local	FP+0	int	—	—

関数 main の局所文脈に入るとき（仮引数・局所変数の登録前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	0	()
main	関数	—	global	L_main	void	—	—

関数 main の局所文脈を出るとき（局所項目の消去直前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	0	()
main	関数	—	global	L_main	void	0	()
x	変数	未定	local	FP+0	int	—	—

図 5.2: 記号表の例

ハッシュ

ハッシュは線形に並べたデータをインデックス（目次）ごとに並べるようにして，データ構造の途中から探索する方法である．追加・削除も途中から行うことができるため，より効率的な管理が可能となる．ハッシュが有効に働くためには，各目次の項目から作られるリストができるだけ均一になるようにする必要がある．このため，格納する記号データの集合 D に対して，$d \in D$ から計算される値 $f(d)$ ができるだけ D を均等に分類する関数によってインデックスを作成する．このような関数を**ハッシュ関数**とよぶ．コンパイルを始める時点で D は既知ではないので，D の要素が一様に分布するというような仮定のもとに f を選択する．ハッシュ関数によってうまくデータが分散されれば，アクセスは高速になり，追加・削除に要する時間も削減できる．ハッシュが大きく偏る場合は，線形にならべた場合と同じになる．

ハッシュ関数として以下のような関数を考える．

$$f(a_1 \cdots a_n) = \left\{ \ \sum_1^n \mathsf{ord}(a_i) \cdot 63^{n-1} \ \right\} \mathsf{mod}\ N + 1$$

ここで，識別子のトークンの文字列は，_(アンダースコア), a, . . . , z, A, . . . , Z,0, . . . , 9 の系列であり，それぞれ 1 から 63 までの整数が割り当てられているとする．ord はその値を返す関数である．この計算は除算法と呼ばれる方法であり，N の値として素数を選ぶのが適当である．N が 2^k とすると f の値が下位の文字のみによって定まってしまう．

$N = 1009$ とすると，文字列 main は以下の値を持つ．

$$f(\mathtt{main}) = (14 \times 63^3 + 2 \times 63^2 + 10 \times 63 + 15)\mathsf{mod}\ 1009 + 1 = 949$$

記号 σ に対する記号表のデータは $f(\sigma)$ でアクセスするようにするが，ハッシュ関数では $f(\sigma_1) = f(\sigma_2)$ となる異なった記号 σ_1,σ_2 が存在するため，σ_1 の情報と σ_2 の情報をリストにしてアクセスする．

ハッシュを用いたインデックスづけでは，ハッシュ表に同じハッシュ値をもつ記号に対するリストの先頭を格納する．ハッシュ値 i に対するリストのことをバケット i と呼ぶ．ハッシュ表に格納される要素の数がハッシュ表のサイズの定数倍で抑えられるとき，ハッシュ表全体に一様にランダムに値がばらまかれると仮定すると，検索，登録，削除の各要素は定数時間で行うことができる．逆に，一つのバケットに格納された場合は線形リストと同じで要素の数に対する線形時間となる．このため，ハッシュ関数としてできるだけ分布が偏らない関数を用いる．

図 5.1 のコンパイルにおける記号表の構成例を以下に示す．

(1) 1 行目のコンパイル:
`int x` の宣言によって `x` を割り当てる (図 5.3a).

(2) 2 行目のコンパイル:
`int y` の宣言によって `y` を割り当てる (図 5.3b).

(3) 3 行目のコンパイル:
関数 `f` の宣言と仮引数 `int y` を割り当てる (図 5.3c).

(4) 7 行目のコンパイル:
6 行目で仮引数 y を削除して関数 main の宣言を追加する (図 5.3d).

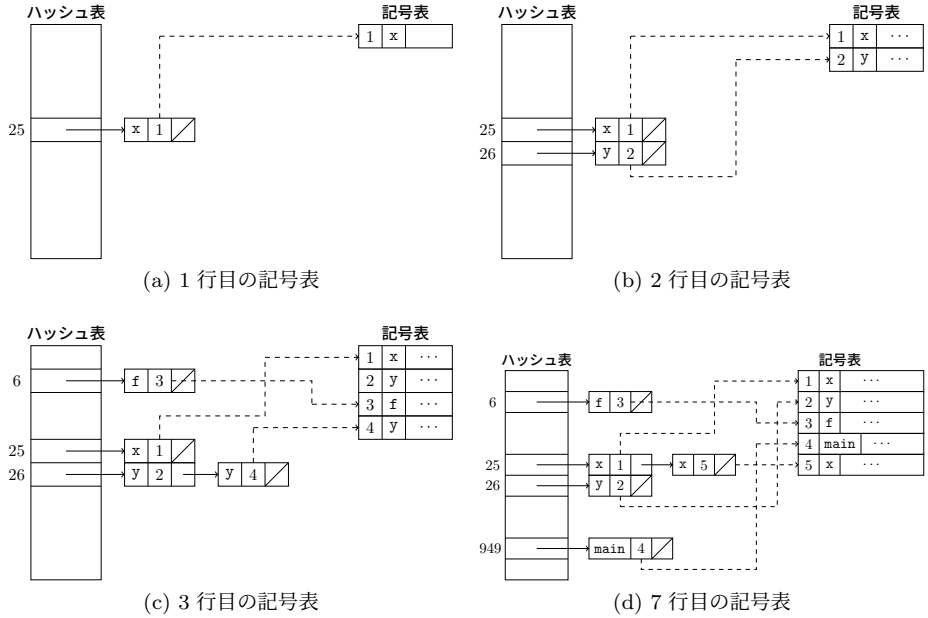


図 5.3: コンパイルの過程における記号表の構成

上記のようにハッシュ表には外部に格納された情報情報へのリンクを格納する方法を特に**外部ハッシュ**と呼ぶことがある．これに対してハッシュ表内部に情報を格納する方法は**内部ハッシュ**と呼ばれる．一般には，情報の柔軟性の高い外部ハッシュが用いられることが普通である．

5.2　型チェック

型チェックは，プログラム中のデータが整合した型を持つかどうかを検査する．コンパイル時に型チェックを行う**静的型チェック**と実行時に型チェックを行う**動的型チェック**がある．静的型チェックはプログラムを実行する前にプログラムの矛盾を検出し，効率的な実行コードを生成することが可能である．動的型チェックはより柔軟な記述を許すことができるが，型が実行時に決まるために実行時のメカニズムが複雑になる．

型チェックの手法はプログラミング言語の型付けのポリシーによる．コード生成の観点からは，型はできるだけ単純な型を持つことが望ましい．しかし，より柔軟性の高い記述のために，多相型や型のオーバーロードのような手法を用いて自然な記述を可能にすることが一般によく行われる．

再帰的な型定義を持つ場合は効率的にコンパイルするためには 2 パスで型チェックを行う必要であることもある．変数の型，関数，メソッドの返り値と引数の型が同じ識別子ですべて同じであるかを構文解析時にチェックすることで，プログラムの不整合を事前に検出する．

静的な型チェックとして，例えば，加算式 $e_1 + e_2$ をコンパイルするとき，e_1，e_2 の型によって内部狂言が異なるため，生成する命令が異なる．両辺が整数型の場合は，整数の加算命令を発行

し，片方が整数で，他方が実数である場合は，整数を実数の型に変換する命令を発行してから実数の加算命令を発行する．ML のように型によって異なる演算記号をもち，型変換を行わずにエラーとする言語もある．また，文字列などのように加算することができない型である場合は，エラーを発行したり，結合など別の操作のためのコードを発行する．

以下ではコンパイラ時に型の整合性を検査する静的型チェックを中心に説明する．

5.2.1 型定義と型チェックアルゴリズム

簡単な型チェック法を次の生成規則で定義される言語を考える．

$$
\begin{aligned}
&P \to D\ ;\ S\\
&D \to D\ ;\ D \mid \mathbf{id}:\ T\\
&T \to \texttt{char} \mid \texttt{int} \mid \texttt{\&}T \mid \texttt{arr[}n\texttt{]}\ \texttt{of}\ T\\
&E \to c \mid n \mid \mathbf{id} \mid E + E \mid *E \mid E\texttt{[}E\texttt{]}\\
&S \to \mathbf{id}{:=}E \mid S\,;S
\end{aligned}
$$

プログラムは，変数の型宣言部 D とプログラム本体 E からなる．E はここでは簡単のため加算と配列参照のみからなる式としている．i : &T は，変数 i が型 T を持つデータ領域へのポインタであることを示す．$*i$ は i が型 T を持つデータ領域へのポインタであるとき，i の指す型 T を持つデータ領域を表す．型に関する制約は，加算 + は整数に対してのみ可能であり，$E[E']$ においては，E の型は配列型 (arr[n] of T) であり，E' の型は整数型 (integer) である．このとき $E[E']$ の型は T である．図 5.4 に宣言における型定義の規則，図 5.5 に型制約規則を示す．

生成規則	型規則
$P \to D\ ;\ E$	
$D \to D\ ;\ D$	
$D \to i\ :\ T$	i.type:=T.type (記号表に登録)
$T \to$ char	T.type:=char
$T \to$ int	T.type:=integer
$T \to$ &T_1	T.type:=pointer(T_1.type)
$T \to$ arr[n] of T	T.type:=array($1..n$.value, T_1.type)

図 5.4: 型定義の規則

生成規則	型制約規則
$E \to c$	E.type:=char
$E \to n$	E.type:=integer
$E \to i$	E.type:=i.type (記号表を検索する)
$E \to E_1$+E_2	E.type:= **if** E_1.type = integer **and** E_2.type = integer **then** integer **else** type_error
$E \to$ *E_1	E.type:= **if** E_1.type = pointer(t) **then** t **else** type_error
$E \to E_1$ [E_2]	E.type:= **if** E_1.type = array(s,t) **and** E_2.type = integer **else** type_error
$S \to i$:=E	i.type= E.type

図 5.5: 型制約規則

c は文字トークン，n は整数定数のトークンを表す.

```
char x;
int y;
&int z;
arr[10] of char a;
arr[5] of arr[5] of int m;
```

ここで，T.type は型識別子 T が表す型であり，通常，T の表現そのものである．i.type は識別子 i が持つ型情報であり，記号表に記録される．n.value は，数トークン n が表す値であり，n の表現そのもの $(0, 1, \cdots$ など) である．型制約のない場合，上記のように宣言されているとき，`y:=*z`, `a[y]` は型づけとして正しい (well-typed)．`x:=1`, `x+y`，`x+1`，`m[x]` は型エラーである．

型制約は構文解析に沿って検査可能である．下向き構文解析の場合は，規則の最後で上のレベルに戻る際に一致をチェックする．上向き構文解析の場合は還元の際に属性として検査することができる．

5.2.2　多相型と型のオーバーロード

原則的には構文的に対応する場所には同じ型がなければならない．直観的な値に対する型の概念は，その値に対してどういった操作を適用できるかということで決定される．したがって，型が厳密に一致していなくても互換性のある操作であれば認める方がプログラムとしては記述しやすい．型の一致については，多層型やオーバーロードという方法で矛盾することなく型の概念が適用できる．例えば，整数に対しては，四則演算を適用できる値の集まりであると特徴づけられる．これに対して，文字列については，文字を表す記号列に対して，連接や接頭語，接尾語などを取り出す操作をもつ値であり，整数型とは異なる．

多相型

多相型 (polymorphic type) は，操作が共通した型を表す．リストや配列は，同じ型を持つ値から構成されるデータ型である．リストに対しては，要素の型にかかわらず連接，空チェックなど共通

の操作が定義できる．List(X) のように記述し，変数 X に要素データの型を代入し，List⟨Int⟩, List⟨Char⟩ のように記述することによって同じ操作を持つデータ型を記載する．このことによって操作を要素ごとに重複して記載する必要がない．

多相型を持つプログラミング言語の記述として，Java，Ada や Modula-3 における *generic* 型，C++におけるテンプレート，ML におけるタイプスキームなどがある．

部分型

型 A に対する操作が型 B に対しても適用できるとき，型 B は型 A の部分型であるとよばれる．部分型はもとの型を拡張することによって定義される．多相型 $T\langle X\rangle$ を構成する場合，型 B が型 A の部分型であるとき，$T\langle A\rangle$ が矛盾のない型であるとき，$T\langle B\rangle$ も矛盾のない型として定義される必要がある．

オーバーロード

```
1                int x;
2                float y;
3                double z;
4                x=1; y=1.1; z=1.57E8;
5                x=x+1;
6                z=y+z;
7                z=z+x;
```

オーバーロードは同じ種類の操作（通常同じ記号で表す）を引数の型によって切り替える機能である．多くの言語は四則演算において，オーバーロード機能を持つ．

左のような加算は型の異なる数に対して加算を実行する．5,6,7 行目の演算はすべて加算であるが，変数 x,y,z の内部表現はすべて異なるのでコンパイラが発行する命令は異なる．計算機の実行上では，5,6,7 行目は異なる命令を発行する必要がある．オーバーロードがない場合，同じ加算であっても型が異なる場合は異なる演算記号を用いる必要がある．オーバーロードがない関数型言語 ocaml では整数の加算は+, 浮動小数点の加算は+. を用いる．

第6章　中間コード生成

6.1　概要

構文解析によって，プログラミング言語に対する入力プログラムの構文的な正しさをチェックするとともに，入力プログラムの意味を構文解析木として与える．構文解析木に沿って，プログラムの動作を実現する命令の系列を生成する．開発者が用いるソース言語の抽象度が高いほど，プログラムは構造的に整理されて理解が容易な形で記述される．これに対して計算機の実行命令は直線的な系列であり，計算機固有のアーキテクチャの影響を受ける．近代のコンパイラでは，命令コードは複数の段階を経て生成されるのが一般的である．まず，単純な制御構造とメモリアクセスを直線的に表現できる中間コードを機械的に生成し，その後，実際のアーキテクチャに沿ったよりハードウェアに近い命令に変換する．

中間コードを一旦生成することで，アーキテクチャに依存しない，入力プログラムが持つ制御構造，メモリアクセスを抽出して，入力プログラムの性質に基づいて最適化することができる．さらに中間コードで用いる変数を数の制限の大きいレジスタに配分して，プログラムに対してメモリ階層の観点から効率的に実行する割り当て方法を実現することができる．

6.1.1　抽象構文と抽象構文木

抽象構文は，プログラムを定義する文脈自由文法が表現する文法のうち，プログラムの振舞いに関連する部分のみを抽象した文法によって定義される．例えば，四則演算に関する構文は，図 6.1 で十分である．この文脈自由文法は曖昧であるが，四則演算の演算子が $+$, $-$, $*$, / であり，それらが変数または数から再帰的に構成される式を 2 つ両側に持つ演算であることを表している．一旦，式の構文解析木が構成されれば，その意味（値）を計算するには，抽象構文が表現する意味に沿って計算する命令系列を生成すればよい．

抽象構文木 (abstract syntax tree) は，本質的に意味を表す抽象構文に沿って構成される木構造である．コード生成において，構文解析木に現れる非終端記号の情報は必要でない場合が多く，生成規則の持つ意味をノードに記述した方がコード生成には都合が良い．木構造がコードの意味を直接的に表現していると考えられることから，抽象構文木が中間コードであるとみなすことができる．例えば，代入式 `w=x+y*z` の抽象構文木は図 6.2 のようになる．

実際には構文木の構成において，x,y,z はそれぞれトークンとして ID として扱われるが，実際のコード生成においては，変数名を必要とする．字句解析においてもトークンの他に実際の識別子の記号列が保存されている．(lex における `yytext` など)

$$
\begin{aligned}
E &\rightarrow E + E \\
E &\rightarrow E - E \\
E &\rightarrow E * E \\
E &\rightarrow E \,/\, E \\
E &\rightarrow \texttt{ID} \\
E &\rightarrow \texttt{NUM} \\
S &\rightarrow \texttt{ID} = E
\end{aligned}
$$

図 6.1: 四則演算式と代入文の抽象構文

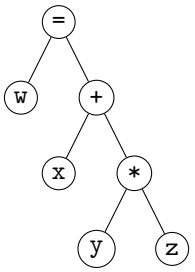


図 6.2: 抽象構文木の例

6.1.2　逆ポーランド記法

逆ポーランド記法は引数の数が固定の演算を後置形式で記載する．計算に用いる値をメモリの値を push 命令で参照することによって計算することができ，算術式の中間コードとして古くから用いられている．逆ポーランド記法からより低レベルの機械語（バイナリコード）へ変換することは容易である．演算の引数となる変数を ID で参照するとき，push ID でその値をスタックに積む，演算を適用するとき，n 引数の演算では，n 個の値をスタックから pop して演算を適用し，その結果をスタックに push する．例えば，x+y*z という入力に対しては，push x, push y, push z, $*$, $+$ という命令列によってスタックの先頭に値の計算が積まれる (図 6.3).

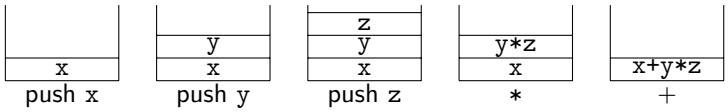


図 6.3: 逆ポーランド記法の計算

6.1.3　3 番地コード

3 番地コード (3-address code) は，命令を一般的に $C = A\theta B$ の形に限定して動作を記述する．ここで，A,B,C はメモリ番地であり，A のメモリの内容と B のメモリの内容に θ の演算を適用して，結果を C の番地に格納することを表す．これ以外に単項演算，条件付きジャンプ命令などを

	一般形	演算子 θ
代入文	$C = A\theta B$ $B = \theta A$ B=A	+,-,*,/, and,or,not >,>=,=,!=,<=,<,-
ジャンプ文	if A goto L if-not A goto L goto L	

図 6.4: 3 番地コード

用いることによって一般的な計算を単純な命令の組み合わせで記述することができる．各命令にはラベル L をつけることができ，ジャンプ命令の飛び先とすることができる．3 番地コードを機械語に変換することは容易であり，3 番地コードによって最適化を行うことでハードウェアに依存しないコード最適化を適用することができる．

```
int fib(int x) {
  if (x=0) {
    return 0;
  } else if (x=1) {
    return 1;
  } else {
    return fib(x-2)+fib(x-1);
  }
}

print fib(4);
```

```
  define @fib(x) {
    t1 := x
    t2 := (x=0)
    if not t1 goto l1
    return 0
l1: t3 := (x=1)
    if not t2 goto l2
    return 1
l2: t4 := t1 - 2
    t5 := call fib(t4)
    t6 := t1 - 1
    t7 := call fib(t6)
    t8 := t5 + t7
    return t8
}

define @main() {
    t1 := 4
    t2 := call fib(t1)
    print t2
}
```

図 6.5: 3 番地コードへの変換例

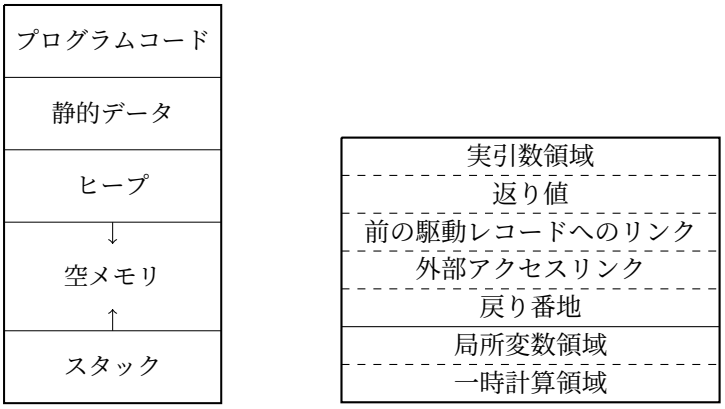
図 6.5 に変換例を示す．ここでは，関数呼び出しとして@関数名　という形をとっている．

6.2　実行環境

プログラミング言語として局所変数を持つことが一般的である．オブジェクト指向言語では，インスタンスが生成されると変数領域を割り当てて実行が終了したとき領域を解放する必要がある．命令型では，関数・手続きが呼び出されたときには，関数・手続きのみで有効な局所変数を割り当てることでプログラムを合成的に記述することが可能になっている．局所変数は関数・手続きの呼び出しごとに動的に割り当てられることで，再帰的な関数定義が可能になっている．関数と手続きの呼び出しは，入れ子構造であることから，**駆動レコード** (Activation record) という単位でスタック構造を操作することで実行環境を実現する．単純に**スタックフレーム** (stack frame) と呼ばれることもある．

6.2.1　駆動レコードの構成

プログラムを実行するプロセスの全体のメモリは図 6.6a のような形となる場合が多い．駆動レコードの構成は一般には図 6.6b のような構成となる．



(a) メモリ配置の例　　(b) 駆動レコードの例

図 6.6: 実行環境のメモリ配置

1. 実引数領域：引数を評価して値を格納して関数・手続きに引き渡す．
2. 返り値：関数の計算が終了して得られる値を格納する．
3. 駆動リンクへのリンク：スタックの格納単位を示す．
4. 外部アクセスリンク：大域変数などへの参照のための番地を格納する．
5. 戻り番地：関数・手続きが終了したときに実行するプログラムカウンタの番地を格納する．
6. 局所変数領域：関数・手続きの実行時に割り当てられる変数領域を示す．
7. 一時計算領域：関数・手続きの実行に必要な一時変数領域を示す．

駆動レコードを単位として，プロセスのメモリ領域に割り当てられる．空きメモリの量はプロセス起動時に決定されることが多い．十分な量を見込んで割り当てられるが，無限ループなどで異常に多くの関数呼び出しが発生するとプロセスのメモリ限界を超える．この場合は，通常，オペレーティングシステムが異常を検出してプロセスが異常終了する．

関数呼び出しとリターン

関数（手続き）の呼び出しは以下の順序で行われる．

1. スタック上に駆動レコードを確保する．
2. 実引数の値を評価して，呼び出しが起こる駆動レコードの領域に書き込む．
3. 呼び出しが発生したプログラムカウンタ PC の値+1 をスタックに格納
4. 前の駆動レコードの先頭のリンクを格納
5. 局所領域をスタック上に確保
6. 呼び出された関数の命令の頭にジャンプする．

関数（手続き）において，リターンが実行されたとき．

1. 関数であれば返し値を駆動レコードの領域に格納
2. 戻りアドレスを駆動レコードから取り出す．
3. 駆動レコードの先頭のリンクを取り出して，現在の駆動レコードを破棄
4. 戻りアドレスへジャンプ

例 6.1. 以下の C 言語風のプログラムに対する実行環境の概要は図**??**のようになる．ここでは，駆動レコードのうちの変数領域のみの状況を示している．Rv は関数の返り値を表す．

```
1  int a;
2
3  int func(int x) {
4    int y;
5    if (x == 1) { return(a);}
6    else {
7      y = func(x-1) * 2;
8    }
9    return (a+y); }
10
11 void main() {
12   int b;
13   b = 3;
14   a = 2;
15   printf("%d\n",func(b)); }
```

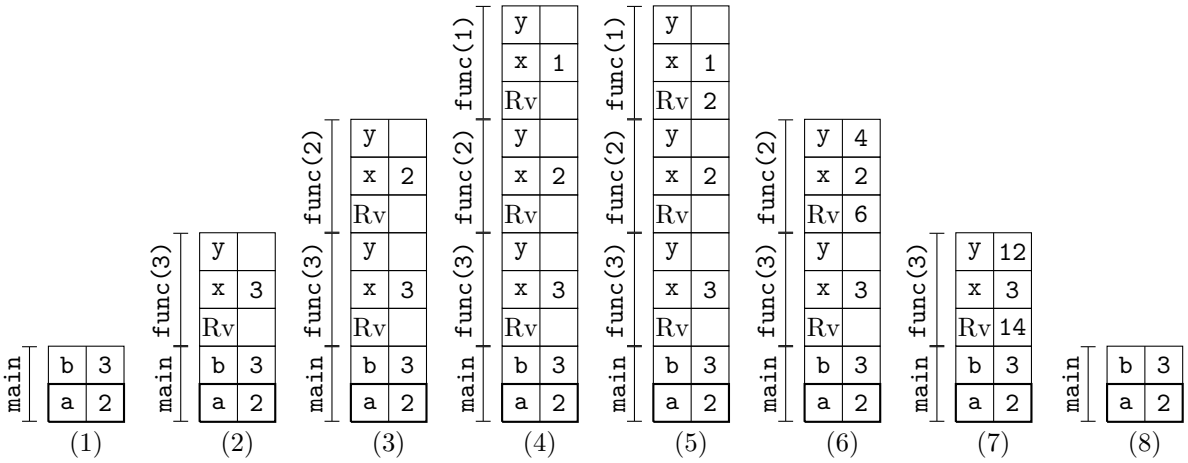


図 6.7: 実行環境の変化

(1) `main` の実行中

(2) `func(3)` の呼び出し直後

(3) `func(3)` から `func(2)` を呼び出した直後

(4) `func(2)` から `func(1)` を呼び出した直後

(5) `func(1)` の返り値を `a` の値として 2 をセット

(6) `func(1)` の駆動レコードを破棄して，`func(2)` の返り値を $2 * 2 + 2$ で *6* にセット

(7) `func(2)` の駆動レコードを破棄して，`func(3)` の返り値を $6*2+2$ で *14* にセット

(8) `func(3)` の駆動レコードを破棄.

6.3 構文主導型変換

中間コードは構文解析と同時に生成することができる．このような方法を**構文主導型変換** (Syntax Directed Translation, SDT) と呼ぶ．この変換では生成規則やその途中にコード生成ルーチンを割り当てて，規則の適用にしたがって対応するコード生成ルーチンを呼び出して中間コードを生成する.

6.3.1 構文主導型変換の基礎

コード生成ルーチンとして以下の 3 つの関数を用いる.

- `newTemp()`
  あたらな領域を割り当ててそのメモリ番地を返す．`newTemp()` を実行すると，新たに割り当てられた領域をさすメモリ番地が返り値となる．`x` に新たに領域を割り当てるには，`x.var` := `newTemp()` とする．ここでは，駆動レコード上の局所領域が割り当てられる．関数などの終了によって局所領域が解放されるときに自動的に解放される[1].

- `newLabel()`
  新たなラベルをわりあててそのラベルの命令番地を返す．割り当てられる領域を $L1, L2, \cdots$ で参照する.

- `generate`
  S引数の 3 番地コードを生成して出力に書き込む．各コードとの対応を以下のように定める.

$C = A\theta B$	`generate`(‘C=$A\theta B$’)
$B = \theta A$	`generate`(‘B=θA’)
$B = A$	`generate`(‘B=A’)
if A goto L	`generate`(‘`if` A `goto` L’)
if_not A goto L	`generate`(‘`if_not` A `goto` L’)
goto L	`generate`(‘`goto` L’)
L_i :	`generate`(‘`label` L_i’)

例 6.2. 以下の生成規則を持つ *CF* 文法を考える.

$$
\begin{aligned}
S &\rightarrow \mathsf{ID} = E\ ; \\
E &\rightarrow E^{(1)} + \mathsf{NUM} \\
E &\rightarrow \mathsf{NUM}
\end{aligned}
$$

中間コード生成は，生成規則に生成ルーチンを対応づけることで実現される.

[1] ヒープ領域のような場合は，明示的に解放する必要がある.

生成規則	コード生成ルーチン
$E \to \mathsf{NUM}$	{$E.var$:=newTemp*();*
	generate*('E.var*=NUM.*val')*}
$E \to E^{(1)} + \mathsf{NUM}$	{$E.var$:=newTemp*();*
	generate*('E.var*=$E^{(1)}$*.var*+NUM.*val');*}
$S \to \mathsf{ID} = E$;	{generate*('*ID.*name*=$E.var$*')*}

ここで，$E.var$ は，非終端記号 E に対して割り当てられる変数，NUM.val はトークン NUM が表す数値，ID.$name$ は，識別子 ID の名前を表す文字列を表す．

ID=NUM_0+NUM_1+NUM_2; に対する中間コード生成は以下の通りである．

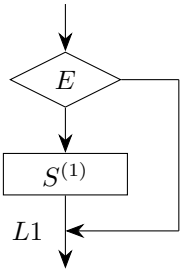
還元系列		生成中間コード
	ID=NUM_0+NUM_1+NUM_2;	$t1 = \mathsf{NUM}_0.val$ ($E_1.var = t1$)
$\Leftarrow$	ID=E_1+NUM_1+NUM_2;	$t2 = t1$+$\mathsf{NUM}_1.val$ ($E_2.var = t2$)
$\Leftarrow$	ID=E_2+NUM_2;	$t3 = t2$+$\mathsf{NUM}_2.val$ ($E_3.var = t3$)
$\Leftarrow$	ID=E_3;	ID.$name = t3$
$\Leftarrow$	S	

実行制御を含む文に対する SDT

条件文のコード生成は以下のようになる．

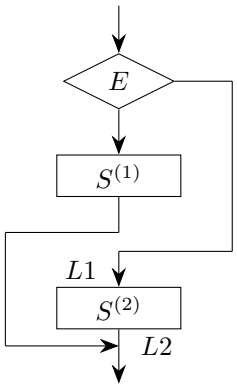
$S \to$ `if` (E) $S^{(1)}$ $\Rightarrow$

1 : E のコードを生成 $E.var = t1$
2 : `if not` $t1$ `goto` $L1$
3 : $S^{(1)}$ のコードを生成
4 : $L1$



$S \to$ `if` (E) $S^{(1)}$ `else` $S^{(2)}$ $\Rightarrow$

1 : E のコードを生成 $E.var = t1$
2 : `if not` $t1$ `goto` $L1$
3 : $S^{(1)}$ のコードを生成
4 : `goto` $L2$
5 : $L1$: $S^{(2)}$ のコードを生成
6 : $L2$:



構文解析を生成規則に沿って進めるためには，条件文の生成規則が適用されたときに全体のコードを生成するのではなく，条件式 E が還元されたときに，E のコードを生成して出力する必要がある．

$$
\begin{array}{lll}
S \rightarrow & \texttt{if (} E \texttt{)} & \{S.label\texttt{:=newLabel}(); \\
 & & E \text{ のコード生成 } (E.var \text{ に結果を格納}) \\
 & & \texttt{generate(`if_not } E.var \texttt{ goto } S.label'\texttt{);}\} \\
 & S & S \text{ のコード生成} \\
 & & \{\texttt{generate(`label } S.label'\texttt{);}\}
\end{array}
$$

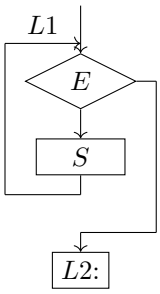
$$
\begin{array}{lll}
S \rightarrow & \texttt{if (} E \texttt{)} & \{S.label\texttt{:=newLabel}(); S.else\texttt{:=newLabel}() \\
 & & E \text{ のコード生成 } (E.var \text{ に結果を格納}) \\
 & & \texttt{generate(`if_not } E.var \texttt{ goto } S.else'\texttt{);}\} \\
 & S^{(1)} & S^{(1)} \text{ のコード生成} \\
 & & \{\texttt{generate(`goto } S.label'\texttt{);}\} \\
 & \texttt{else} & \{\texttt{generate(`label } S.else'\texttt{);}\} \\
 & S^{(2)} & S^{(2)} \text{ のコード生成} \\
 & & \{\texttt{generate(`label } S.label'\texttt{);}\}
\end{array}
$$

`while` 文に対する構文主導型変換は以下の順番に行われる．

$$S \rightarrow \texttt{while (} E \texttt{)} S$$

この生成規則に対する中間コードは以下のようになる．

1： L_1 :
2： E のコード (t1=$E.var$)
3： `if_not` t1 `goto` L_2
4： S のコード
5： `goto` L_1
6： L_2 :



コード生成ルーチンを伴う生成規則は以下のようになる．

$$
\begin{array}{lll}
S & \rightarrow & \texttt{while (} \quad \{S.label1 := \texttt{newLabel}(); \\
 & & \quad \texttt{generate}(\text{`}\texttt{label } S.label1')\} \\
 & & E \qquad (E \text{ のコード } (E.var \text{ に結果格納})) \\
 & & \texttt{)} \qquad \{S.label2 := \texttt{newLabel}(); \\
 & & \quad \texttt{generate}(\text{`}\texttt{if_not } E.var \texttt{ goto } S.label2') \\
 & & S \qquad S \text{ のコード生成} \\
 & & \quad \{\texttt{generate}(\text{`}\texttt{goto } S.label1'); \\
 & & \quad \texttt{generate}(\text{`}\texttt{label } S.label2')\}
\end{array}
$$

6.3.2　上昇型構文主導型変換

LR パーザーにコード生成ルーチンを組み込む場合，コード生成は還元のタイミングで行う．したがって，条件分岐，ループのように生成規則の途中でコード生成を行う必要がある場合には，新たな非終端記号を導入すれば還元の際にのみコード生成ルーチンを呼び出すことができる．

$$S \rightarrow \texttt{if (} E \texttt{)} \ S^{(1)} \ \texttt{else} \ S^{(2)}$$

についてはフローチャートに沿って以下のような生成ルーチンを付加することになる．

この生成規則に対して，コード生成が規則を還元する場合にのみ発生する等価な生成規則を以下のように構成する．

$$\{I \rightarrow \texttt{if (} E \texttt{)}, \quad S' \rightarrow IS^{(1)}, \quad S \rightarrow S' \ \texttt{else} \ S^{(2)}\}$$

$$
\begin{array}{lll}
I & \rightarrow & \texttt{if (} E \texttt{)} \qquad \{I.label = \texttt{newLabel}(); \\
 & & \quad \texttt{generate}(\text{`}\texttt{if_not } E.var \texttt{ goto } I.label')\} \\
S' & \rightarrow & IS^{(1)} \qquad \{S'.label = \texttt{newLabel}(); \\
 & & \quad \texttt{generate}(\text{`}\texttt{goto } S'.label'); \\
 & & \quad \texttt{generate}(\text{`}\texttt{label } I.label')\} \\
S & \rightarrow & S' \ \texttt{else} \ S^{(2)} \quad \{\texttt{generate}(\text{`}\texttt{label } S'.label')\}
\end{array}
$$

`while` 文について以下のような生成規則を構成する．

$$\{I \rightarrow \texttt{while}, \quad S' \rightarrow \texttt{(} E \texttt{)}, \quad S \rightarrow IS'S^{(1)}\}$$

$$
\begin{array}{lll}
I & \rightarrow & \texttt{while} \quad \{I.label = \texttt{newLabel}(); \\
 & & \quad \texttt{generate}(\text{`}\texttt{label } I.label')\} \\
S' & \rightarrow & \texttt{(} E \texttt{)} \quad \{S'.label = \texttt{newLabel}(); \\
 & & \quad \texttt{generate}(\text{`}\texttt{if_not } E.var \texttt{ goto } S'.label')\} \\
S & \rightarrow & IS'S^{(1)} \quad \{\texttt{generate}(\text{`}\texttt{goto } I.label'); \\
 & & \quad \texttt{generate}(\text{`}\texttt{label } S'.label')\}
\end{array}
$$

6.3.3 下降型構文主導型変換

LL(1) 下降型構文解析にコード生成に組み込む場合は，より直観的にコード生成することができる．生成規則の右辺で生成ルーチンに到達すれば，そのルーチンを実行すればよい．このことは下降型の構文解析では，構文図を用いて示される．

構文図をたどる際に構文要素の項目に応じてコードを生成する．

6.3.4 構文主導定義による意味定義

構文主導定義 (Syntax-Directed Translation, SDD) は生成規則の構文要素に「属性」を割り当てることによって，特定の意味を与えることができる．構文主導定義では，コード生成ルーチンを陽に与える代わりに，構文要素間に成立する等式を意味規則として与える．形式言語理論における属性文法 (Attributed Grammar) に基づく．非終端記号 X に対して $X.a$ は，X に対する属性 a を表す．例えば，生成規則 $E \to E^{(1)}+\mathsf{NUM}$ に対して，各構文要素がもつ値属性 (val) に対する関係 $E.val = E^{(1)}.val + \mathsf{NUM}.val$ によってこの生成規則が与える意味を定義する．属性は，数，型，表参照，文字列など，どういった種類の値でもよい．

例 6.3. 以下の意味規則つきの生成規則を持つ文法 $G_A = (\{L, E, T, F\}, \{+, *, (,), \mathsf{NUM}, \mathbf{n}\}, P, L)$ を考える．

$$P = \left\{ \begin{array}{lll} 0: & L \to E\ \mathbf{n} & \{L.val = E.val\} \\ 1: & E \to E_1 + T & \{E.val = E_1.val + T.val\} \\ 2: & E \to T & \{E.val = T.val\} \\ 3: & T \to T_1 * F & \{T.val = T_1.val \times F.val\} \\ 4: & T \to F & \{T.val = F.val\} \\ 5: & F \to (E) & \{F.val = E.val\} \\ 6: & F \to \mathsf{NUM} & \{F.val = \mathsf{NUM}.val\} \end{array} \right\}$$

規則 $0 : L \to E\ \mathbf{n}$ は，計算終了記号 $\mathbf{n}$ を入力することで，値計算をすることを表す．NUM は数のトークンであり，$\mathsf{NUM}.val$ は，そのトークンが表す属性としての値である．図 *6.8* は $3 + 4 * 5$ に対する構文解析木とその属性評価である．

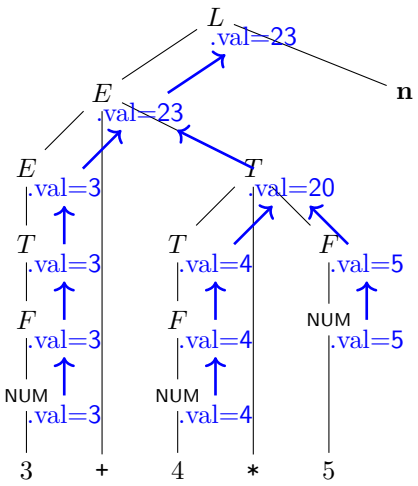


図 6.8: 解析木と属性評価

定義 6.1. 文脈自由文法 $G = (N, \Sigma, P, S)$ に対する *SDD* は $(G, \mathcal{A}, E)$ として与えられる．

- $\mathcal{A}$ は属性の集合である．$X \in N \cup \Sigma$ に対して，$\mathcal{A}|_X$ は，X に割り当てられた属性の集合を表す．$X.a$ は，X に割り当てられた属性値を表し，$X.a \in \mathcal{A}$ である．
- E は生成規則 $p \in P$ に対して，意味規則を与える．意味規則は，属性間に成立する等式の集合である．属性間に成立する等式は以下の形を持つ．

  $$X_{i_1}.a_{j_1} = f(X_{i_2}.a_{j_2}, \cdots, X_{i_n}.a_{j_m}) \in E(p)$$

  であるとき，X_k は p に現れる $N \cup \Sigma$ である．f は意味関数であり，引数となる属性に対して定義されているとする．f は引数となる属性に対して計算可能な関数であり，計算手続きはメモリ更新などの副作用を持たない．
- 意味規則の等式において，X_{i_1} が p の左辺の非終点記号であるとき，$X_{i_1}.a_{j_1}$ を**合成属性 (syntheiszed attribute)** とよび，そうでないとき**相続属性 (inherited attribute)** とよぶ．$X \in N \cup \Sigma$ に対して，合成属性の集合を $S(X)$, 相続属性の集合を $I(X)$ と書く．$X \in \Sigma$ に対して $I(X) = \varnothing$ であり，すべての属性 $X.a$ は，字句解析時に決定している．

E の定義において，以下の *3* つの条件を満たす．

1. $S(X) \cap I(X) = \varnothing$
2. $S(X) \cup I(X) = \mathcal{A}|_X$
3. $Y_1 \to Z_1, \cdots, Z_n \in P$ に対して，$Y.a \in S(Y)$ および $Z_i.a \in I(Z_i)$ のそれぞれに対して，ちょうど 1 つの意味規則の等式が存在する．

例 6.4. 次に与えられる属性文法について考える．

$G = (\{T, L\}, \{\mathsf{max}, \mathsf{min}, \mathsf{NUM}, \text{`}(\text{'}, \text{`})\text{'}, \text{`,'}\}, P, T)$ $\mathcal{A} = \{val, op\}$,

$$P, E = \left\{ \begin{array}{ll} T \to \mathsf{min}\text{`}(\text{'}L\text{`})\text{'} & \{L.op = \mathsf{min}, T.val = L.val\} \\ T \to \mathsf{max}\text{`}(\text{'}L\text{`})\text{'} & \{L.op = \mathsf{max}, T.val = L.val\} \\ L \to \mathsf{NUM} & \{L.val = \mathsf{NUM}.val\} \\ L \to L^{(1)}\text{`,'}\mathsf{NUM} & \{L.val = f(L.op, L^{(1)}.val, \mathsf{NUM}.val), L^{(1)}.op = L.op\} \end{array} \right\}$$

$f(L.op, L^{(1)}.val, \mathsf{NUM}.val)$ は，$L.op(L^{(1)}.val, \mathsf{NUM}.val)$ を計算する意味関数とする．

$S(T) = \{val\}$, $I(T) = \varnothing$, $S(L) = \{val\}$, $I(L) = \{op\}$

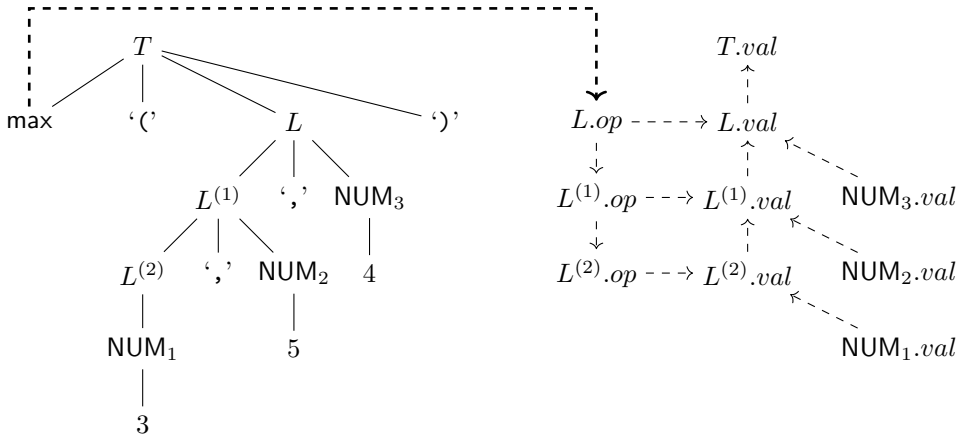


図 6.9: 属性の評価

属性文法において，属性の計算方法が巡回すると属性の計算ができなくなる．例えば，以下の意味規則を持つ場合は属性の計算ができない．

$$A \to B \quad \{A.s = B.i, B.i = A.s + 1\}$$

属性文法に基づく意味を計算する場合は，合成属性に限定する (S-Attribute Definition)，あるいは，相続属性の出現を限定する (L-Attribute Definition) ことによって巡回を避けるための十分条件が知られている．

6.3.5 コード生成の実際

ソースプログラムの仕様

ブロック構造とループ構造を持つ単純なプログラミング言語 PL-Simple を以下のように定義する．(手続き・関数呼び出しは含まない)

⟨program⟩::='program' 'IDENT' ';' ⟨outblock⟩ '.'

⟨outblock⟩::=⟨var_decl_part⟩ ⟨statement⟩

⟨var_decl_part⟩::=⟨var_decl_list⟩';'|/* empty */

⟨var_decl_list⟩::=⟨var_decl_list⟩';'⟨var_decl⟩|⟨var_decl⟩

⟨var_decl⟩::='var'⟨id_list⟩

⟨id_list⟩::='IDENT'|⟨id_list⟩','IDENT'

⟨statement_list⟩::=⟨statement_list⟩';'⟨statement⟩|⟨statement⟩

⟨statement⟩::=⟨assignment_statement⟩|⟨if_statement⟩
|⟨while_statement⟩|⟨for_statement⟩|⟨null_statement⟩
|⟨block_statement⟩|⟨read_statement⟩|⟨write_statement⟩

⟨assignment_statement⟩::='IDENT':='⟨expression⟩

⟨if_statement⟩::='if'⟨condition⟩'then'⟨statement⟩⟨else_statement⟩

⟨else_statement⟩::='else'⟨statement⟩|/ ∗ *empty* ∗ /

⟨while_statement⟩::='while'⟨condition⟩'do'⟨statement⟩

⟨for_statement⟩::='for''IDENT':='⟨expression⟩'to'⟨expression⟩'do'⟨statement⟩

⟨block_statement⟩::='begin'⟨statement_list⟩'end'

⟨read_statement⟩::='read''('IDENT')'

⟨write_statement⟩::='write''('⟨expression⟩')'

⟨null_statement⟩::=/* empty */

⟨condition⟩::=⟨expression⟩'='⟨expression⟩|⟨expression⟩'<>'⟨expression⟩
|⟨expression⟩'<'⟨expression⟩|⟨expression⟩'<='⟨expression⟩
|⟨expression⟩'>'⟨expression⟩|⟨expression⟩'>='⟨expression⟩

⟨expression⟩::=⟨term⟩|'+'⟨term⟩|'-'⟨term⟩|⟨expression⟩'+'⟨term⟩|⟨expression⟩'-'⟨term⟩

⟨term⟩::=⟨factor⟩|⟨term⟩'*'⟨factor⟩|⟨term⟩'div'⟨factor⟩

⟨factor⟩::=⟨var_name⟩|'NUMBER'|'('⟨expression⟩')'

⟨var_name⟩::='IDENT'

ここで，識別子トークン IDENT は [a-zA-Z][a-zA-Z_0-9]*, 数値トークン NUMBER は (+-—)?[0-9]+—で定義されるとする．ここで?は 0 回または 1 回にマッチする文字列，+は 1 回以上の繰り返しにマッチする文字列を表す.

6.3.6 ３番地コード生成規則の挿入

基本構造・宣言

declare(識別子名) は識別子名を記号表に登録する手続き，lookup(識別子名) は記号表を参照して識別子名が格納されている番地を返す手続きであり，記号表に登録がない場合はエラーを返す．generate(s) は３番地コードを１行出力する．

IDENT.lexme は識別子トークン IDENT に格納されている識別子名の文字列を示す．$N.attr$ は，非終端記号 N が属性 $attr$ として保持する値である．(例えば，E.var には，E が属性 var として保持する値である．)

```
P  → 'program' IDENT ';' D S '.'

D  → DL ';' | ε
DL → DL ';' V | V
V  → 'var' I
I  → IDENT
       { declare(IDENT.lexeme); }
     | I ',' IDENT
       { declare(IDENT.lexeme); }

SL → SL ';' S | S
S  → A | IF | WHILE | FOR | N | B | READ | WRITE
N  → ε
B  → 'begin' SL 'end'

A  → IDENT ':='
       { A.target := lookup(IDENT.lexeme); }
       E
       { generate(A.target " := " E.var); }

READ → 'read' '(' IDENT ')'
         { generate("read " lookup(IDENT.lexeme)); }

WRITE → 'write' '(' E ')'
          { generate("write " E.var); }
```

算術式・条件式

```
E → T
      { E.var := T.var; }
    | '+' T
      { E.var := T.var; }
    | '-' T
      { E.var := newtemp();
        generate(E.var " := 0 - " T.var); }
    | E1 '+' T
      { E.var := newtemp();
        generate(E.var " := " E1.var " + " T.var); }
    | E1 '-' T
```

```
      { E.var := newtemp();
        generate(E.var " := " E1.var " - " T.var); }

T → F
      { T.var := F.var; }
    | T1 '*' F
      { T.var := newtemp();
        generate(T.var " := " T1.var " * " F.var); }
    | T1 'div' F
      { T.var := newtemp();
        generate(T.var " := " T1.var " div " F.var); }

F → X                 { F.var := X.var; }
    | NUMBER           { F.var := NUMBER.value; }
    | '(' E ')'        { F.var := E.var; }
X → IDENT             { X.var := lookup(IDENT.lexeme); }

C → E1 R E2
      { C.var := newtemp();
        generate(C.var " := " E1.var " " R.op " " E2.var); }
R → '='  { R.op := "="; }  | '<>' { R.op := "<>"; }
    | '<'  { R.op := "<"; }  | '<=' { R.op := "<="; }
    | '>'  { R.op := ">"; }  | '>=' { R.op := ">="; }
```

条件文

```
IF → 'if' C 'then'
       { IF.false := newlabel();
         generate("if not " C.var " goto " IF.false); }
       S
       { generate("label " IF.false); }

IF → 'if' C 'then'
       { IF.false := newlabel();
         IF.end := newlabel();
         generate("if not " C.var " goto " IF.false); }
       S1
       { generate("goto " IF.end); }
       'else'
       { generate("label " IF.false); }
       S2
       { generate("label " IF.end); }
```

While 文

```
WHILE → 'while'
          { WHILE.start := newlabel();
            generate("label " WHILE.start); }
          C 'do'
```

```
        { WHILE.end := newlabel();
          generate("if not " C.var " goto " WHILE.end); }
        S
        { generate("goto " WHILE.start);
          generate("label " WHILE.end); }
```

For 文

```
FOR → 'for' IDENT ':='
        { FOR.target := lookup(IDENT.lexeme); }
        E1 'to'
        { generate(FOR.target " := " E1.var); }
        E2 'do'
        { FOR.bound := newtemp();
          generate(FOR.bound " := " E2.var);
          FOR.start := newlabel();
          FOR.end := newlabel();
          FOR.test := newtemp();
          generate("label " FOR.start);
          generate(FOR.test " := " FOR.target " <= " FOR.bound);
          generate("if not " FOR.test " goto " FOR.end); }
        S
        { FOR.next := newtemp();
          generate(FOR.next " := " FOR.target " + 1");
          generate(FOR.target " := " FOR.next);
          generate("goto " FOR.start);
          generate("label " FOR.end); }
```

6.3.7 例

以下にコンパイル例を示す.

```
program PL0A;
var n, sum;
begin
  n := 10;
  sum := 0;
  while n > 0 do
  begin
    sum := sum + n;
    n := n - 1;
  end;
  write(sum)
end.
```

```
 1  n := 10
 2  sum := 0
 3  label l1
 4  t1 := n > 0
 5  if not t1 goto l2
 6  t2 := sum + n
 7  sum := t2
 8  t3 := n - 1
 9  n := t3
10  goto l1
11  label l2
12  write sum
```

```
program PL0B;
var m, n, x;
begin
  read(n);
  while 1 < n do
    begin
      x := n;
      begin
        m := x div 2;
        while x <> (x div m) * m do
          m := m - 1;
        if m = 1 then
          write(x)
      end;
      n := n - 1
    end
end.
```

```
1  read n
2  label l1
3  t1 := 1 < n
4  if not t1 goto l2
5  x := n
6  t2 := x div 2
7  m := t2
8  label l3
9  t3 := x div m
10 t4 := t3 * m
11 t5 := x <> t4
12 if not t5 goto l4
13 t6 := m - 1
14 m := t6
15 goto l3
16 label l4
17 t7 := m = 1
18 if not t7 goto l5
19 write x
20 label l5
21 t8 := n - 1
22 n := t8
23 goto l1
24 label l2
```

```
program PL0C;
var i,r;
begin
  for i:=1 to 10 do
    r:=r+i*i;
  write(r);
end.
```

```
1  i := 1
2  t1 := 10
3  label l1
4  t2 := i <= t1
5  if not t2 goto l2
6  t3 := i * i
7  t4 := r + t3
8  r := t4
9  t5 := i + 1
10 i := t5
11 goto l1
12 label l2
13 write r
```

コンパイラおよび 3 番地コードのインタープリタについては講義ページを参照.

第7章　コード最適化

本章では，中間コードのレベルで意味を保ちながらより高速なコードに変換する手法について述べる．ソースコードから構文的に生成された中間コードは不要な繰り返しや変数間のコピーが多く生成されるため，制御フローとデータフローを中間コードを解析して同じ効果をもつより効率的なコードに変換する．この過程を**コード最適化 (code optimization)** と呼ぶ．さらに，最近のコンパイラで用いられている　**静的単一代入形式 (static single assignment form, SSA)** を用いた最適化手法を紹介する．SSA の特徴は，その名前の通り，変数に対する代入を一度に制限して上書きしない．SSA では，変数がより多く必要になる一方，変数の名前によってデータフローが構文として表現されるため，コンパイラの最適化の際に必要なフロー解析が容易になる．

7.1　基本ブロックとフローグラフ

3 番地コードレベルの抽象度を持つ言語の命令の系列に対して，以下の条件を満たす系列を**基本ブロック**という．

実行制御がその系列の先頭から最後まで，条件付きジャンプおよび無条件ジャンプは最後の場所でのみ出現し，他の命令からのジャンプ先は必ず系列の先頭しかない．

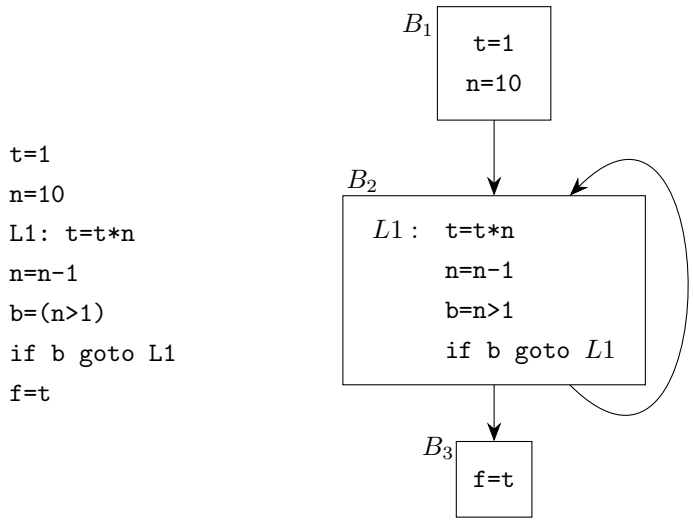


図 7.1: 3 番地コードとその CFG

基本ブロックでは，すべての命令の前提としてそれより前にある命令が必ず実行されている．3

番地コードから構成される基本ブロック内では，参照される変数について，それより前のブロック内で定義されていれば，最後に定義されたところの値を持つことがわかる．そうでなければ，先頭にジャンプしているブロック内の値が引き継がれる．

基本ブロックを頂点として，制御のフローを有向グラフで表した構造を**フローグラフ（あるいは，コントロールフローグラフ，CFG）**と呼び．(V, E, v_{init}) で表す．ここで，V は頂点の集合で，各頂点 v には基本ブロック $B(v)$ が割り当てられている．$(v_1, v_2) \in E$ であるとき，$B(v_1)$ の実行後，直ちに $B(v_2)$ が実行されることが可能であることを表し，$v_1 \rightarrow v_2$ と書くことにする．$v_{init} \in V$ は，フローグラフの実行が $B(v_{init})$ から開始されることを表す．

基本ブロック中では，制御の合流や分岐が存在しないので，CFG はプログラムの振舞いを抽象的に表現していることになる．

7.2　局所最適化

局所最適化は，3 番地コードの系列で構成される基本ブロックに対して同じ意味を持つより短くて変数の少ない系列を求める．

7.2.1　局所最適化の例

定数伝播 (constant propagation)

定数畳み込み (constant folding) とも言う．実行によって値が変わらない変数は，プログラムとして値を埋め込んでメモリ上の値の参照を減らす．

```
a=2*3.14   →   a=6.28      定数計算をコンパイル時に行う．
b=a/100    →   b=0.0628    b はコンパイル時に値が決定できる．
```

変数の値を参照する場合には，メモリから値を読み出す必要があるが，定数にすると値自体をコードとして生成できるので，高速になる．

共通部分式削除

同じ部分式で同じ計算の繰り返しをまとめて，計算の繰り返しを防ぐ．

```
x=a*b          x=a*b
y=x+c          y=x+c
d=a*b    ⇒     d=x
e=d-c          e=d-c
```

以下の系列では，同じ形の部分式 `b+c` があるが，`a` と `e` に対する代入で値が更新されているので，`e=a` のような消去はできない．

```
a=b+c        a=b+c
b=b-d        b=b-c
c=c+d        c=c+d
e=b+c   ⇏    e=a
```

コピー伝播

a=b のような変数のコピーがある場合，a と b は同じ値を保持しているので，a を代入する必要はない．

```
a=b          a=b
c=a+d   ⇒    c=b+d
e=e-a        e=e-b
```

このブロックの後で，a が使われることがなければ，a=b は消去しても構わない．基本ブロックで用いられる変数については，データフロー解析などの手法を用いて求めることができる場合がある．不要コードの消去 (Dead-code elimination) によってより高速にすることができる．

代数的性質の利用

```
a+0          ⇒   0
b*1          ⇒   b
1+(c+2)      ⇒   3+c
d*(1+e/d)    ⇒   d+e
x*2          ⇒   x+x
x/5          ⇒   x*0.2
```

演算の単位元などによる簡単化，分配則による簡単化などによって式を短縮化する．さらに命令を低レベルのマシン語に変換するときに，除算よりも積算，積算よりも和算の方がサイクルが少ないので，より軽い命令に変更することで高速化が期待できる．

局所最適化については，個々の最適化について大きな効果は期待できない．しかし，基本ブロックがループによって繰り返し実行されるとプログラム全体では大きな効果が期待できる．

7.2.2 基本ブロックの DAG 表現

基本ブロックを有向非巡回グラフ (Directed Acyclic Graph, DAG) を用いて表すことによって，各 3 番地コードにおける参照関係を表現できる．参照関係を壊さない等価なグラフの変形によって最適化を行う．基本ブロックに対して，以下のアルゴリズムによって DAG を生成する．

アルゴリズム 7.1. *DAG* の頂点が始点となる辺を持たない頂点を葉頂点と呼ぶ．葉頂点には，変数または定数のラベルをつける．葉以外の頂点には，演算 θ と，演算の結果が収納される変数のラベルのリストをつける．

1. 基本ブロックに入る前に定義された値を参照する変数名と定数値に対して，葉頂点を生成し，変数 x に対して，x_0 を頂点にラベルづけし，$[x]$ を変数リストとする．定数 c に対しては，c を頂点にラベル付けする．

2. 基本ブロック内のすべての代入文について上から順に以下の操作を適用する．

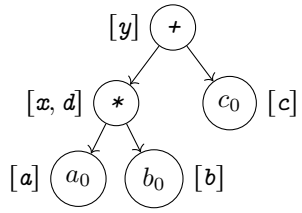
   (a) 代入文 $X = A\theta B$ に対して，A,B が変数であるときは変数ラベルに持つ頂点を v_A, v_B とする．定数の場合は定数をラベルにもつ v_A, v_B を選ぶ．

   (b) X を変数リストに含む頂点があれば，その変数リストから X を削除する．

   (c) v_A, v_B を子に持ち，θ を演算ラベルとするノード v_θ がすでに存在するならば，v_θ の変数リストに X を追加する．v_θ が存在しないならば，v_θ を新たに生成して，v_A, v_B を子ノードとする辺を追加し，θ を v_θ の演算ラベルとし，$[X]$ を変数リストとする．

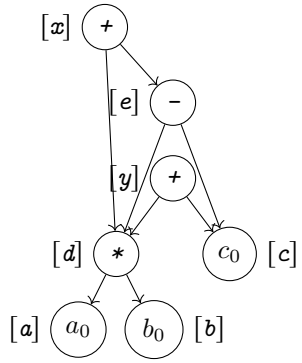
   (d) $X = A$ の場合は，すでに存在する頂点の変数リストに X が含まれれば，消去した上で，v_A の変数リストに X を追加する．

例 7.1. 以下にアルゴリズムで生成される *DAG* を示す．５行の基本ブロックにおいて，最初の３行までで，まず *(a)* に示す *DAG* が生成される．




```
x=a*b
y=x+c
d=a*b
```

(a) ３行目までで生成される *DAG*

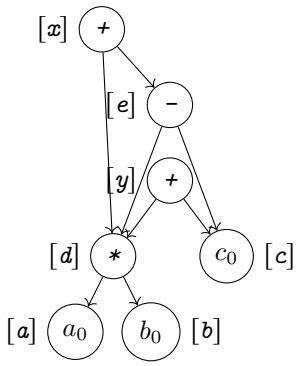



```
x=a*b
y=x+c
d=a*b
e=d-c
x=d+e
```

(b) 5 行目までで生成される *DAG*

４行目の命令でさらに `e` に対するノードが新たに生成され，５行目では `x` が更新されるため，既存の変数リストから `x` を消去して新しく一番上のノードに `x` を変数リストに追加している．

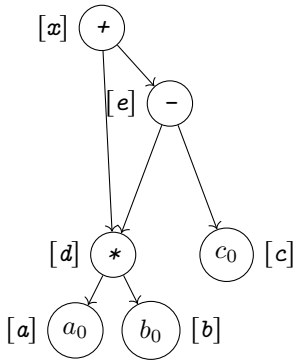
例 *7.1* の *(a)* において，$[x, d]$ のように複数の変数が一つのノードにつけられる場合は，共通部分式が存在するということであるので共有部分式の消去が適用できる．さらに，コピー伝播を適用すると *(c)* のようになる．




```
x=a*b
y=x+c
d=x
e=x-c
x=x+e
```

(c) 共有部分式とコピー伝播の適用

この基本ブロックの後で y が使われない場合は，$[y]$ を変数リストにもつ頂点とその辺を消去しても意味は変わらない．これに対して，d が使われない場合は，$[d]$ の頂点を消すことはできない．これは，x が d の頂点に依存しているためである．




```
x=a*b
d=x
e=x-c
x=x+e
```

(d) 不要コード消去の適用

7.3 大域最適化

複数の基本ブロックにわたる最適化を大域的最適化とよぶ．この最適化を行うために，フローグラフの構造にマッチする構造を見つける方法について示す．

7.3.1 意味等価な変換

プログラムにおいて，全体の１，２割の部分が，実際の実行の８，９割の時間を占める．これは，プログラムの一部がループ構造になって繰り返し実行されることによる．繰り返しに含まれる基本ブロックを局所最適化によって最適化することによって大きな効果が期待できることは述べた．これに加えて，ループ構造として最適化する手法について示す．

ループ構造の最適化は，制御フローグラフに依存するため，ソースコードでも３番地コードでもほぼ変わらない．以下では，C 言語プログラムの一部で最適化の手法について示す．

ループ最適化

ループ不変式:　ループ内で評価が変わらない式である．

ループ不変式の移動: ループ内に `t :=` $a_1 \oplus a_2$ という式があり，この定義式を d とする．
($\oplus$ は任意の 2 項演算子) a_i に対して以下の少なくとも 1 つの条件がなりたつとき，ループ不変式である．

1. a_i は定数
2. a_i のすべての定義はループの外
3. a_i のうちの 1 つだけが，d に到達し，それはループ不変式の定義

ループ不変式をループの前に移動し，計算に影響しない命令が何度も実行されずに同じ意味を表現するようにする．

例 7.2.

```
i=1;                                  i=1;t=a/b;
do {                                  do {
    c=a/b;x=i*c; .... i++;      ⇒         c=t;x=i*c;...i++;
} while(i<n);                         } while(i<n);
```

副作用がある場合は，条件分岐によって副作用を考慮して不変式を移動する．
`do` ... `while` で，`a`, `b` のいずれも更新されないとき，`c` はループ実行で変化しないため，ループ外で値を一度代入する同じ計算となる．

例 7.3.

```
for(i=1;i<n;i++) {                    if(n>1) {
    c=a/b;x=i*c;...                       t=a/b;
}                               ⇒         for(i=1;i<n;i++) {
                                              c=t;x=i*c;...
                                          }
                                      }
```

一般的な `for` ループでは，条件によってループの内容が実行されないこともあるため，不変コードはループが実行された場合だけ，実行されるように変換する必要がある．

帰納変数：　帰納変数はループの実行ごとに一定のパターンで値が更新される変数である．**基本帰納変数**はループが一回実行されるごとに 1 加算されるか減算される変数である．i が基本帰納変数であるとき，i の**派生帰納関数**は，$j = a_j + i \times b_j$ の形をもち，(i, a, b) と書く．ここで，a, b はループ不変式である．例 7.4(a) において，`i` は，L1 のループで，`i` は，$(i, 0, b)$ であり，`j` は，$(i, 0, 4b)$ である．より効率的にするために j' を $(i, 0, 4b)$ として導入する．

例 7.4.

```
    s = 0
L1: if s > 0 goto L2
    i = i + b
    j = i * 4
    x = M[j]
    s = s - x
    goto L1
L2: i = i + 1
    s = s + j
    if i < n got L1
```

(a) 最適化前

⇒

```
    s = 0
    j'= i * 4
    b'= b * 4
    n'= n * 4
L1: if s > 0 goto L2
    j'= j'+b'
    j = j'
    x = M[j]
    s = s - x
    got Ll
L2: j'= j' + 4
    s = s + j
    if j'<n' goto L1
```

(b) 最適化後

帰納変数の消去　直接参照されない i を t で置き換える.

例 7.5.

```
t=b;
for(i=1;i<n;i++) {
    a=a+t;t=t+b;
}
```

⇒

```
m=n*b;
for(t=b;t<m;t=t+b)
    a=a+t;
```

ループ内演算の強さ軽減　帰納変数に対する演算を乗算から加算に置き換える．$(j', 0, c \cdot b)$ の帰納変数で，c,b がループ不変式である場合は，j' を $(k, 0, m)$, $m = c \cdot b$ となる新たな帰納変数におきかえる.

例 7.6.

```
for(i=1;i<n;i++)
    a=a+b*i;
```

⇒

```
t=0;
for(i=1;i<n;i++) {
    t=t+b;a=a+t;
}
```

ループ融合　同じ帰納変数によって一度のループで計算する．個別のループで共通な変数がある場合，基本ブロックの数が減ることによって，局所最適化の可能性が高まる.

例 7.7.

```
for(i=1;i<n;i++)
    a[i+1]=a[i]+i;
for(i=1;i<n;i++)
    b[i]=a[i]+i;
```

⇒

```
for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+i;
}
```

ループ分解　ループ分解はループ融合と逆の操作である．ループを分解して得られるメリットは，２つのループが独立していて並列性を高められる可能性がある．例**??**では，ループが一つの場合には，逐次的にループが実行される．ループを分解すると，２つのループが a[i] の値の順番が変わらない限り並列に実行できる．２つのスレッドで計算できれば高速化できる余地が生まれる．

例 7.8.

```
for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+i;
}
```

⇒

```
for(i=1;i<n;i++)
    a[i+1]=a[i]+i;
for(i=1;i<n;i++)
    b[i]=a[i]+i;
```

ループ展開　複数回の繰り返しを帰納変数をずらして展開して，基本ブロックを大きくして最適化の可能性を大きくする．

例 7.9.

```
for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+b[i-1];
}
```

⇒

```
for(i=1;i<n;i++) {
    a[i]=a[i-1]+i;
    b[i]=a[i]+b[i-1];a[i+1]=a[i]+1;
    b[i+1]=a[i+1]+b[i];
}
```

ソフトウェアパイプライン　ループ内の変数更新の依存関係を解析し，帰納変数をずらして並列性が向上するように変換可能な場合がある．

例 7.10.

```
for(i=1;i<1000;i++) {
  a[i]=a[i-1]+1; // Ai
  b[i]=a[i]+b[i-1]; // Bi
  c[i]=b[i]+2;
}
```

(a) パイプライン適用前

⇒

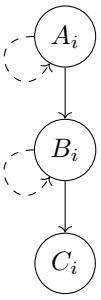
```
a[1]=a[0]+1; // A1
b[1]=a[1]+b[0]; // B1
a[2]=a[1]+1; // A2
for(i=3;i<998,i++) {
  c[i-2]=b[i-2]+2; //Ci-2
  b[i-1]=a[i-1]+b[i-2]; //Bi-1
  a[i]=a[i-1]; // Ai
}
```

(b) パイプライン適用後

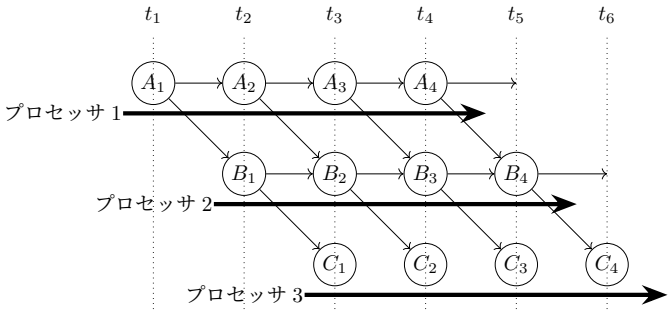
(a) のプログラムでは，ループ内で `a[i]` は `a[i-1]` に依存，`b[i]` は `a[i]` と `b[i-1]` に依存，`c[i]` は `b[i]` に依存する．

このループの依存関係を図で表すと *(c)* のような**ループ運搬依存関係 (loop-carried dependency)** となる．ここでは，各頂点 A_i,B_i,C_i はそれぞれ `a[i]`, `b[i]`, `c[i]` の更新命令を表す．実線の辺は，矢印の根元の命令の実行が終わらないとそれに依存した命令が実行できないことを表し，破線は，帰納変数に依存して自分自身の前の回の実行が終了しないと命令が実行できないことを表す．３個以上のプロセッサが並列に用いることができれば a,b,c の帰納変数をひとつずつずらせば，計算の依存を解消することができ高速に計算できる．帰納変数で展開するとループ内での依

存関係は *(d)* のようになる．３つのプロセッサがある場合，各段階 t_i では，計算に必要なデータは t_{i-1} までに計算されているので，各時点での依存関係はなくパイプラインとして並列に実行できる．



(c) ループ運搬依存関係



(d) 実行における依存関係

手続き呼び出し最適化

インライン展開　手続き呼び出しは，スタック上に局所環境を作成し，リターンの際にもスタック上の局所環境をクリアする必要がある．局所環境を作成するほどでもない単純な手続き呼び出しでは，スタック操作を行わなくてもよい場合もあり，ループ構造内では大きな高速化の効果が期待できる．

```
int square(int x) {
    return x*x;
}
y = square(a);
```

⇒

```
y = a * a;
```

c 言語では，関数の前に `inline` を付加することでスタックを用いずに呼び出しを行う．ただし，`inline` をつけた場合は，実際にインライン展開するかどうかはコンパイラで判断する．必ず展開する場合は，`#define` でマクロ定義すれば呼び出しと同様の形式でプログラムを記述できる．

末尾再帰　再帰呼び出しは，呼び出し定義の中で自分を呼び出すことである．定義の中の呼び出しが定義の最後の命令であるとき，関数からリターンしたあともとの環境の変数は参照されないので，環境を復元する必要がない．従って，インラインでない関数呼び出しを行う場合であっても，毎回，局所環境を作成する必要がなく，最初の１回目の呼び出しで作成した領域を上書きすればよい．返り値も一度しか参照されないので上書きが可能である．

最初の呼び出しで作成した駆動レコードを再利用することによって，再帰呼び出しを goto に置き換えることで高速化が可能である．

例 7.12.

```
int sum_tail(int n, int acc) {
if (n == 0) {
    return acc;
}
return sum_tail(n-1,acc+n);
}
```

⇒

```
int sum_tail(int n, int acc) {
L:  if (n == 0) {
    return acc;
}
acc+=n; n--;
goto L;
}
```

goto によるループ構造を *goto* なしで記述すると以下のプログラムと等価である．

```
int sum(int n,int acc) {
    while(n!=0) {
        acc+=n; n--;
    }
    return acc;
}
```

7.3.2　データフロー解析

プログラムの実行意味が状態遷移に基づいて決まるとすると，プログラムの状態を決定する変数の実行に従った更新（**データフロー**）を解析することでプログラムの実行意味を捉えることが可能になる．基本ブロックを越えて最適化を行うためには，フローグラフ上で各基本ブロックにどの変数が入力されてどの変数を出力するか，ということを解析して，代入された値がどこまで参照として有効か（定義到達解析），以降の実行でどこまでその変数が用いられるか（生存解析），同じ値を計算する式の検出（利用可能式解析）などの解析手法が用いられる．このような解析はプログラムを実行することなく行われるため，**静的解析**という．この解析において，一般にプログラムの成魚がループを持つことから変数の更新は自身の更新に依存するため，この解析においてはデータフロー方程式を解く必要が生じる．

定義到達解析

定義到達解析は，v=E という形の 3 番地コードは v を定義しているとし，ここで定義された v の値がどこまで変更されずに到達するか，ということを解析する．v が別の命令で上書きされるとき，先の定義は kill される．各定義の 3 番地命令にラベル d_i をつけ，基本ブロック B ごとに以下の定義ラベルの集合を定める．

- $Gen[B]$: B のなかで定義され，上書きされない定義の集合
- $Kill[B]$ B のなかで上書きされる定義の集合
- $In[B]$ B が実行される際に参照する変数に対する定義の集合

· $Out[B]$ B が実行された後に有効な定義の集合

ここで Gen, $Kill$ は定義の実行順は関係なく，定義集合全体に対して定める．

基本ブロックから構成されるフローグラフが与えられたとき，以下の関係がある．

$$In[B] = \cup_{P \in \mathsf{Pred}(B)} Out[P] \tag{7.1}$$

$$Out[B] = Gen[B] \cup (In[B] \backslash Kill[B]) \tag{7.2}$$

ここで，$\mathsf{Pred}(B)$ は，フローグラフにおいて，B の先行頂点 (predecessor) の集合を表す．基本ブロック内では，制御の分岐はない．基本ブロックには，その直是のブロックから出力されるフローが入力され，ブロックからの出力フローは，ブロック内で新たに発生した定義と，入力定義のうち kill されなかった定義となる．

図**??**のコードにおいて以下のように定義のラベル付けをする．

```
d1 :        t=1
d2 :        n=10
d3 :  L1 :  t=t*n
d4 :        n=n-1
d5 :        b=(n>1)
            if b goto L1
d6 :        f=t
```

B_i に対する Gen, $Kill$ は以下のようになる．

$Gen[B_1] = \{d_1, d_2\}$, $Kill[B_1] = \{d_3, d_4\}$

$Gen[B_2] = \{d_3, d_4, d_5\}$, $Kill[B_2] = \{d_1, d_2\}$

$Gen[B_3] = \{d_6\}$, $Kill[B_3] = \varnothing$

In, Out については以下の関係がなりたつ．

$In[B_1] = \varnothing$, $Out[B_1] = Gen[B_1] = \{d_1, d_2\}$

$In[B_2] = Out[B_1] \cup Out[B_2]$, $Out[B_2] = \{d_3, d_4, d_5\} \cup (In[B_2] \backslash \{d_1, d_2\})$

$In[B_3] = Out[B_2]$, $Out[B_3] = \{d_6\} \cup (In[B_2] \backslash \{d_6\})$

$In[B_2]$ についての式を代入すると，

$$In[B_2] = \{d_1, d_2\} \cup (\{d_3, d_4, d_5\} \cup (In[B_2] \backslash \{d_1, d_2\}))$$

となる．$In[B_2] = X$ とおくと，$X = \{d_1, d_2, d_3, d_4, d_5\} \cup (X \backslash \{d_1, d_2\})$ を満たす X を求める必要がある．これを解くには，$X_0 = \varnothing$ として，$X_{i+1} = \{d_1, d_2, d_3, d_4, d_5\} \cup (X_i \backslash \{d_1, d_2\})$ で，$X_{i+1} = x_i$ となるまで繰り返す．

$X_1 = \{d_1, d_2, d_3, d_4, d_5\}$, $X_2 = \{d_1, d_2, d_3, d_4, d_5\}$ となるので $X = \{d_1, d_2, d_3, d_4, d_5\}$ が条件を満たす．したがって，$Out[B_2] = \{d_3, d_4, d_5\}$，$out[B_3] = \{d_3, d_4, d_5, d_6\}$ である．

この解析によってこのプログラムが終了した時点での，t, n の有効な定義は，d_3 および d_4 であり，d_1, d_2 の定義は伝播しないということがわかる．また，$In[B_2]$ において，t, n に対する定義は一意的に定まらないので，B_2 中でそれらの変数を定数に置き換えることはできないこともわかる．

生存変数解析

基本ブロック B において，$In[B]$ に含まれる定義の変数が Kill される前に参照されるとき，その定義は B で実際に使用される．この変数の集合を $Use[B]$ とする．B の中で定義される変数の集合を $Def[B]$ とすると，以下の関係が成り立つ．

$$In[B] = Use[B] \cup (Out[B] \backslash Def[B]) \tag{7.3}$$

$$Out[B] = \cup_{S \in \mathsf{Succ}(B)} In[S] \tag{7.4}$$

ここで，$\mathsf{Succ}(B)$ はフローグラフでの B の後続頂点集合である．

図 7.1 について，生存変数解析は以下のようになる．

$Use[B_1] = \varnothing$, $Def[B_1] = \{t, n\}$

$Use[B_2] = \{t, n\}$, $Def = \{t, n, b\}$

$Use[B_3] = \{t\}$, $Def[B_3] = \{f\}$

(7.3)(7.4) を適用する．

$In[B_1] = Out[B_1] \backslash \{t, n\}$, $Out[B_1] = In[B_2]$

$In[B_2] = \{t, n\} \cup (Out[B_2] \backslash \{t, n, b\})$, $Out[B_2] = In[B_2] \cup In[B_3]$

$In[B_3] = \{t\} \cup (Out[B_3] \backslash \{f\})$,$Out[B_3] = \varnothing$

In, Out は出口から計算できる．$In[B_3] = \{t\}$ $Out[B_2] = In[B_2] \cup \{t\}$, $In[B_2] = \{t, n\} \cup ((In[B_2] \cup \{t\}) \backslash \{t, n, b\})$ $In[B_2] = X$ とすると，$X = \{t, n\} \cup (X \backslash \{n, b\})$ となる．$X_0 = \varnothing$ として $X_{i+1} = \{t, n\} \cup (X_i \backslash \{n, b\})$ に代入すると $X_1 = \{t, n\}$, $X_2 = \{t, n\}$ となる．よって，$Out[B_2] = \{t, n\}$, $Out[B_1] = \{t, n\}$, $In[B_1] = \varnothing$ と決定できる．

生存変数解析によって，基本ブロックの実行の後で使用されない変数を特定できる．この解析に基づいて使われない変数の計算を基本ブロックから消去することができる．

図 7.1 に対する利用可能式解析は以下のようになる．

$Gen[B_1] = \varnothing$, $Kill[B_1] = \{$`t*n`, `n-1`, `n>1`$\}$

$Gen[B_2] = \{$`n>1`$\}$, $Kill[B_2] = \{$`t*n`, `n-1`, `n>1`$\}$

$Gen[B_3] = \varnothing$, $Kill[B_3] = \varnothing$

IN,OUT は以下のようになる．

$IN[B_1] = \varnothing$, $OUT[B_1] = \varnothing$

$IN[B_2] = OUT[B_1] \cap OUT[B_2] = \varnothing$ $OUT[B_2] = \{$`n>1`$\}$

$IN[B_3] = OUT[B_2] = \{$`n>1`$\}$, $OUT[B_3] = \{$`n>1`$\}$

実際に利用可能解析は，配列のアドレス計算において `a[i]` のデータにアクセスするのに `a` のアドレス計算に `i*4` を計算する場合などに有効である．

利用可能式解析

$v = a + b$ という代入コードがある場合，この基本ブロックの後で $a + b$ という式が出現しているとき a, b の値が更新されていなければ，計算の重複を避けることが可能になる．変数や定義の代わりに評価式を蓄積する．式はその要素が一つでも更新されると kill される．各基本ブロック B について，B の中において計算された後 B を抜けるまでに kill されることのない式の集合を $Gen[B]$，B の中で kill される式の集合を $Kill[B]$ とする．ここでは，以下の関係が成り立つ．

$$IN[B] = \cap_{\mathsf{Pred}[B]} Out[P] \tag{7.5}$$

$$OUT[B] = Gen[B] \cup (In[B] \backslash Kill[B]) \tag{7.6}$$

7.4 静的単一代入形式と最適化

最近のコンパイラにおける最適化では，静的単一代入形式 (Static single assignment, SSA) を用いる事が多い．SSA 形式では中間コード列の中に各変数の定義（3 番地コードの場合は代入の左辺）が一箇所しか現れないという制約を持つ．定義が 1 つしかないため，どこで定義された変数がどこで参照されるかが，プログラムとして明示される．プログラムを解析すれば，プログラムの構文情報からデータフローを解析することができる．

7.4.1 例

C 言語を SSA に変換する例を示す．

```
scanf("%d",&x);          scanf("%d",&x0);
a=1;                     a0=1;
b=2;                     b0=2;
c=a+b;            ⇒      c0=a0+b0;
if(x>0)a=3;              if(x0>0)a1=3;
d=b;                     a2=φ(a0,a1);
c=a+b;                   d0=b0;
                         c1=a2+b0;
```

この例では，変換前の変数に対して更新される毎に添字を更新して変数を新たに割り当てる．更新を一回に限定することで，添字が同じ変数は，実行フローにおいて同じ値を持つことが保証される．b は，最初に 2 に更新された後，このプログラムの中では更新されることがないので，b_0 が最後まで用いられるので，この値が伝播していることがわかる．a_0 は，最初に 1 に更新され，再度更新されると a_1 に変数名を変えて，3 に更新する．ここでは，x に入力される値は，ユーザーからの入力なのでプログラムを走らせないとわからない．この入力に応じて，最終行の a は，1 または 3 の値となり，プログラムだけでどちらの値になるか不明であるが，どちらかの値になることは解析できる．SSA では，このような静的に解析できないフローに対して φ という特別な関数の形で

表す．このために a_2 という新たな変数を割り当て，$\varphi(a_0,a_1)$ によって値を更新する．φ 関数は，評価する対象でなく，いずれかの選択が実行時になされることを表現している．

ループを持つ場合においても，データのフローを単一代入の形で表すことができる．

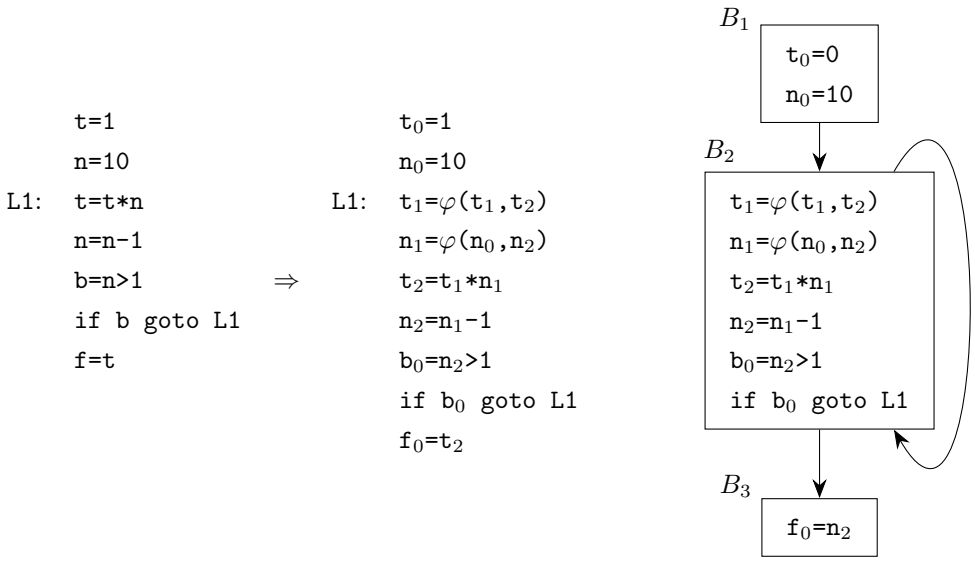


図 7.2: ループを含むプログラムの SSA への変換とフローグラフ

ここでは，L1 の時点で，ループ内の変数は，ループに入る前のデータか，ループの繰り返しからくるデータなのかプログラムからはわからない．t=t*n の右辺の t は，t_0 か t_2 のどちらかである．

7.4.2　φ 関数

SSA 形式のプログラムに変換するためには，プログラムから解析可能な各変数の依存関係を計算し，依存関係が実行によって決まるポイントに φ 関数を挿入する．以下に，φ 関数を挿入する場所を決定する手法について示す．

ここで，コントロールグラフは，有向グラフ (V, E, v_{init}) として与えられているとする．

定義 7.1 (定義合流条件)．変数 x に対する φ 関数が頂点 v に挿入されるための必要十分条件は，以下に示す条件 *(1)*~*(4)* を満たすコントロールグラフ上の路 $p : v_0 \to v_1 \to \cdots \to v_k$, $q : u_0 \to u_1 \to \cdots \to u_\ell$ が存在することである．

(1) 路の長さが *1* 以上である．*($k > 0$,$\ell > 0$);*

(2) v_1 と u_1 の両方に x の定義が存在する．*;*

(3) 路の終点 v が一致している．*($v_k = v_\ell = v$);*

(4) 路の途中で交わることがない．*($\{v_1, \cdots, v_{k-1}\} \cap \{u_1, \cdots, v_{\ell-1}\} = \varnothing$)*

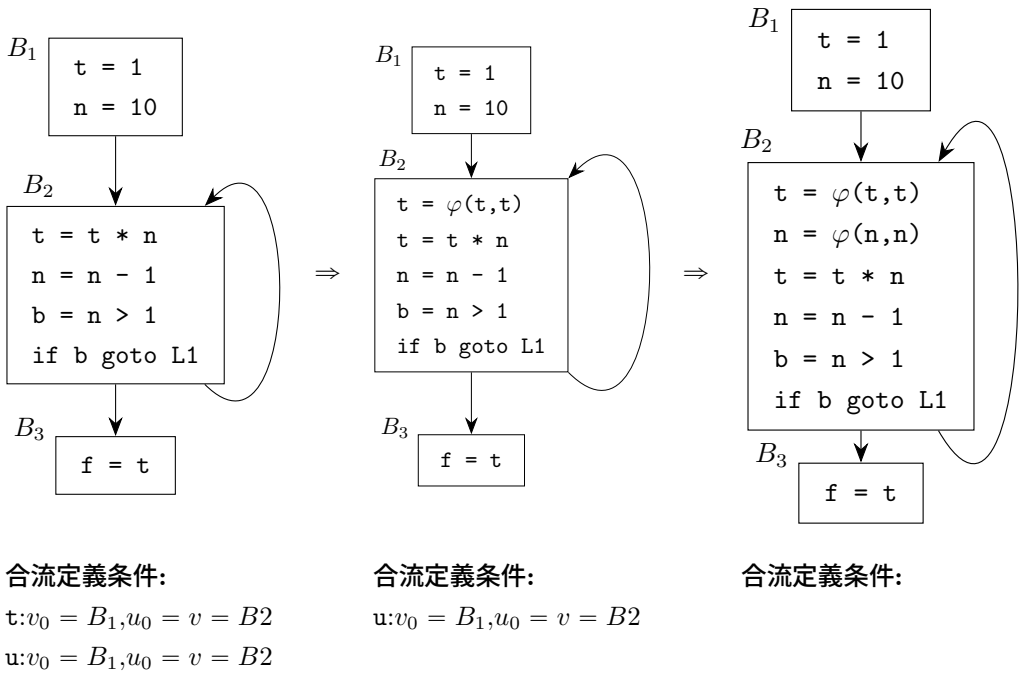


図 7.3: φ 関数の挿入

頂点に φ 関数が挿入されて，$x = \varphi(x, \cdots, x)$ が頂点 v に挿入されると，新たなコントロールフローグラフが得られるため，この更新されたグラフに対して繰り返して上記の条件をチェックする必要がある．図 7.1 の変数 t に関して，$v_0 = B_1$, $u_0 = B_2$, $v = B_2$ とすると $B_1 \to B_2$ と $B_2 \to B_2$ であるため，定義合流条件を満たす．B_2 に φ 関数 t=φ(t,t) が挿入される．n についても同様である．これらの挿入によって，新たなフローは生じないのでこれ以上 φ 関数を挿入する必要はない．定義 7.1 に沿って φ 関数を挿入するためにはコントロールフローグラフの路を網羅的に検査すればよい．

φ 関数の挿入

変数 x に対して定義 7.1 の条件を満たす v_0, u_0, v がフローグラフに存在し，頂点 v の基本ブロックに φ 関数が含まれていないならば，v に x=φ(x,...,x) を挿入する．
定義合流条件を満たす v_0, u_0, v がなくなるまで，上記を繰り返す．

フローグラフにおいて，条件の適用をチェックして φ 関数を挿入する方法をそのまま実装しようとすると，グラフの頂点の組み合わせ (v_0, u_0, v) を尽くして，頂点間の到達可能性と合流可能性を毎回チェックする必要があり，効率的とは言えない．コンパイラへの実装においては，有向グラフの支配関係を用いたより効率のよい挿入アルゴリズムが用いられる．

定義 7.2. フローグラフを (V, E, v_{init}) とする．頂点 v, u に対して，v_{init} から u へのすべての路において v が出現するとき，v は u の**支配頂点 (dominator)** であると呼び，$u \underline{\ll} v$ と書く．$u \neq v$ かつ $u \underline{\ll} v$ であるとき，$u \ll v$ と書く．u の支配頂点の集合 $\mathsf{dom}(u) = \{v \mid u \underline{\ll} v\}$ で定義する．

どの頂点 v についても $v_{init} \in \mathsf{dom}(v)$ かつ $v \in \mathsf{dom}(v)$ である．$\lessdot$ は半順序である．

- (反射律) $v \lessdot v$
- (反対称律) $v \lessdot u$ かつ $u \lessdot v$ ならば $u = v$
- (推移律) $v \lessdot u$ かつ $u \lessdot w$ ならば $v \lessdot w$

$u \neq v_{init}$ のとき，$\mathsf{dom}(u) \backslash \{u\}$ には $\lessdot$ の意味で最小の頂点が一つ存在する．もし複数の頂点 $v_1, v_2 \in \mathsf{dom}(u) \backslash \{u\}$ が存在したとすると，v_1, v_2 はともに v_{init} から u のすべての路に出現することになるため，$v_1 \lessdot v_2$, $v_2 \lessdot v_1$ となるので，反対称律から $v_1 = v_2$ でなければならない．v_{init} から v_{init} に長さ 0 の路があるので，$\mathsf{dom}(v_{init}) = \{v_{init}\}$ であり，$\mathsf{dom}(v_{init}) \backslash \{v_{init}\} = \varnothing$ なので以上のような最小頂点は存在しない．$u \neq v_{init}$ のとき，$\mathsf{dom}(u) \backslash \{u\}$ の中の最小頂点を $\mathsf{idom}(u)$ と書くことにする．このとき，$\mathsf{idom}(v) \to \cdots \to v$ である．

フローグラフ $G = (V, E, v_{init})$ に対して，u に対する親ノードを $\mathsf{idom}(u)$ とする辺 $D = \{(\mathsf{idom}(v), v)\}$ から構成される有向グラフ $T = (V, D)$ は，v_{init} から V の各頂点への路が存在することから，v_{init} を根とする木構造が構成される[1]．この木構造 T を G に対する**支配木 (dominator tree)** という．

例 7.13. 図 7.4 に支配木の構成例を示す．$v_{init} = v_1$ とすると v_7 については，$v_1 \to v_2 \to v_3 \to v_5 \to v_7$, $v_1 \to v_2 \to v_3 \to v_6 \to v_6$ の 2 つの路があるので，$\mathsf{idom}(v_7) = v_3$ となる．

頂点 v, u に対して，$u \neq v$ かつ $u \lessdot v$ であるとき，v は u を**真に支配する**といい，$u \ll v$ と書くことにする．

定義 7.3. フローグラフ (V, E, v_{init}) が与えられたとき，$v \in V$ の**支配辺境 (dominance frontier)** $DF(v)$ を以下のように定義する．

$$DF(v) = \{u \in V \mid \text{適当な } w \text{ が存在して } w \lessdot v \text{ かつ } u \not\ll v \text{ かつ } w \to u\}$$

支配辺境 $DF(v)$ は，直前の頂点までは v に支配されているが，v の支配が及ばない頂点の集合である．v において変数 x の定義があれば，$DF(v)$ の頂点において x の φ 関数を挿入する必要がある．φ 関数は新たな定義を生むため，新たな φ 関数の挿入が生じないようになるまで，各 $DF(v)$ に φ 関数を挿入する必要が生じる．ここで，支配木構造に着目すれば，各頂点に対して 1 回だけ φ 関数の挿入をチェックすればよいことが示される．

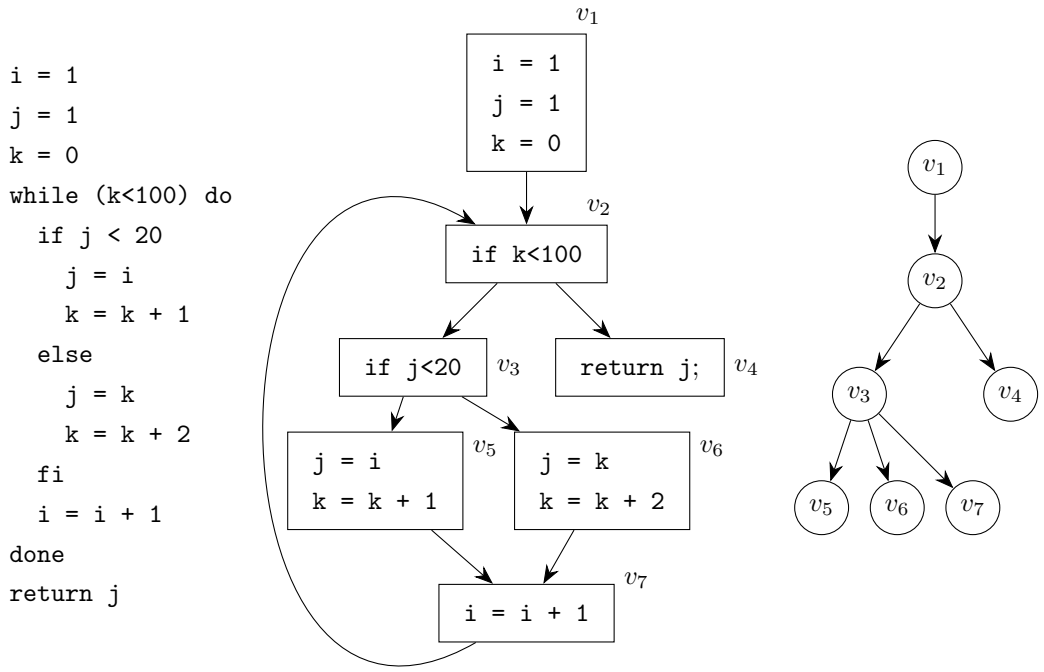
補題 7.1.
- $DF_{local}(v) = \{u \mid v \to u, \mathsf{idom}(u) \neq v\}$
- $DF_{up}(v) = \{u \in DF(v) \mid u \not\ll \mathsf{idom}(v)\}$

とするとき，

$$DF(v) = DF_{local}(v) \cup \bigcup_{w \in \mathsf{Child}(v)} DF_{up}(w)$$

ここで，$\mathsf{Child}(v) = \{w \mid \mathsf{idom}(w) = v\}$

この補題の証明は本章の最後に付録として示す．補題 7.1 に沿って $DF_{up}(v)$ を計算しようとすると，$\mathsf{idom}(w) = v$ であるような w の $DF(w)$ を計算しておく必要がある．idom が支配木を構成



(a) プログラム (b) フローグラフ (c) 支配木

図 7.4: 支配木の例

することに注目すれば，支配木を深さ優先で探索して，各頂点 v に対する $DF(v)$ を計算できることがわかる．(アルゴリズム 1)

ここで，$\mathsf{succ}[n] = \{v \mid n \to v\}$ である．

例 7.14. 図 *7.4* のフローグラフに対して，各頂点の支配辺境を次のように求められる．まず支配木の葉頂点として $\{v_5, v_6, v_7\}$ の DF_{local} を計算する．支配木の葉頂点においては，$\mathsf{Child}(v) = \varnothing$ であり，$DF_{local}(v_4) = DF(v_4) = \varnothing$,$DF_{local}(v_5) = DF(v_5) = \{v_7\}$, $DF_{local}(v_6) = DF(v_7) = \{v_7\}$, $DF_{local}(v_7) = DF(v_2) = \{v_2\}$.

$DF(v_3) = DF_{local}(v_3) \cup DF_{up}(v_5) \cup DF_{up}(v_6) \cup DF_{up}(v_7)$.

$DF_{up}(v_5) = DF_{up}(v_6) = \{u \in DF(v_5) | u \not\ll v_3)\} = \varnothing$, $DF_{up}(v_7) = \{u \in DF(v_7) | u \not\ll v_3\} = \{v_2\}$ $DF_{local}(v_3) = \varnothing$ であり，$DF(v_3) = DF_{up}(v_5) \cup DF_{up}(v_6) \cup DF_{up}(v_7) = \{v_2\}$.

$DF_{local}(v_2) = \varnothing$, $DF(v_2) = DF_{up}(v3) \cup DF_{up}(v4) = \{v_2\}$ $DF_{up}(v_2) = \{u \in DF(V_2) | u \not\ll v_1\} = \varnothing$

$DF_{local}(v_1) = \varnothing$ より $DF(v_1) = DF_{up}(v_2) = \varnothing$ となる．

[1] v_{init} は idom をもたないことから，親となる頂点がない

Algorithm 1 支配辺境の計算

```
procedure COMPUTEDF(n)
    S ← {}
    for all y ∈ succ[n] do                          ▷ DF_local の計算
        if idom(y) ≠ n then
            S ← S ∪ {y}
        end if
    end for
    for all c ∈ Child(n) do                         ▷ DF_up の計算
        computeDF(c)                    ▷ 支配木の子頂点から計算する
        for all w ∈ DF[c] do
            if w ∉ dom(n) or n = w then
                S ← S ∪ {w}
            end if
        end for
    end for
    DF[n] ← S
end procedure
```

v	DF_{local}	DF_{up}	DF
v_1	$\varnothing$	$\varnothing$	$\varnothing$
v_2	$\varnothing$	$\varnothing$	$\{v_2\}$
v_3	$\varnothing$	$\varnothing$	$\{v_2\}$
v_4	$\varnothing$	$\varnothing$	$\varnothing$
v_5	$\{v_7\}$	$\varnothing$	$\{v_7\}$
v_6	$\{v_7\}$	$\varnothing$	$\{v_7\}$
v_7	$\{v_2\}$	$\{v_2\}$	$\{v_2\}$

変数 x の φ 関数は頂点 v に挿入される条件は定義合流条件による．変数の定義を含む頂点の集合を S とすると，定義 7.1 は次のようになる．

定義合流条件 (S)

$u_0, w_0 \in S$ に対して，長さ 1 以上で途中で交わらずに v で合流する路 $p{:}u_0 \to \cdots \to u_k = v$, および $q{:}w_0 \to \cdots \to w_\ell = v$ が存在する．$(\{u_0, \cdots, u_k\} \cap \{w_0, \cdots, w_\ell\} = \varnothing)$

S に対してこの条件を満たす合流頂点 v の集合を $J(S)$ とする．

変数 x の定義をもつ頂点集合を S_0^{x} とする．$J(S_0^{\mathrm{x}})$ は，x に対する φ 関数を最初に挿入する頂点である．$J(S_0^{\mathrm{x}}) = S_1^{\mathrm{x}}$ では φ 関数の挿入は x の定義を生むので，$J(S_0^{\mathrm{x}} \cup S_1^{\mathrm{x}})$ が x の定義を含む頂点の集合となる．

$S_{i+1}^x = J(S_0^x \cup S_i^x)$ とすると $S_0^x, S_1^x, S_2^x, \cdots$ により変数 x の φ 関数を挿入する頂点を数え上げる

ことができる．$S_i^x \subseteq S_{i+1}^x$ であり，フローグラフの頂点は有限個であることから，i の増加によって変化しなくなるときの集合を $J^+(S_0^x)$ とする．$J^+(S_0^x)$ は DF によって求めることができる．

定理 7.1. 頂点の集合 S_0 に対して $DF(S_0) = \bigcup_{v \in S_0} DF(v)$ とする．F_i の系列を以下のように定義する．

$$\begin{aligned} F_0 &= \varnothing \\ F_{i+1} &= DF(S_0 \cup F_i) \end{aligned}$$

$DF(S_0 \cup F_i) = DF(S_0) \cup DF(F_i)$ であることから F_i の系列は単調に増加する．この系列が変化しなくなるまで i を増加させて得られる頂点集合を $DF^+(S_0)$ とする．

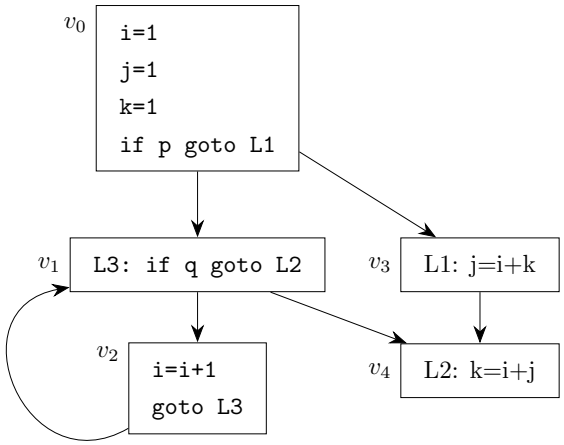
このとき $DF^+(S_0) = J^+(S_0)$ である．

この定理の証明については，文献 [?] を参照.

例 7.15. 以下に φ 関数の挿入の例を示す.

```
     i=1
     j=1
     k=1
     if p goto L1
L3:  if q goto L2
     i=i+1
     goto L3
L1:  j=i+k
L2:  k=i+j
```

(a) プログラム



(b) フローグラフ

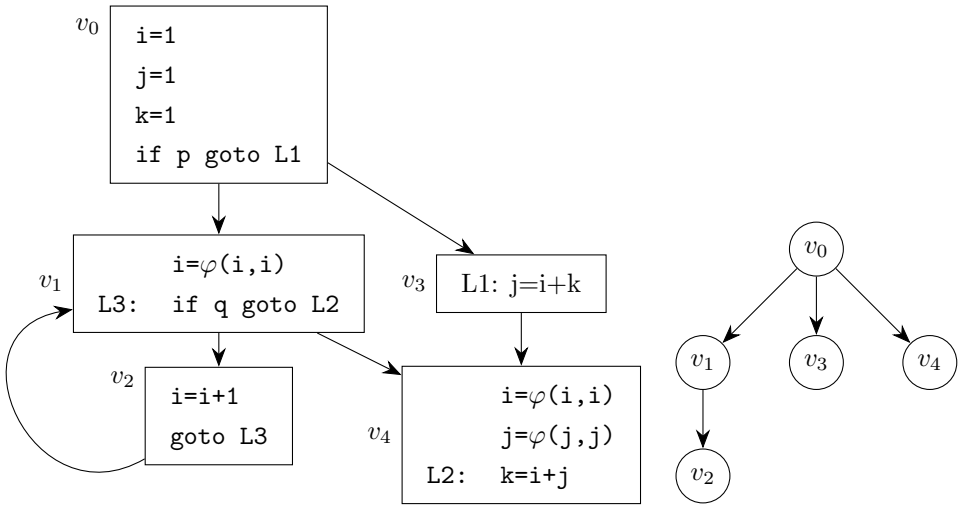
v	$DF(v)$
v_0	$\varnothing$
v_1	$\{v_1, v_4\}$
v_2	$\{v_1\}$
v_3	$\{v_4\}$
v_4	$\varnothing$

(c) 支配辺境

各変数の合流条件は以下のように計算される.

S_0	$S_0^i = \{v_0, v_2\}$ $DF(S_0^i) = \{v_1\}$	$S_0^j = \{v_0, v_3\}$ $DF(S_0^j) = \{v4\}$	$S_0^k = \{v_0, v_4\}$ $DF(S_0^k) = \varnothing$
S_1	$DF(S_0^i \cup \{v_1\}) = \{v_1, v_4\}$ $DF(S_0 \cup \{v_1, v_4\}) = \{v_1, v_4\}$	$DF(S_0^j \cup \{v_4\}) = \{v4\}$	$DF(S_0^k) = \varnothing$
	$DF^+(S_0^i) = \{v_1, v_4\}$	$DF^+(S_0^j) = \{v_4\}$	$DF^+(S_0^k) = \varnothing$

したがって，v_1, v_4 の先頭に i の φ 関数，v_4 の先頭に j の φ 関数を挿入する.



φ 関数の挿入　　支配木

7.4.3　SSA 形式への変換

φ 関数を挿入した後，各変数に添字を付加して各 3 番地命令を SSA 形式に変換する．フローグラフを支配木によってトラバースすることによってフローに従って字を付加する．

各変数 x について，まず左辺で定義する変数に添字をつけ，新たに定義されるまで右辺で参照する変数にその添字をつける．アルゴリズムをアルゴリズム 2 に示す．

例 7.15 に対してアルゴリズムを適用する．

初期化: 各変数 i, j, k に対して，カウンタを 0 にセットして，スタック Stack[x] に 0 をプッシュする．

Step1: Count[i] = Count[j] = Count[k] = 1 とし，RENAME(v_0) において，9 行目から 13 行目で，i, j, k の左辺に添字 1 をつける．
v_0 の後続頂点のうち，v_1 に対して，16 行目から 20 行目を適用する．v_0 を v_1 の 1 番目の先行頂点とする．($j = 1$)
i = 1 であるので，v_1 の i=φ(i, i) を i=φ($\mathtt{i}_1$, i) におきかえる．v_3 には φ 関数が含まれない．
v_0 の支配木の子頂点 v_1 を選んで RENAME(v_1) を呼び出す．

Step2: v_1 において，φ 関数以外で参照される変数はなく，i が定義されるので，10 行目から 13 行目のブロックにおいて，Count[i] をインクリメントして，$\mathtt{i}_2$=φ($\mathtt{i}_1$, i) とする．
15 行目から 22 行目までの実行で，v_4 において，φ 関数内の i と j の参照を置換して，i=φ(i, $\mathtt{i}_1$), j=φ($\mathtt{j}_1$, j) とする．(フローグラフでは v_4 は v_1 の 1 番目の後続頂点である．)
24 行目で，v_1 の支配木における子頂点は v_2 なので，RENAME(v_2) を呼び出す．

Step3: v_3 において，4 行目から 7 行目の適用によって，i=$\mathtt{i}_2$+1 となる．つぎに，v_3 では，i が定義されているので，Count[i] を増やして，$\mathtt{i}_3$=$\mathtt{i}_2$+1 とする．この時点で，Count[i] = 3,Stack[i] = [3, 2, 1, 0]

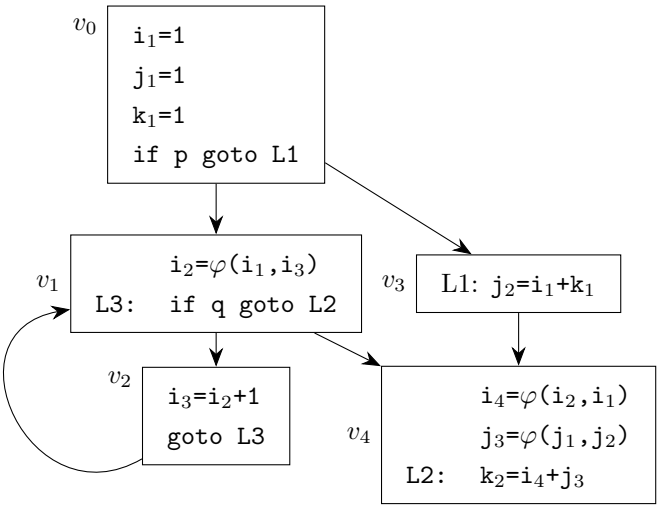


図 7.5: 例 7.15 に対するアルゴリズム 2 の適用結果

v_2 は支配木の子頂点を持たないので，26 行目から 30 行目のループが実行される．v_2 で定義された変数は i なので，ポップし，Stack[i] $= [2, 1, 0]$ とする．

Step2': Step2 の RENAME(v_2) が終了したので，その後の 26 行目から 30 行目を実行する．v_2 で i が φ 関数で定義されたので Stack[i] をポップして，Stack[i] $= [1, 0]$ とする．

Step1 の 23 行目で，支配木の次の子頂点 v_3 を選んで RENAME(v_3) を呼び出す.

Step4: 4 行目から 7 行目のループで Stack[i] = Stack[j] $= [1, 0]$ であることから，$\mathtt{j}=\mathtt{i}_1+\mathtt{j}_1$ に置換する．9 行目から 14 行目のループで，j が左辺で定義されているので，Count[j] をインクリメントし，Stack[j] $= [2, 1, 0]$ としたあと，$\mathtt{j}_2=\mathtt{i}_1+\mathtt{k}_1$ とする．15 行目から 22 行目のループにおいて，v_4 は v_3 の後続頂点であり，v_3 は v_4 にとって 2 番目の先行頂点であるので，v_4 の φ 関数の 2 番目の引数を置き換えて，$\mathtt{i}=\varphi(\mathtt{i}_2, \mathtt{i}_1)$, $\mathtt{j}=\varphi(\mathtt{j}_1, \mathtt{j}_2)$ とする．
v_3 には支配木の子頂点はなく，j を定義したので，Stack[j] をポップして，$[1, 0]$ とする．この時点で，Count[i] $= 3$, Count[j] $= 2$ である．

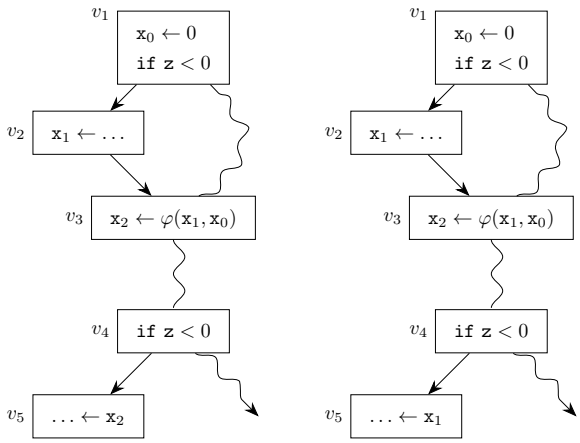
Step1": 23 行目で 3 番めの子頂点として v_4 を選択して，RENAME(v_4) を呼び出す.

Step5: 2 行目から 15 行目までのループで順に置換する．
$\mathtt{i}_4=\varphi(\mathtt{i}_2, \mathtt{i}_1)$ ここで，Stack[i] $= [4, 1, 0]$, Count[i] $= 4$
$\mathtt{j}_3=\varphi(\mathtt{j}_1, \mathtt{j}_2)$ ここで，Stack[j] $= [3, 1, 0]$, Count[j] $= 3$
最後に k が新たに定義されるので，$\mathtt{k}_2=\mathtt{i}_4+\mathtt{j}_3$ と置換される．

図 7.5 に適用結果を示す.

7.4.4 SSA 上での最適化

SSA 形式に変換することによって，手続き内において基本ブロックにとらわれない最適化が可能になる．



(a) 最適化前　　(b) 最適化後

図 7.6: 支配関係を壊す変換

不要コード除去　SSA 形式における不要コード除去は，変数名によってどの定義によって変数が三 sh 要されているかがわかる．左辺で定義されている変数が右辺で参照されていない（あるいは何等かの副作用を及ぼさない）場合は，その定義を除去しても意味は変わらない．

定数伝播　SSA 形式であれば，同じ変数名であるとき同じ値を持つ．$x_i \leftarrow c$ という定数 c の代入があれば，x_i は c に置き換えることができる．もし，$x_k = \varphi(x_1, \cdots, x_n)$ において，$x_1, \cdots, x_n$ がすべて同じ定数に置き換えられた場合は，x_k もその定数に置き換えて伝播させることができる．

同じ手続きの中 (intra-procedure) であれば，コピー伝播，ループ変数の畳み込み，共通部分式除去などの方法を基本ブロックを意識せずに適用することができる．手続きにまたがる (inter-procedure) 最適化はさらに解析が必要となる．

支配関係の保存　SSA による最適化においては，変換前と変換後がそれぞれ支配関係を保存することを要求する[2][?]．図**??**のようなフローグラフを (a) から (b) に変換することを考える．v_1 と v_3,v_3 と v_4 の間に $\mathtt{z}$ が更新されることがないとすると，v_1 と v_4 の分岐条件は変わらないので，(b) のようにしても意味は変わらない．v_5 の $\mathtt{x}$ は (a) では，v_3 で定義された値，(b) では，v_1 で定義された値である．(b) では，v_1 は v_5 を支配しない．

このような最適化は v_1 と v_4 における $\mathtt{z} < 0$ の評価が変わらないことが保証されれば正しい．これを保証するためには，実行フローが同じであることを証明しなければならず，このような解析は一般には決定可能でない．

SSA 形式では，変数の代入による定義が唯一であるという条件の他に,「変数定義が変数使用を支配する」という条件も成り立っている必要がある．SSA 形式による最適化手法では，この条件によってフローグラフの構造から可能な最適化を行い，すべての最適化を可能とするものではない．

[2]前後で同じ支配関係である必要はない．

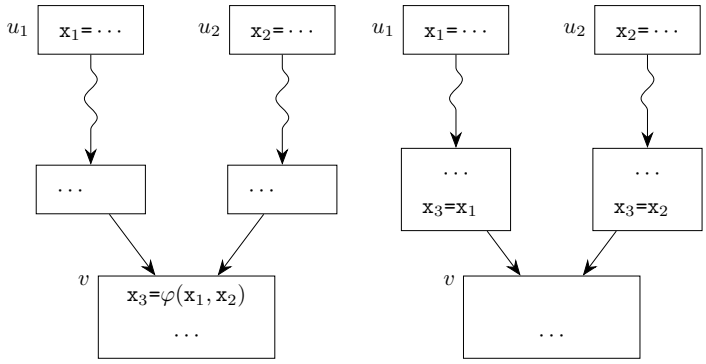


図 7.7: SSA 形式からの φ 関数の消去

7.4.5 φ 関数を持たない SSA 形式への変換

φ 関数は実行可能な関数ではないので，コンパイルをすすめるためには，φ 関数を消去した等価なプログラムに変換する必要がある．φ 関数は頂点 v の基本ブロックの先頭に挿入される．$x_3 = \varphi(x_1, x_2)$ が先頭に挿入されている場合，x_1，x_2 に対応するフローグラフ上の路において v の直前の頂点に $x_3 = x_1$, $x_3 = x_2$ を挿入すれば，φ 関数を消去しても意味は変わらない．(図 7.7) 各路の v への先行頂点 u が v を唯一の後続頂点として持つときは，新たに頂点は挿入せず，u の最後に $x_3 = x_1$ ないしは $x_3 = x_2$ を追加する (図 7.7)．u の最後が goto である場合は，goto 文の前に挿入する．唯一の先行頂点を持つ場合には，さらにその前に挿入する[3]．

以上の方法で φ 関数を消去するには，φ 関数を含む頂点 v の先行頂点が v を唯一の後続頂点として持つか，v が唯一の先行頂点を持つ必要がある．

φ 関数を含むブロックの頂点がこの条件を満たさない場合は，v を唯一の後続頂点とする空の頂点を挿入することで，この条件を満たすフローグラフを構成できる．このフローグラフに上記の方法を適用して φ 関数を消去可能である．

例えば図 7.5 のフローグラフでは，v_1 は φ 関数を含み，v_1 の先行頂点のうち v_0 は複数の後続頂点 v_1, v_3 を持ち，v_1 の先行頂点も複数存在する．従って，v_0 と v_1 の間に新たな頂点 w_0 を挿入することで条件を満たす．v_4 についても v_2 が複数の後続頂点を持つので，v_1 から v_4 の間に新たな頂点 w_1 を挿入することでこの条件を満たす．(図 7.8)

[付録] 補題 7.1 およびアルゴリズム 1 の証明

補題 7.1.

$$DF(v) = DF_{local}(v) \cup \bigcup_{w \in \mathsf{Child}(v)} DF_{up}(w)$$

証明.($\supseteq$) 定義 7.3 の定義式において，w として v をとると，$v \leqq v$ であるので，$DF_{local}(v) \subseteq DF(v)$ である．v の子頂点 w に対しては，$idom(w) = v$ である．v の支配木の子頂点 w に対して $DF_{up}(w) = \{u \mid \exists w' : w' \leqq u, u \not\leqslant w, w' \to u, u \not\leqslant \mathsf{idom}(w)\}$．$DF(v) = \{u \mid \exists w' : w' \leqq v, u \not\leqslant v, w' \to u\}$．こ

[3] φ 関数の挿入時には唯一の先行頂点を持つようなブロックは存在しない．最適化後には現れる可能性がある．

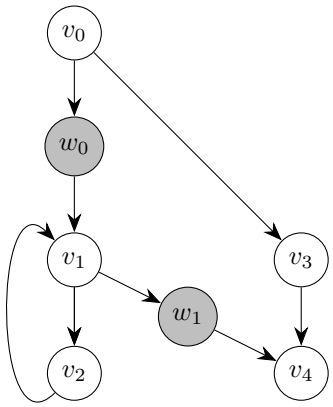


図 7.8: 挿入条件を満たすために空頂点を追加

こで，$w' \underline{\ll} w$, $w \underline{\ll} \mathsf{idom}(w)$ のとき，$w' \underline{\ll} \mathsf{idom}(w)$ なので，$v = \mathsf{idom}(w)$ ならば $DF_{up}(w) \subseteq DF(v)$. よって右辺の要素はすべて $DF(v)$ に含まれるから，$\supseteq$ が示される．

($\subseteq$) $x \in DF(v)$ となる任意の頂点 x に対して，$y \to x$ であり $y \underline{\ll} v$ となる y が存在する．$y = v$ ならば，$x \in DF_{local}(v)$ である．$y \neq v$ のときは，$y \ll v$ であるので，v の支配木における子頂点 v' が存在して，$y \underline{\ll} v'$ かつ $x \not\ll v'$. 定義から $x \in DF_{up}(v')$ である．

□

補題 7.2.

$$DF_{local}(v) = \{u \mid v \to u, \mathsf{idom}(u) \neq v\}$$

証明. $v \to u$ であるとき，$u \ll v \Leftrightarrow v = \mathsf{idom}(u)$ を証明する．

($\Leftarrow$) 定義から，v は u を真に支配するので，成立.

($\Rightarrow$) $u \ll v$ より，v の子頂点 v' で $u \underline{\ll} v'$ となる v' が存在する．ここで $v \to u$ であるので，$v' = u$ となる．従って，$\mathsf{idom}(u) = \mathsf{idom}(v') = v$.

$\Leftrightarrow$ の対偶をとると，上記が成り立つ. □

補題 7.3. 頂点 v と支配木における v の子頂点 w に対して

$$DF_{up}(w) = \{u \in DF(w) \mid \mathsf{idom}(u) \neq v\}$$

証明. v の (支配木における) 子頂点 w に対して $u \in DF(w)$ ならば $(u \ll v) \Leftrightarrow (\mathsf{idom}(u) = v)$ を証明する．

($\Leftarrow$) 支配関係の定義より明らか.

($\Rightarrow$) v が真に u を支配するならば，u を支配する v の子頂点 x が存在する．($x \underline{\ll} u$) u の先行頂点 y で，$y \underline{\ll} w$ となる y を選ぶ．すると，x は開始頂点から u への路のすべてに出現し，y もすべての路に出現する．すると x が u を支配するか，$u = x$ である．$u = x$ であれば，$\mathsf{idom}(x) = \mathsf{idom}(u) = v$ である．$u \neq x$ であれば，x が u を支配する．とすると u を支配できる v の子頂点は唯一である．(唯

一でなければ，x が u を支配することはできない．) つまり，$x = w$ となり，これは，$u \in DF(w)$ に反する．

□

定理 7.2. アルゴリズム *1* は $DF(v)$ を正しく計算する．

証明. 補題 7.2, 補題 7.3 より明らか．アルゴリズムの前半が補題 7.2，後半が補題 7.3 にそのまま当てはまる． □

[付録] 定理 7.1 の証明

補題 7.4. フローグラフにおいて，長さ 1 以上の路 p: $v \to \cdots \to u$ に対して，p 上に，頂点 $w \in \{v\} \cup DF^+(v)$ が存在する．p 上に v の支配頂点でない頂点があれば，$DF^+(v)$ に属する．

証明. (作成中) □

補題 7.5. フローグラフにおいて異なる頂点 u_1, u_2 を始点として v で合流する路 p:$u_1 \to \cdots \to v$, q:$u_2 \to \cdots \to v$ が存在するとき，$v \in DF^+(u_1) \cup DF^+(u_2)$ である．

証明. (作成中) □

補題 7.6. フローグラフの頂点集合 S に対して，$J(S) \subseteq \cup_{v \in S} DF^+(v)$

証明. (作成中) □

補題 7.7. S をフローグラフの始点を含む頂点集合とすると，$DF(S) \subseteq J(S)$

証明. (作成中) □

定理 7.1. 変数 x に対する代入を含む基本ブロックに対する頂点集合を S とする．このとき，$DF^+(S)$ は変数 x の φ 関数を挿入する頂点集合 $J^+(S)$ である．

証明. (作成中) □

Algorithm 2 SSA への変換

```
/* 初期化 */
for all a ∈ Var do
    Count[a] ← 0
    Stack[a] ← ∅
    Stack[a] に 0 をプッシュ
end for
```

```
1:  procedure RENAME(n)
2:      for all S in ブロック n do
3:          if S が φ 関数でないとき then
4:              for all x in S で参照される変数 do
5:                  i ← top(Stack[x])
6:                  S の x の参照を x_i で置き換える
7:              end for
8:          end if
9:          for all S の左辺で定義される変数 a do
10:             Count[a] ← Count[a] + 1
11:             i ← Count[a]
12:             Stack[a] に i を push
13:             S の左辺の a を a_i で置き換える
14:         end for
15:     end for
16:     for all Y in ブロック n の後続頂点 do
17:         n は Y の j 番目の先行頂点
18:         for all  doY に含まれる φ 関数
19:             φ 関数の j 番目の引数が a のとき
20:                 i ← top(Stack[a])
21:             φ 関数の j 番目の引数の a を a_i に置換
22:         end for
23:     end for
24:     for all X in 支配木における n の子頂点 do
25:         call RENAME(X)
26:     end for
27:     for all ブロック n の S do
28:         for all  doa in S における定義
29:             Stack[a] をポップ
30:         end for
31:     end for
32: end procedure
```

第8章　目的コード生成

３番地コードとして生成された命令列は３番地コードを解釈する実行系で実行可能である．３番地コードは計算機のアーキテクチャからは独立した命令であり，変数はメモリ上にあるとして実行される．コンパイラにおける目的コード生成は，中間コードに対応する目的計算機の命令語を選択する．ここで，現在の計算機の多くのアーキテクチャでは，メモリにあるデータを演算器のレジスタにロードして演算を行い，レジスタに得られた結果をメモリに書き戻す動作をする．演算器のレジスタとメモリの間のデータの交換速度は一般に計算機の演算に比べて遅いため，プログラムの解析によってレジスタとメモリの間のデータ交換ができるだけ少なくなるような目的コードを生成することが望ましい．中間コードの変数の数に比べてレジスタの数は非常に少ないため，実行プログラムにおいてその時点で頻繁に演算に用いられる変数をレジスタに置くようなコードを生成することによって実行速度の大幅な改善が可能になる．変数とレジスタの対応を決定することをレジスタ割当という．

一般にプログラムに対して最適なレジスタ割当を行うことは難しく，３番地コードのフローグラフが与えられている時に割当アルゴリズムがNP完全な問題である [?] ため，目的コード生成でのレジスタ割当は最適性を保証することは困難である．従って，目的コード生成では，部分的なプログラムに対して必要なレジスタ数を見積もり，近似的に効率のよい割当手法を与える．具体的には，３番地コードやSSA形式における変数のデータフローからライフサイクル（生存期間）を解析して，変数からレジスタへの割当を決定する．

目的コードの命令の選択については目的マシンのアーキテクチャに応じてできるだけサイクル数の少ない命令（軽い命令）を選択するために命令のコストを見積もるなどの手法がある．しかし，実際の計算機ではハードウェアによる並列的な実行などの効果や実行ユニットの数などに依存することから，実際的な計算機に対するコード生成の系統的なモデル化は教科書的内容の範囲を越えるため，本書では扱わない．

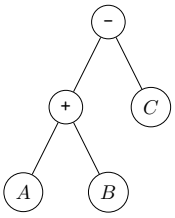
8.1　算術式のコード生成

ここでは，算術式に対するコード生成において必要なレジスタの数を見積もる手法を示す．簡単化のため，以下の範囲に限定する．

- 演算子は四則演算 $(+, -, *, /)$ に限定する．
- 命令はメモリ転送命令 $(\texttt{Load}\ r, e)$ と演算命令 $(\theta\ r, e)$ とする．
  メモリ転送命令 $\texttt{Load}\ r, e$ は，e で示されるメモリの内容をレジスタ r に転送する．演算命令 $\theta\ r, e$ は，e で示されるメモリ（またはレジスタ）の内容をフェッチして r と $\theta \in \{+, -, *, /\}$ で定められた演算を行い，結果を r に格納する．

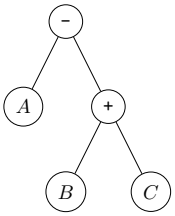
例えば，算術式 $A+B-C$ のコードは以下のようになる．

```
Load  r1  A
+     r1  B
-     r1  C
```



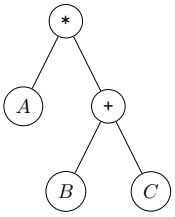
以上の制約のもとでは，レジスタは 1 個で計算が可能である．これに対して，$A-(B+C)$ の場合は以下のような命令系列になる．

```
Load  r2  B
+     r2  C
Load  r1  A
-     r1  r2
```



この算術式の計算に必要なレジスタ数は 2 である．$A*(B+C)$ については，

```
Load  r2  B
+     r2  C
*     r2  A
```



とすればよいので必要なレジスタは 1 つである．これは，$-$ が可換でなく，$*$ が可換であるため 2 番めの項をまず評価して 1 番目の項を適用することができるためである．算術式を表す抽象構文木 T に対して，その計算に必要なレジスタ数を $RN(T)$ と書く．$RN(T)$ はアルゴリズム 3 で求められる．

このレジスタ数の見積もり解析に応じて，使用するレジスタが実際に利用できるレジスタ数の範囲内であれば，レジスタを利用する目的コードを生成できる．しかし，レジスタ数が足りない場合は，レジスタ溢れ (spill) が発生する．このような場合は，フロー解析などの手法を使って最も後で使われる変数をレジスタに割り当てるなどの方法によって効率的に実行する目的コードを生成する．

8.2　グラフ彩色によるレジスタ割当

3 番地コードの基本ブロックに対して計算に用いられる変数をできる限り多くレジスタに割り当てる方法について紹介する．基本的なアイデアとして変数の生存期間，すなわち，変数が定義された時点からその定義最後に使われるまでの期間，が重ならない 2 つの変数は同じレジスタに割り当てることができるという性質を利用する．

Algorithm 3 $RN(T)$ の計算

```
procedure RN(T)
    if isleaf(T) thenx return 0                                  ▷ T が葉
    else
        T_1 = Left(T)                                            ▷ T_1 は左部分木
        T_2 = Right(T)                                           ▷ T_2 は右部分木
        if RN(T_1) = 0 かつ RN(T_2) = 0 then return 1    ▷ 左右とも葉なのでレジスタ 1 つ
        else
            if RN(T_1) = 0 かつ RN(T_2) > 0 then
                if root(T) が可換演算子 then return RN(T_2)          ▷ 後ろから適用する
                else return max(2, RN(T_2))                      ▷ 最大 2 つ必要
                end if
            else
                if RN(T_1) > 0 then
                    if RN(T_1) = RN(T_2) then return RN(T_1) + 1
                    else return max(RN(T_1), RN(T_2))
                    end if
                end if
            end if
        end if
    end if
end procedure
```

8.2.1　レジスタ干渉グラフ

レジスタ干渉グラフは，生存期間の重なりを無向グラフによって表現したものである．無向グラフの頂点に変数を割り当ててその生存期間が重なる場合には，その頂点間に辺をつける．互いに生存期間が重ならない変数については対応する頂点は辺を持たない．従って，$(u, v) \notin E$ のとき，u に割り当てられた変数と v に割り当てられた変数は同じレジスタに割り当てることができる．

例 8.1.

無向グラフに対する k-彩色問題とは，辺の両端にある頂点に違う色を割り当てるときに k 種類の色によって塗り分けが可能かどうかを判定する問題である．

定義 8.1. 無向グラフ (V, E) に対する k-彩色問題とは，すべての辺 $(u, v) \in E$ にたいして，$f(u) \neq f(v)$ となる関数 $f{:}V \to \{1, \ldots, k\}$ が存在するとき，*YES,* 存在しないとき *NO* と答える問題である．

レジスタ干渉グラフを構成すると k 彩色問題で YES となる関数 f は，レジスタ割り当てそのものである．グラフの彩色問題は NP 完全な問題であり，一般に最適なレジスタ割り当てを求めることはコンパイラの構成として実際的ではない．

実際的なレジスタ割り当てを行うために以下の定理を利用することができる．

定理 8.1. 無向グラフ (V, E) の任意の頂点の次数が $k - 1$ 以下ならば k-彩色可能である．ここで，$k \geqslant 1$ とする．

この定理を利用してレジスタ割り当てアルゴリズム**??**が構成できる．

このアルゴリズムにおいて，定理 8.1 の条件が成り立たない時，要素数最大の独立集合を求める必要がある．要素数最大の独立集合を求める問題は NP 完全問題であり，このアルゴリズムをそのまま実現することは実用的とは言えない．

8.2.2　SSA に対するレジスタ割当

φ 関数を消去して得られる SSA 形式からは，比較的容易にレジスタ割り当てを導くことが知られている．SSA 形式では，最適化後において支配関係が保存されている．従って，各変数の生存期間は定義において新たな添字をもつ変数が定義されてから，次の添字を持つ変数が定義されるまでの間となる．各変数に対して生存期間に対応する頂点を抽出し，その頂点から構成される支配木の部分木を構成する．その部分木の交差グラフを構成するとレジスタ干渉グラフが構成できる．

補題 8.1. *SSA* 形式から支配木の部分木から構成された交差グラフによる干渉グラフは弦グラフである．

弦グラフの頂点に対しては，完全消去順序が存在する．完全消去順序の逆順で彩色をすると最小の数の色でグラフを彩色することができる．SSA 形式による干渉グラフの頂点数を n, 辺数を m とするとグラフ構築後の最適彩色は $O(m + n)$ で構成できる．

ここでは，SSA から構成される干渉グラフの大きさに対して線形時間で最小の色数で彩色できる．ただし，最初から色数を指定して彩色する問題ではないため，レジスタ溢れについての問題を

解いたわけではないことに注意する．SSA 形式からのレジスタ割り当ては，生存解析が容易になることによって効率的で最適な割り当て手法を与える．レジスタ数が少ない場合のレジスタ溢れといった問題は，依然，計算量の大きい問題として残ることに注意しよう．

付 録A　VMコードの仕様

6 章にあげたスタックマシンの目的コードを拡張して，手続き呼び出しを可能にする．このために 10~12 の 3 つの命令を追加する．

命令長は 1 word であり、命令は次の形式を持つ。

OPCODE	b	i	a

b ベースレジスタの指定。

i インデックスレジスタの指定。

a アドレス（displacement)。命令により定数や演算の指定にも使う。

命令領域（読出し専用）は word 単位で番地を持ち、0 番地から実行する。PC が示す番地の命令を IR に格納し、PC を 1 増加させてから、IR の命令を実行する。初期状態は次のとおりである。

$$\mathrm{PC} = 0, \quad R_1 = 0, \quad R_2 = -1, \quad R_3 = 0, \quad R_4 = 0.$$

1. Load literal

`lit`	b	i	a

```
R2 = R2 + 1; s[R2] = a
```

b と i は無視される。 原文の`'a'` は即値 a を表す。

2. Load

`lod`	b	i	a

```
R2 = R2 + 1; s[R2] = s[Rb + Ri + a]
```

b と i が指定するレジスタの内容と a を加えたものをオペランドアドレスとする。ただし、$b = 0$ のときベースによるアドレス修飾は行わず、$i = 0$ のときも同様である。

3. Store

`sto`	b	i	a

```
s[Rb + Ri + a] = s[R2]; R2 = R2 - 1
```

アドレスの計算は `lod` と同様であり、$b = 0$ または $i = 0$ の項は加えない。

4. Move to index register

`mvx`	b	i	a

```
Ri = s[R2]; R2 = R2 - 1
```

b と a は無視される。

5. Arithmetic operation

opr	b	i	a

a により次の 11 種類の演算を指定する。
b と i は無視される。

a	動作
0	`s[R2] = -s[R2]`
1	`R2 = R2 - 1; s[R2] = s[R2] + s[R2+1]`
2	`R2 = R2 - 1; s[R2] = s[R2] - s[R2+1]`
3	`R2 = R2 - 1; s[R2] = s[R2] * s[R2+1]`
4	`R2 = R2 - 1; s[R2] = s[R2] div s[R2+1]`
5	`R2 = R2 - 1;` $s[R_2] = s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
6	`R2 = R2 - 1;` $s[R_2] \neq s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
7	`R2 = R2 - 1;` $s[R_2] < s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
8	`R2 = R2 - 1;` $s[R_2] \leqslant s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
9	`R2 = R2 - 1;` $s[R_2] > s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。
10	`R2 = R2 - 1;` $s[R_2] \geqslant s[R_2+1]$ なら `s[R2] = 1`、そうでなければ `s[R2] = 0`。

6. Unconditional Jump

jmp	b	i	a

```
PC = a
```

b と i は無視される。

7. Jump on condition

jpc	b	i	a

```
if s[R2] = 0 then PC = a;
R2 = R2 - 1
```

b と i は無視される。

8. Read data

get	b	i	a

`R2 = R2 + 1` として、標準入力から次のデータ（整数値）を読み、`s[R2]` に格納する。
b、i、a は無視される。

9. Write data

put	b	i	a

`s[R2]` の内容を標準出力に出力し、`R2 = R2 - 1` とする。
b、i、a は無視される。

10. Increment stack top

int	b	i	a

```
R2 = R2 + a
```

b と i は無視される。

11. Procedure call

cal	b	i	a

```
s[R2+1] = R1; s[R2+2] = PC;
s[R2+3] = R3; s[R2+4] = R4;
R1 = R2 + 5; R2 = R2 + 4; PC = a
```

b と i は無視される。

12. Return

rtn	b	i	a

```
R2 = R1 - 5; PC = s[R1-3];
R3 = s[R1-2]; R4 = s[R1-1]; R1 = s[R1-4]
```

b、i、a は無視される。